MSc Artificial Intelligence
Master Thesis

Reducing And-Inverter Graphs for Scalable Optimizability Prediction

by
Isabella V. Gardner
314159265

2026/01/05–2026/08/31
36 ECTS

Supervisor: Michael Cochez

Examiner: Pascal Mettes

Universiteit van Amsterdam
Instituut voor Informatica

Reducing And-Inverter Graphs for Scalable Optimizability Prediction

Abstract

Problem
Placeholder.

Gap
Placeholder.

Approach
Placeholder.

Findings
Placeholder.

Keywords: Placeholder.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

[TODO: missing nice intro segue.]

1.1 Motivation

1.1.1 Logic Synthesis and Optimization Script Design

Logic synthesis is a critical phase in the Electronic Design Automation (EDA) flow, relying heavily on structural representations like And-Inverter Graphs (AIGs), a special form of Directed Acyclic Graph (DAG), to optimize circuits for area, power, and delay. However, crafting an optimal sequence of synthesis algorithms (a "script" or "pipeline") for a given circuit is a highly expensive, heuristic-driven trial-and-error process.

1.1.2 Machine Learning for Optimizability Prediction

Structural representations such as AIGs provide a foundation for applying ML to address a multitude of complex challenges in logic synthesis. With ML, one could predict a circuit's "optimizability" if subjected to certain optimization algorithms, therefore circumventing the trial-and-error phase by facilitating data-driven script generation. However, industrial-sized AIGs scale to millions or billions of nodes. Training advanced GNNs directly on these massive structures exceeds the memory available on a single device and drives training times beyond what is practical, preventing the field from exploring these applications and from scaling to more capable models.

1.1.3 The Scale Bottleneck

Consequently, to remain within available memory and keep execution tractable, researchers must restrict model size and capacity, accepting weaker architectures, or else rely on large multi-device clusters and distributed training. Even then, individual AIGs can exceed the memory of a single device, which forces the graph itself to be partitioned.

1.1.4 Limitations of Generic Graph Reduction

While generic graph reduction techniques (partitioning, sparsification, and summarization) are widely used on other graphs, standard methods could potentially degrade predictive (ML) performance on logic graphs because they blindly sever strict causal cones, obscure logical depth, and break the longest paths, therefore altering the underlying logic function and destroying the structural features necessary for accurate prediction. This necessitates the development of domain-aware reduction techniques.

1.1.5 Problem Statement and Research Gap

[TODO: review at end] The field scales GNNs on AIGs in two ways today. The first is to extract small subcircuits (tens to a few thousand gates) and train on those, which is a crude form of sampling that discards global structure; this is the de-facto reduction in most circuit-GNN work, including the DeepGate line [Li et al., 2022]. The second is to scale the *architecture*: DeepGate3 [Shi and TODO, 2024] pools over subcircuits with a transformer, and DeepGate4 [TODO, 2025] uses sparse attention for sub-linear memory. Both change the model, not the input.

Graph reduction (partitioning, sparsification, and summarization/coarsening) is the standard way to shrink the *input* instead, and it is well studied on citation, social and knowledge graphs [Hashemi et al., 2024]. It has never been systematically applied to whole AIGs as a preprocessing step for GNN training, and no work measures which reduction preserves a downstream synthesis label. The two literatures do not meet: generic summarization work [Bollen et al., 2023, Generale et al., 2022, Loukas, 2019] does not know about structural hashing or AIG semantics, and EDA work [Mishchenko et al., 2005, Li et al., 2022] does not frame its reductions as GNN-input summarization.

The gap this thesis addresses is therefore twofold: (i) there is no systematic evaluation of graph reduction for AIGs, and (ii) there is no evidence on whether AIG-specific adaptation of those reductions is worth its cost. Section 2.2.6 returns to this once the literature has been laid out.

1.2 Research Questions

Throughout, *optimizability* is the fraction of AIG nodes a given synthesis algorithm removes from a circuit (formalised in 2.1.6 and 3.3.4), and *compression ratio* is the node or edge retention of a reduced graph relative to its source (2.1.5). The distinction between node and edge retention is not cosmetic: some reduction methods preserve node counts exactly and only remove edges, so a single "compression" figure is ambiguous and RQ4 depends on the definition being pinned down first.

1.2.1 RQ1: Task Feasibility and Baseline

How well can a Graph Neural Network predict the optimizability of a circuit directly from its full, unreduced AIG representation, relative to trivial predictors, standard graph-level GNN encoders, and published circuit-representation models?

RQ1a: Protocol Sensitivity

[TODO: redo dont like the phrasing] How much of the measured score is a property of the evaluation protocol rather than of the model? Three train/test protocols are run under other-

wise identical conditions: a plain random split, a recipe-disjoint split, and the design-disjoint split used for every reported result (3.4.2). The gap between them is a direct measurement of how much of the apparent accuracy under the more permissive protocols is design recognition rather than structural inference. The per-design breakdown of the design-disjoint test set then shows whether the remaining error is uniform across held-out designs or concentrated in a few (4.1.6).

1.2.2 RQ2: Reduction Efficiency

[TODO: when applied to AIGs] How do graph reduction techniques, spanning partitioning, sparsification, and summarization/coarsening, differ in graph shrinkage rate, peak memory usage, training and inference speed, and the offline cost of applying the reduction itself?

1.2.3 RQ3: Predictive Retention and Trade-offs

To what extent do models trained on reduced AIGs retain predictive accuracy (measured via Smooth L1 Loss, RMSE, and R^2) and circuit-ranking ability (Spearman correlation) relative to the full-graph baseline established in RQ1, and which techniques offer the best accuracy-to-efficiency trade-off?

Hypothesis H1 (path contraction). [TODO: redo this completely and also why only have hypothesis here and not in others?] Sparsification *removes edges* and can only impede message propagation, whereas summarization *contracts paths* and shortens the distance a signal must travel. AIGs are very deep relative to the encoder's four message-passing layers, so the model is receptive-field starved and subject to over-squashing [Alon and Yahav, 2021]. Coarsening therefore has a mechanism by which it could *raise* accuracy rather than trade it away, and is expected to retain more than sparsification at matched compression.

1.2.4 RQ4: Value of Domain-Informed Adaptation

[TODO: aig-specific/ domain-knowledge informed....] Do AIG-specific reduction heuristics, which exploit topological level, reconvergence structure, and gate roles, retain more predictive accuracy than generic techniques at equivalent compression? If domain knowledge does not pay, what does that indicate about which structural features the model actually relies on?

Hypothesis H2 (weak heuristics). The domain-informed adaptations tested here are relatively simple (edge-span weighting, level slicing, gate-role filtering, reconvergence-bounded merging) and are not expected to win outright. A negative result is still a result: it would indicate that the predictive signal is distributed across the AIG rather than concentrated in the causal cones and long paths these heuristics were built to protect. Section 4.4.4 bounds what such a result would and would not license.

1.2.5 RQ5: Reduced to Unreduced? Generalization

Can a model trained exclusively on reduced AIGs (partitioned, sparsified, or summarized) be queried on normal, full-sized AIGs at inference time, and does it retain predictive accuracy across this change of structural state?

Inference is the regime where this pays off. Training stores activations and gradients for every node in a batch, whereas forward-only inference stores neither. A model that had to be trained on reduced graphs to fit in memory may therefore still serve full graphs on substantially more modest hardware. The practical claim under test is *train cheap, infer full.*

1.2.6 Scope and Delimitations

Each item below states what is included and what is deliberately excluded. These are choices made in advance; limitations discovered during the work are separated out into 5.4.

Target Synthesis Algorithm

Training and evaluation target a single synthesis script, `Orchestrate`. The dataset also contains graphs optimized by `Deepsyn`, `Syn4` and `C2RS`, and those are left unused here. Focusing on one algorithm removes cross-algorithm variance as a confound: the question is how much a *reduction* costs, and comparing reductions across several target algorithms at once would not isolate that. Extending to the remaining three is the most direct piece of future work (6.3.1).

Definition of Optimizability

Optimizability is defined as relative *node* reduction only. Area, delay and power, the objectives synthesis actually optimizes for, are not modelled, and node count is used as a proxy for area. Whether that proxy is adequate is a construct-validity question taken up in 5.4.4.

Graph Scale and Dataset Boundaries

The corpus is bounded at an intermediate scale (see 3.4.2), not at industrial scale. This is a deliberate consequence of the design: RQ1 requires a full-graph baseline, and a baseline that does not fit in memory cannot be measured. The motivation invokes AIGs of millions to billions of nodes; the evaluation does not reach them, and 5.4.2 elaborates on what does and does not extrapolate.

Reduction Families Considered

[TODO: chosen because of survery on graph reduction paper] Three families are covered: partitioning, sparsification, and summarization/coarsening. Graph *condensation* (synthesising a small graph by gradient or distribution matching) is excluded, for reasons given in 2.2.3: it produces no correspondence between synthetic and real nodes, it is label-dependent by construction, and it is tied to a specific architecture, so it cannot

answer RQ5, which requires the same weights to ingest both reduced and full graphs.

Model Architecture

One encoder family is used throughout (an edge-aware GCN+, 3.1), held fixed so that differences in outcome are attributable to the reduction rather than to the model. Published circuit models are used as *baselines* for RQ1 (3.1.6) but are not themselves trained on reduced graphs. Whether the reduction rankings transfer to other encoders is untested.

Prediction Rather Than Script Generation

This work predicts optimizability; it does not generate synthesis scripts. The gap between the two is worth stating precisely, because the motivation is script selection and the metrics are not. Spearman correlation here ranks *circuits* by how optimizable they are under one fixed script. Selecting a script would require ranking *algorithms* for one circuit, which a single-algorithm model cannot do. Closing that loop is future work (6.3.4); the contribution claimed here is the representation and reduction machinery it would need, not the selector itself.

[TODO: to consider something more specific on summairzation exact vs approximate? the proving of exactness bollen style?]

1.3 Contributions

[TODO: at the end when all is done]

1.3.1 Summary of Results

1.4 Thesis Outline

Chapter 2 introduces the necessary background (logic synthesis and AIGs, graph neural networks, and the three families of graph reduction) and then reviews the literature the gap in 1.1.5 sits in. Chapter 3 describes the encoder, the reduction methods implemented for each family, the dataset and its labels, and the experimental protocol. Chapter 4 reports results per research question. Chapter 5 interprets them against the literature and states the limitations, and Chapter 6 answers each research question in turn and outlines future work.

Chapter 2

Preliminaries & Related Work

2.1 Preliminaries

2.1.1 Logic Synthesis and the EDA Flow

[TODO: lets call this subsection Electronic Design Automation and Logic Synthesis] [TODO: add subsubseciton on EDA itself what is it what is it used for why should i care]

Position of Logic Synthesis in the Design Flow

Electronic Design Automation (EDA) organises the design of a digital circuit as a flow of successive refinements [De Micheli, 1994]. A Register-Transfer-Level (RTL) description is compiled into a network of Boolean gates, Logic Synthesis (LS) restructures that network against cost objectives, technology mapping binds the result to the cells of a target library, and placement and routing assign physical positions and wires. LS is pre-physical: the network it operates on has no coordinates, no wire lengths and no layout-extracted timing, which is what makes the spatial coarsening methods of 2.2.4 inapplicable to it.

[TODO: consider shoudl these subsubseciotn not be after mayeb introducing AIGs and with the algorithm.tex]

Optimization Objectives: Area, Power, Delay

Synthesis quality is measured against three costs: the silicon area a circuit occupies, the power it draws, and the delay of its longest combinational path. None of the three is physical before technology mapping, so synthesis tracks each through a structural proxy, gate count for area and logic depth for delay, both formalized on the AIG representation in 2.1.2. This work uses node count as its area proxy and models nothing else (1.2.6). The validity cost of that restriction is assessed in 5.4.4. **[TODO: mentioned already earlier,prelim should just be introduction to concepts not arguments abotu psitions and thesis]**

Synthesis Scripts and Pipelines

A synthesis script is an ordered sequence of function-preserving transformations applied as a composition,

$$S = t_m \circ t_{m-1} \circ \cdots \circ t_1, \tag{2.1}$$

and order matters: the t_i do not commute, and the gain of each depends on the structure its predecessors produced, so the same script yields different gains on different designs. Script choice can therefore not be fixed once and reused across designs. The scripts used in this work are the subject of 2.1.3. **[TODO: work in somehow the work recipe since it is used and steps and what they are as this is used to explain teh dataset generation] [TODO: also i dont htink this mentions why one needs and does AIG optimization like why smaller why is that better]**

2.1.2 And-Inverter Graphs

Definition and Structure

An And-Inverter Graph (AIG) is a Directed Acyclic Graph (DAG) $G = (\mathcal{V}, \mathcal{E})$ representing a Boolean network over the basis $\{\wedge, \neg\}$ [Mishchenko et al., 2006]. Every internal node is a two-input AND gate, and every edge carries an optional inversion, so conjunction is the only operation that costs a node and negation costs none. The basis is functionally complete, which is what allows an arbitrary Boolean network to be expressed in one uniform structure rather than in a gate library.

The vocabulary used throughout follows from the edge set. The fanin and fanout of a node are

$$\mathrm{fi}(v) = \{\, u \in \mathcal{V} : (u, v) \in \mathcal{E} \,\}, \qquad \mathrm{fo}(v) = \{\, w \in \mathcal{V} : (v, w) \in \mathcal{E} \,\}, \tag{2.2}$$

with $|\,\mathrm{fi}(v)| = 2$ at every AND gate, $\mathrm{fi}(v) = \emptyset$ at every Primary Input (PI) and at the constant, and $|\,\mathrm{fi}(v)| = 1$ at every Primary Output (PO). Acyclicity yields a topological order $\prec$ on $\mathcal{V}$ with $u \prec v$ for every $(u, v) \in \mathcal{E}$, so signals flow from the PIs to the POs and never return.

The POs are materialised as nodes here, rather than as pointers into the AND network as is also common. That choice is why the node role defined below takes four values instead of three, and it puts the circuit interface inside $|\mathcal{V}|$, the quantity the target of 3.3.4 is measured on.

Node Types and Inverted Edges

Two labellings are carried on every graph. A node role

$$\tau \colon \mathcal{V} \to \{\, \mathrm{const},\ \mathrm{PI},\ \mathrm{AND},\ \mathrm{PO} \,\} \tag{2.3}$$

separates the constant, the PIs, the internal AND gates and the POs, and an inversion flag $\mathrm{inv} \colon \mathcal{E} \to \{0, 1\}$ marks each edge as a regular connection or an inverted one. Inversion is semantic rather than decorative. Writing $b_u \in \{0, 1\}$ for the value produced at u, the value an edge $e = (u, v)$ delivers to v is

$$b_u \oplus \mathrm{inv}(e), \tag{2.4}$$

so two edges agreeing in their endpoints and differing in inv carry complementary signals. The one-hot encodings of τ and inv are the four-valued node feature and two-valued edge feature of 3.3.5, and are not restated there.

Structural Properties: Levels, Logic Cones, Fanout

[TODO: fanin] The topological level of a node is its longest distance from the input frontier,

$$\ell(v) = \begin{cases} 0 & \text{if } \mathrm{fi}(v) = \emptyset, \\ 1 + \max_{u \in \mathrm{fi}(v)} \ell(u) & \text{otherwise,} \end{cases} \tag{2.5}$$

and the depth of the graph is $\ell(G) = \max_{v \in \mathcal{V}} \ell(v)$. Level is an absolute coordinate that an AIG has and a general graph does not, and four constructions in this thesis read it: the positional encoding of 3.3.5, the level slicing and span-weighted partitioners of 3.2.1, and the level bands of the cone candidate in 3.2.3.

Writing $u \rightsquigarrow v$ for the existence of a directed path, of length zero when $u = v$, the fanin cone (equivalently the logic or causal cone) and the fanout cone of v are

$$\mathcal{C}(v) = \{\, u \in \mathcal{V} : u \rightsquigarrow v \,\}, \qquad \mathcal{F}(v) = \{\, w \in \mathcal{V} : v \rightsquigarrow w \,\}. \tag{2.6}$$

Both cones contain v itself. The fanin cone is the set of gates on which v functionally depends, and (2.5) measures the longest path through it. The claim of 1.1.4, that generic reduction severs causal structure, is a claim about $\mathcal{C}$: deleting one edge on a long path removes gates from the cones of everything downstream of it, whether or not the result resembles the original under an undirected measure. The depth-bounded form $\mathcal{C}_k$ of (3.33) is what makes the effect measurable.

Fanout degree $|\operatorname{fo}(v)|$ separates gates whose output is consumed once from gates whose output is shared. Chains of single-fanout gates carry no sharing information; multi-fanout gates are the points at which logic is reused, and they are what constrains rewriting. Reconvergence is the same property seen from below: v is a reconvergence point of u when two internally disjoint directed paths run from u to v. AIGs are dense in reconvergence, because structural hashing (2.1.3) shares every duplicated subexpression instead of replicating it [Kuehlmann et al., 2002].

Two dominance relations are needed, and only one of them is informative on an AIG. Add a virtual source s preceding every PI and a virtual sink t succeeding every PO. A node u dominates v when every path $s \rightsquigarrow v$ passes through u, and post-dominates v when every path $v \rightsquigarrow t$ passes through u; the immediate dominator $\operatorname{idom}(v)$ and immediate post-dominator $\operatorname{ipdom}(v)$ are the closest such nodes [Lengauer and Tarjan, 1979]. A gate below the input frontier is reachable from s along many independent paths, so $\operatorname{idom}(v)$ degenerates to s for most AND gates and partitions nothing. The post-dominator does not degenerate the same way. Here $\operatorname{ipdom}(v)$ is the gate at which the fanout cone $\mathcal{F}(v)$ reconverges, or t when that cone never reconverges, and grouping by it is what the cone candidate of 3.2.3 uses.

The Maximal Fanout-Free Cone (MFFC) of a root gate v collects the gates feeding v and nothing else [Mishchenko et al., 2006],

$$\operatorname{MFFC}(v) = \{\, u \in \mathcal{C}(v) : \text{every path } u \rightsquigarrow t \text{ passes through } v \,\}, \tag{2.7}$$

so removing v from the network removes exactly $\operatorname{MFFC}(v)$ and nothing more. That property makes the MFFC the natural unit of local restructuring, and refactoring collapses and re-expresses one MFFC at a time (2.1.3). It is also the unit merged by the second domain candidate of 3.2.3, which is why that candidate carries an argument about label leakage and the others do not.

Why AIGs Differ from General Graphs

Four properties separate an AIG from the citation, social and knowledge graphs on which graph reduction was developed, and each costs a generic method something specific.

(i) An AIG is directed and acyclic. Machinery built for undirected graphs does not transfer unchanged, because a quotient of a DAG need not be acyclic, and a reduction computed on the undirected projection discards the orientation that carries the logic. The spanning forest of 3.2.2 is the concrete case.

(ii) An AIG is deep relative to any practical encoder. Against an encoder of L message-passing layers, a node representation sees an L-hop prefix of a fanin cone whose length is 35 levels at the corpus median and 24,802 at its maximum (3.3.1). Receptive-field starvation is therefore the normal condition on this data, and it is the starvation that hypothesis H1 (1.2.3) proposes coarsening relieves.

(iii) Edges carry function. The inversion in (2.4) is part of what the graph represents, so a merge that averages or discards inv changes the Boolean function rather than approximating it. Every summarization method here is therefore constrained to keep normal and inverted connections distinguishable, a constraint no general method imposes on itself.

(iv) The interface is fixed. The PIs and POs are the circuit's contract with the rest of the design, and merging them away changes what the graph is rather than coarsening it. The boundary is enforced uniformly in 3.2.3.

A reduction operator can respect or ignore all four at no syntactic cost, since the graph it produces is well formed either way. Whether respecting them buys accuracy is the question RQ4 (1.2.4) measures.

2.1.3 Synthesis Algorithms

Rewriting, Refactoring, and Balancing

Every script in this thesis is assembled from a small set of function-preserving local transformations [Mishchenko et al., 2006]. A cut of a gate v is a node set through which every PI-to-v path passes, and it is K-feasible when it has at most K members, so the logic between the cut and v computes a Boolean function of at most K variables. Rewriting enumerates K-feasible cuts at each AND gate and replaces the enclosed logic with the smallest precomputed equivalent, accepting a replacement only when sharing with the rest of the network makes it a net gain [Bjesse and Borälv, 2004, Mishchenko et al., 2006]. Refactoring makes the same exchange with the MFFC of (2.7) as the unit, collapsing one cone at a time and re-expressing it [Mishchenko and Brayton, 2006]. Balancing restructures AND trees under associativity to reduce the depth $\ell(G)$ of (2.5) at an unchanged function. Resubstitution re-expresses a gate in terms of gates already present in the network, its divisors, and removes the MFFC made redundant when a valid re-expression is found [Mishchenko and Brayton, 2006].

Two constructions fixed here are read repeatedly later. The first is structural hashing [Kuehlmann et al., 2002]. Writing

$$\kappa(v) = \{\, (u, \mathrm{inv}(u, v)) : u \in \operatorname{fi}(v) \,\} \tag{2.8}$$

for the signature of an AND gate, hashing maintains on construction the invariant that $\kappa(u) \neq \kappa(v)$ for all distinct AND gates u and v. One-hop structural redundancy, two gates over the same fanins with the same inversions, is therefore already gone before any summarization runs (3.2.3).

The second is the line between structural and functional equivalence. Two nodes are functionally equivalent when they agree under every assignment to the PIs,

$$u \equiv v \iff b_u = b_v \text{ under every PI assignment,} \tag{2.9}$$

with b_u as in (2.4), and functionally complementary when $b_u = \neg b_v$ throughout. A Functionally Reduced AIG (FRAIG) merges every such pair, using simulation to refute candidates cheaply and SAT to prove the survivors [Mishchenko et al., 2005]. Structural hashing decides (2.9) only in the special case $\kappa(u) = \kappa(v)$; the general case is a semantic property that no local inspection reveals. Every summarization method in this thesis stays on the structural side of that line, because a merge by (2.9) performs part of the optimization the model is trained to predict (5.4.5).

The Orchestrate Script

The target algorithm, Orchestrate, is an orchestrated flow in the sense of Li et al. [2024]. A conventional script commits to one ordering of whole-graph passes. Orchestration moves the choice into the graph: at each AND gate, rewriting, refactoring and resubstitution are attempted and the transformation with the best local area-delay score is committed, and the sweep repeats for a bounded number of passes. Two properties make it the target: (i) it subsumes the orderings of the fixed scripts built from the same primitives, so the label measures optimization potential rather than the accident of one command sequence, and (ii) it is deterministic, so $S(G)$ is a single graph rather than a distribution over runs and "optimizability under Orchestrate" is well defined as the $y(G)$ of 3.3.4. The exact invocation is recorded in A.1.

The Non-Target Scripts: Deepsyn, Syn4, and C2RS

Three further scripts of the synthesis system were applied during dataset generation [Brayton and Mishchenko, 2010], and every tier-1 graph that enters training is the output of one of them (3.3.3). `Deepsyn` searches: randomized restructuring passes are applied repeatedly under a wall-clock budget and the best network encountered is returned. Its seed was drawn afresh for every graph and never recorded, so the `Deepsyn` portion of the corpus cannot be regenerated (5.4.1). `Syn4` applies a fixed, deterministic restructuring flow built from the primitives of 2.1.3, invoked with no arguments, so its behaviour is entirely that of the tool's built-in script. `C2RS` expands to a chain of balancing, refactoring, rewriting and resubstitution steps [Mishchenko and Brayton, 2006] whose resubstitution cut size widens from $K = 6$ to $K = 12$, so successive steps reason over progressively larger windows. The invocations are recorded in A.1.

2.1.4 Graph Neural Networks

Message Passing Framework

A Graph Neural Network (GNN) computes a representation $h_v^l \in \mathbb{R}^w$ for every node v by repeated neighbourhood aggregation [Kipf and Welling, 2017, Hamilton et al., 2017]. Writing $\mathcal{N}(v)$ for the neighbours of v and e_{uv} for the attribute of edge (u, v), one layer computes

$$h_v^l = \phi^l\left(h_v^{l-1}, \bigoplus_{u \in \mathcal{N}(v)} \psi^l\left(h_u^{l-1}, e_{uv}\right)\right), \tag{2.10}$$

where ψ^l builds a message per edge, $\bigoplus$ is a permutation-invariant aggregation such as a sum, and ϕ^l updates the node state. The encoder of this thesis is one instantiation of (2.10), written out in 3.1.1.

After L layers, h_v^L is a function of exactly the L-hop neighbourhood of v and of nothing outside it. On an AIG that neighbourhood is a depth-L prefix of the cones of (2.6), so an encoder of depth L reads L levels of logic around each gate. Every depth-coupling argument in this thesis rests on this one fact: the refinement depth of 3.2.3 is set to L, the receptive-field metric of 3.4.3 counts the gates inside L hops, and hypothesis H1 (1.2.3) is a claim about what path contraction does to those L hops.

Graph-Level Readout and Pooling

A graph-level prediction pools the node representations into one vector, here by a mean over nodes, optionally after summing the per-layer outputs so that shallow and deep features both reach the readout (jumping knowledge, Xu et al., 2018). Pooling imposes the constraint that shapes the whole experimental setup: every node of a graph must be present in the same forward pass, so a graph is the indivisible unit of batching and cannot be split across steps. Batches are therefore assembled under a node budget rather than as a fixed graph count (3.4.2), and the memory cost of one step is set by the largest graphs, which is exactly the cost reduction is meant to lower.

Positional and Structural Encodings

Message passing sees only relative structure: two nodes whose L-hop neighbourhoods are isomorphic receive identical representations wherever they sit in the graph. A Positional Encoding (PE) breaks that indifference by attaching a coordinate to each node. On an undirected graph no canonical coordinate exists and encodings are built from random walks or spectra; on a DAG the topological level $\ell(v)$ of (2.5) is an absolute coordinate that the graph carries for free. The level-based encoding used here (3.3.5) is therefore a DAG-native choice rather than a generic one.

Expressivity and the Weisfeiler-Leman Hierarchy

Colour refinement, equivalently the one-dimensional Weisfeiler-Leman algorithm (1-WL), iteratively refines a node colouring by the multiset of neighbour colours,

$$c_v^{(t)} = \mathrm{hash}\left(c_v^{(t-1)},\ \{\!\{\, c_u^{(t-1)} : u \in \mathcal{N}(v) \,\}\!\}\right), \tag{2.11}$$

starting from initial labels $c_v^{(0)}$. A partition of $\mathcal{V}$ is *equitable* when every node of a class has the same number of neighbours in every class; run to a fixed point, (2.11) computes the coarsest equitable partition refining the initial colouring [Grohe et al.,

2014]. The connection to (2.10) is an upper bound: a message-passing network is at most as discriminative as 1-WL, so two nodes sharing a colour after L rounds receive identical representations from every L-layer encoder [Xu et al., 2019].

Replacing the multiset in (2.11) by a set yields bisimulation, which ignores how many neighbours of a colour exist and asks only whether one does. A count cap c interpolates between the two, truncating each multiplicity at c: the cap $c = 1$ is bisimulation and $c = \infty$ is colour refinement [Bollen et al., 2023]. The bound above is what makes quotienting by such a partition lossless rather than approximate: merging nodes the network provably cannot distinguish removes no information the network could have used, and the automorphism orbits of a graph form an equitable partition, so genuinely symmetric structure is always compressible this way. AIGs have such structure in quantity. The two fanins of an AND gate are interchangeable, datapath logic replicates one bit slice across the word width, and standard sub-circuits recur throughout a design, each of which creates orbits that a random graph does not have. How much of this redundancy survives structural hashing is measured in 3.2.3.

Training Cost, Memory, and Inference

Training and inference differ in what must be held in memory. Backpropagation stores the activations of every layer for every node in the batch, so training memory grows with $L \cdot |\mathcal{V}|$ on top of parameters and optimizer state, and the largest graph in the corpus sets the peak. Forward-only inference stores none of this. The asymmetry is the mechanism RQ5 (1.2.5) depends on: a model that must be trained on reduced graphs to fit in device memory may still be queried on full graphs afterwards.

Four techniques recur in Chapter 3 and are defined here once. Reported memory is *allocated* memory, the tensors actually held, as opposed to the larger *reserved* pool the allocator keeps (3.4.3). Mixed precision stores activations in half-width floating point and roughly halves their cost. Gradient checkpointing discards intermediate activations and recomputes them during the backward pass, trading compute for memory, and is what makes the largest baseline runnable at all (3.1.6). Gradient accumulation sums gradients over several small steps before updating, so the effective batch size exceeds what fits in one step. All four trade something for memory at a *fixed* input size, which is what separates them from the input reduction this thesis studies (2.2.2).

Depth has a cost of its own. A fixed-width representation cannot carry the information of a receptive field that grows exponentially with depth, so signal from distant nodes is compressed away, a failure known as over-squashing [Alon and Yahav, 2021]. Contracting paths shortens the distance signal must travel and is a recognised remedy [Dwivedi et al., 2022]. That contraction is the mechanism behind hypothesis H1 (1.2.3), which expects summarization to help where sparsification can only remove.

[TODO: do i need something maybe on trainign and inference and memory bottlenecks with gnns the formulas and math behind it]

2.1.5 Graph Reduction: Definitions

A reduction operator R maps a graph G to a smaller graph $R(G)$ intended to stand in for it. The taxonomy used throughout follows the survey of Hashemi et al. [2024], which divides the field into sparsification, coarsening and condensation, with one substitution: partitioning replaces condensation as the third family. Partitioning is what a distributed training setup forces on a graph that does not fit one device (2.2.2), whereas condensation is excluded from this study outright, for the reasons given in 2.2.3. The substitution is deliberate, not an error of transcription.

Partitioning

Partitioning splits $\mathcal{V}$ into k disjoint blocks and deletes every edge whose endpoints fall in different blocks. It is the one family that keeps every node: only spanning edges are lost, so the node count of $R(G)$ always equals that of G and any compression it achieves is compression of the edge set alone. What varies between partitioners is the objective by which blocks are chosen (3.2.1).

Sparsification

Sparsification removes edges, or nodes together with their incident edges, while preserving some structural property of the original, such as pairwise distances up to a stretch factor. The two variants are not interchangeable for measurement: an edge-removing method leaves $|\mathcal{V}|$ untouched while a node-removing method shrinks it, and the distinction is what forces the two-ratio convention of 2.1.5.

Summarization and Coarsening

Summarization merges nodes into *super-nodes*. A coarsening is specified by a merge map, a surjection $\mu\colon \mathcal{V} \to \mathcal{V}'$ onto the cluster set, and the resulting quotient graph connects two clusters whenever any of their members were connected. The number of member edges a super-edge stands for is its multiplicity,

$$w_{u'v'} = \big|\{\,(u,v) \in \mathcal{E} : \mu(u) = u',\ \mu(v) = v'\,\}\big|, \qquad (2.12)$$

and when super-edges differing in an attribute are kept parallel rather than collapsed, the quotient is a multigraph. Because μ is a function, its preimages partition $\mathcal{V}$: a coarsening *is* a partition of the node set. A tolerance band such as "within one level" therefore cannot define one, since the relation it induces is not transitive, a fact the domain-specific candidate of 3.2.3 has to work around.

[TODO: shoudl this be here in preliminaries or more in evaluatoin where i actually tlak abotu how i am takign measurements on my data?]

Measuring Reduction: Compression Ratio

Reduction is reported as retention, kept over original, separately per node and per edge:

$$\eta_{\mathcal{V}}(G) = \frac{|\mathcal{V}(R(G))|}{|\mathcal{V}(G)|}, \qquad \eta_{\mathcal{E}}(G) = \frac{|\mathcal{E}(R(G))|}{|\mathcal{E}(G)|}, \tag{2.13}$$

with reduction $1 - \eta$. One number cannot carry both: partitioning and edge-removing sparsification hold $\eta_{\mathcal{V}} = 1$ while lowering $\eta_{\mathcal{E}}$, and node-removing methods move the two together. Both ratios are therefore reported everywhere, and never collapsed (3.4.3). Methods further differ in whether compression is a knob or an outcome: some accept a target ratio, while for others η is a fixed property of the input graph. Comparing methods at matched compression is consequently a design problem rather than a reporting convention, and 3.2.3 is its solution.

[TODO: need to formalize exact sumamrization what is required for model and data and task]

2.1.6 Problem Formalization

The objects fixed above assemble into one problem statement. Let G be an AIG with the node roles of (2.3) and edge inversions of (2.4) as its features, and let S be a deterministic synthesis algorithm (2.1.3) mapping G to the function-equivalent, optimized $S(G)$. The prediction target is the *optimizability* of G under S, the fraction of nodes the algorithm removes,

$$y(G) = 1 - \frac{|\mathcal{V}(S(G))|}{|\mathcal{V}(G)|} \in [0, 1]. \tag{2.14}$$

Obtaining $y(G)$ means running S, which is the expensive step prediction is meant to avoid. An encoder f_θ, a GNN in the sense of 2.1.4 with a graph-level readout, is trained to approximate y. A reduction operator R, drawn from the three families of 2.1.5, shrinks the encoder's input.

Each research question of 1.2 is one statement about these four objects. RQ1 asks how closely $f_\theta(G)$ approximates $y(G)$ when training and evaluation both use unreduced graphs. RQ2 asks what R buys and costs in itself: the retention ratios (2.13), the memory and time of training on $R(G)$, and the offline cost of computing R. RQ3 asks how much accuracy f_θ loses when trained and evaluated on $R(G)$ instead of G. RQ4 asks whether an R built from AIG structure beats a generic R at equal retention. RQ5 asks whether f_θ trained on $R(G)$ still approximates $y(G)$ when queried on G itself.

RQ5 is well posed only under a compatibility requirement worth stating now: $R(G)$ and G must occupy the same input space, so that one set of weights θ ingests both. Every summarization method here therefore produces features of which the full-graph features are the single-member special case (3.2.3). A reduction that changed the schema would make cross-state inference undefined rather than merely inaccurate.

[TODO: should not be here in prelim arguably sould be in problem RQ area?! if it adds anything not already said]

2.2 Related Work

[TODO: this setnence is not subtle enough imo] The gap this chapter documents can be stated before the evidence: graph reduction is a developed literature with surveys of its own [Hashemi et al., 2024, Shabani et al., 2023], circuit representation learning is a developed literature with a growing model family, and no published work applies the first to whole AIGs as input reduction for the second. The subsections below establish each half, then the two nearest attempts at a bridge, and 2.2.6 returns to the claim once the evidence for it is on the table.

2.2.1 Machine Learning for Logic Synthesis

[TODO: order of subsections seems odd here shoudl be tasks it is used in then maybe datset then maybe eval protocols and unseen design then baselines? also think about the tasks we elaborate on and if they are relevant we should list all tasks that ML is used with but also instead of ML4EDA mayeb say ML4LS and specify AIG. but tasks missing are like AIG generation etc.]

Quality-of-Result and Optimizability Prediction

The direct precedents predict synthesis Quality of Result (QoR) from a circuit before running the tool. OpenABC-D [Chowdhury et al., 2021] is the reference dataset for the task and publishes, among its graph-level labels, the percentage of nodes a recipe optimizes away, essentially the target (2.14); that overlap is what makes its accompanying SynthNet a fair baseline here rather than a repurposed model (2.2.1). Recipe-conditioned models jointly encode the circuit and the synthesis sequence [TODO, 2022, Wu et al., 2022]. One terminological collision is worth heading off: LOSTIN [Wu et al., 2022] attaches a supernode that encodes the synthesis *sequence*, a temporal use of the same device this thesis uses structurally, and the two share nothing beyond the name.

Evaluation Protocols and the Unseen-Design Gap

"Unseen" means three different things in this literature: an unseen recipe, an unseen design, or an unseen design-recipe pair whose parts were both seen, and the third is far easier than the second. Generalization to unseen designs is the stated motivation of much of the field, yet matched seen and unseen results are rare: of the papers surveyed for this thesis, 4 of 63 report both sides, OpenABC-D [Chowdhury et al., 2021], LOSTIN [Wu et al., 2022], LSOformer [Karimi et al., 2024], and the XGBoost AIG-timing predictor of Jiang et al. [2025], while HOGA [Deng et al., 2024] motivates on the gap without publishing a seen-design baseline, so its degradation cannot be computed. The canonical unseen-IP split trains on 16 small designs and tests on 8 large ones, confounding design novelty with size extrapolation. Reported error moves more with the benchmark than with the architecture, from a few percent on small homogeneous suites to above twenty percent on OpenABC-D, and within one test set the error concentrates in a few designs rather than spreading uniformly. Those three observations motivate RQ1a's matched

three-protocol comparison and the per-design reporting used throughout (1.2.1).

Learned Synthesis Script Generation

A separate line learns to construct or select synthesis scripts, by reinforcement learning or sequence prediction over the same primitive transformations. It is the downstream application the motivation invokes, and it is out of scope here (1.2.6): script generation consumes exactly the kind of cheap optimizability estimate this thesis builds, which is why the prediction task is worth solving on its own. DRiLLS [Hosny et al., 2020] is representative, an RL agent choosing among the transformations of 2.1.3 at each step rather than committing to a fixed script in advance.

Circuit Representation Learning

The DeepGate line [Li et al., 2022, Shi and TODO, 2024, TODO, 2025] learns general-purpose gate embeddings on AIGs, and PolarGate [Liu et al., 2024] identifies the handling of inverted connections as the representational bottleneck of AIG encoders, external support for treating inversion as a relation summarization must preserve rather than average away (3.2.3). Two facts from this line shape the present study. First, DeepGate3 and DeepGate4 answer scale by growing the *model*, with sparse attention and larger training corpora, where this thesis shrinks the *input*. DeepGate4 doubles as a baseline (2.2.1), so the contrast is measured rather than rhetorical. Second, these models train on extracted subcircuits of tens to a few thousand gates. Extraction is a reduction, applied at corpus construction and never named or evaluated as one, and that unexamined step is half of the gap this chapter closes in 2.2.6.

Models Used as Baselines

[TODO: make each baseline model its own subsubsubsection or paragraph explain why it is important and useful as a baseline without sayign this is why it was chosen as a basleine so be liek its SOTA or its foundational or its standard or its the best at this or claims to be etc. also include the math and what makes these models and apparoaches special] Three published models serve as tier-2 baselines, and 3.1.6 states only how each was adapted, so what each one *is* belongs here. SynthNet [Chowdhury et al., 2021] is a recipe-conditioned graph convolutional QoR regressor, the closest published analogue of the label predicted here. HOGA [Deng et al., 2024] applies hop-wise attention to multi-hop features precomputed per node, so its depth of field is fixed before training and its trunk never sees connectivity, a design aimed at scalable, distributable training. DeepGate4 [TODO, 2025] runs sparse attention over a distance-bounded virtual edge set and is the architecture-scaling counterpart to the input scaling studied here. Each comparison licenses a different conclusion: against SynthNet the claim is task accuracy, against HOGA it is whether connectivity at training time matters, and against DeepGate4 it is input against architecture scaling at matched memory.

Benchmarks and Datasets for ML in EDA

OpenABC-D [Chowdhury et al., 2021] remains the largest published synthesis-prediction corpus: 29 designs, 1500 recipes each, labels per design-recipe pair. The dataset built for this thesis (3.3) differs in axis rather than only in size: it is tiered by optimization state, pairing each base graph with already-optimized variants of itself (3.3.3), because the reduction questions RQ3 and RQ5 need graphs at several points along the optimization trajectory, not several recipes on one starting point. OpenLS-DGF [Ni et al., 2025] takes a third axis again, generating designs adaptively rather than fixing a benchmark suite, and Chowdhury et al. [2023] argue the position this dataset acts on: ML4EDA lacks the standard, prototypical datasets that drove progress in computer vision.

2.2.2 Scaling Graph Neural Networks to Large Graphs

Sampling-Based Approaches

The oldest answer to a graph that does not fit is to train on samples of it: neighbour sampling caps the fanout of each aggregation [Hamilton et al., 2017], and subgraph sampling trains on induced pieces. Sampling is a per-epoch stochastic operation built for node-level targets, and both properties disqualify it here. A graph-level regression needs every node of a graph in one forward pass (2.1.4), and the reduction under study must be a deterministic, cacheable artifact computed once offline, so that its cost amortises across runs and its output can be inspected as a graph in its own right.

Partitioning and Distributed Training

The standard industrial answer is to partition the graph across devices and exchange messages at the boundaries, or forgo them there. This is the setting in which cutting edges is forced rather than chosen, and it is why partitioning is carried as a reduction family here (2.1.5) despite removing no nodes: the accuracy cost of severed spanning edges is exactly what a distributed practitioner pays, measured on one device. DistDGL [Zheng et al., 2020] is representative of the practice at billion-node scale.

Memory-Efficient Training Techniques

Gradient checkpointing, mixed precision and gradient accumulation, defined in 2.1.4, are used in this thesis rather than studied by it. They must still be named, because a reader will ask why input reduction is needed when checkpointing exists. The answer is composition: each of these techniques trades compute, precision or latency for memory at a fixed input size, reduction changes the input size itself, and the two stack. The baseline configurations depend on the first fact, DeepGate4 being runnable only under checkpointing (3.1.6), and the contribution depends on the second.

[TODO: evaluatre if this is needed?]

2.2.3 Graph Reduction Techniques

[TODO: make sure to introduce all that i will be using here higher level why they are important. some math formulas that are important and relavant etc. should not mention AIGs here i will address that in methodology/reduction section]

Graph Partitioning

METIS [Karypis and Kumar, 1998] is the classical multilevel partitioner and the one used here: it minimises the number of cut edges subject to balanced block sizes. That objective is topology-blind in the sense that matters for an AIG: it counts cut edges without asking which edges they are, and a cut that severs a long combinational path costs the same as one that severs a local shortcut. The span-weighted variant of 3.2.1 modifies exactly this term.

Graph Sparsification

The principled end of the family preserves a stated property within a stated tolerance: a t-spanner keeps pairwise distances within a factor t [Peleg and Schäffer, 1989, Baswana and Sen, 2007], and spectral methods preserve the Laplacian quadratic form. One negative result from this thesis belongs in the record: spanner-based sparsification was implemented and abandoned, because a strashed AIG lacks the dense cyclic redundancy spanners exploit and the surviving spanning structure is nearly the whole graph (3.2.2). The methods actually run are correspondingly simpler (3.2.2).

Graph Summarization and Coarsening

The methods of 3.2.3 come from four strands. The exact strand merges by equivalence: colour refinement and bisimulation with the count-cap interpolation between them [Bollen et al., 2023], resting on the equitable-partition foundation of Grohe et al. [2014], while k-SNAP [Tian et al., 2008] groups by attributes and neighbour classes and, being the depth-1 case of graded refinement, is cited rather than run. The spectral and classical strand descends from scientific computing: Local Variation coarsening [Loukas, 2019], the lineage surveyed by Chen et al. [2022], and Kron reduction with its directed extension [Sugiyama and Sato, 2023]. The GNN-aware strand is ConvMatch [Dickens et al., 2024], which merges nodes equivalent under the graph convolution itself rather than under a graph property, reporting about 95% of full-graph performance at 1% of the size on node classification. The cheap strand is hashing: UGC coarsens by locality-sensitive hashing in linear time [TODO, 2024]. Shabani et al. [2023] survey the field's use with GNNs.

Condensation is the family this thesis excludes (1.2.6), and the four reasons belong here, where a reader will look for them. GCond and its descendants [Jin et al., 2022] synthesise a new small graph by gradient matching rather than merging the real one, so (i) no correspondence exists between synthetic and real nodes, (ii) the method is label-dependent by construction, (iii) gradient matching ties the result to one architecture, and (iv) with no real nodes there is nothing to run cross-state inference on, so RQ5 is unanswerable even in principle.

Reduction as Preprocessing for GNN Training

The specific precedent for this setup is small. SCAL [Huang et al., 2021] trains on a coarsened graph and infers on the original with the same weights, the shared-weights mechanism RQ5 tests, whereas Generale et al. [2022] transfer weights back through a node-to-super-node mapping for a node-level task on knowledge graphs; their finding that the *content* of super-node features was critical is the direct justification for carrying member counts and level statistics on super-nodes here (3.2.3). The same situation is framed by Buffelli et al. [2022] as distribution shift in graph size. The distinction that makes RQ5 well posed falls out of these three: a node-level task needs a mapping back to the original nodes, whereas a graph-level regression needs only a compatible input space (3.2.3), so shared weights suffice and no transfer step exists to go wrong.

[TODO: summarizaiton it is important to look at Bollen exact compression stuff what model needs to fulfill what the graph does too]

2.2.4 Structure-Preserving and Domain-Aware Reduction

Reduction on Directed Acyclic Graphs

Direction changes more than notation. DAG-native architectures process nodes in topological order and read direction as causality [Thost and Chen, 2021, Zhang et al., 2019], and on the reduction side only isolated pieces exist, such as the directed extension of Kron reduction [Sugiyama and Sato, 2023]. Coarsening theory is otherwise stated for undirected graphs: its guarantees concern spectra of symmetric Laplacians, and none of them carry automatically to a quotient of a DAG, which need not even be acyclic (2.1.2). That absence of transferable guarantees is one reason every method run here is compared against a random control at matched compression (3.2.3) instead of being trusted on guarantees stated for a different object.

Reduction in Circuit and Netlist Domains

Two distinct bodies of work sit in this space, and they must not be conflated. The first is AIG-native reduction that predates and ignores the summarization literature. Structural hashing merges structurally identical gates on construction [Kuehlmann et al., 2002], FRAIG merges functionally equivalent ones by simulation and SAT [Mishchenko et al., 2005], and MFFC decomposition is how refactoring already carves a network into mergeable cones [Mishchenko et al., 2006]. Read through the lens of 2.1.5, all three are equivalence-based coarsenings by another name, and connecting them to the exactness framework of 2.1.4 is part of this thesis's contribution.

The second is CTS-Bench [Khadka et al., 2026], the nearest published competitor. It benchmarks coarsening trade-offs for GNNs on post-placement gate-level netlists for clock-skew prediction. Its coarsening is a single bespoke heuristic that con-

sumes physical coordinates, so it cannot be applied to a pre-physical AIG at all (2.1.1), and it compares no second method. Its findings support the premise of this thesis rather than competing with it: up to 17.2× memory reduction and 3× training speedup, accuracy collapsing to negative R^2 under zero-shot cross-state evaluation, and an explicit call for domain-aware coarsening. Two of its numbers are carried forward. The negative zero-shot R^2 is the standing warning for RQ5, and the dissociation between its error metrics, MAE nearly unmoved at 0.16 to 0.17 while R^2 fell below zero, is the citation behind reporting RMSE, R^2 and Spearman correlation together rather than any one alone (3.4.3).

[TODO: overlap with optimizaiton should be mentioned]

2.2.5 Generalization Across Graph Distributions

RQ5 trains on one structural distribution and evaluates on another, so its plausibility rests on what is known about GNNs under such shift.

Size and Distribution Shift in GNNs

Message-passing weights are shared across nodes and layers, so a trained model accepts a graph of any size, a point the inductive line established in principle [Hamilton et al., 2017]. Accepting an input is not generalising to it: Buffelli et al. [2022] show accuracy degrading when test graphs are much larger than training graphs, and propose regularising toward size-invariant representations. Train-reduced, infer-full is therefore plausible rather than guaranteed, which is precisely the posture RQ5 takes.

Train-Test Structural Mismatch

The direct evidence is one positive and one negative result. SCAL [Huang et al., 2021] trains on coarsened graphs and evaluates on the originals with usable accuracy on node-level benchmarks. CTS-Bench [Khadka et al., 2026] does the analogous cross-state evaluation on netlists and lands below $R^2 = 0$ (2.2.4). Together they make RQ5 an open question rather than a formality, and they are why the study carries a positive control, the provably lossless compression of 3.1.5, without which a poor cross-state result could not be told apart from a broken evaluation path (1.2.5).

[TODO: related work shoudl be much more subtly connected to RQ and scope. also is this necessary? if so why and how much is needed. shoudl eb related work on this cross trainign i do like RGCN paper in Documents/ThesisPapers]

2.2.6 Research Gap

The two halves surveyed above do not meet. Circuit representation learning scales its models and quietly reduces its inputs by training on extracted subcircuits, without ever naming or measuring the reduction (2.2.1). Graph reduction offers a taxonomy of measured methods, none of which has been evaluated on AIGs, on a logic-synthesis target, or under the directed, inversion-carrying constraints of 2.1.2. The one published bridge, CTS-Bench [Khadka et al., 2026], sits one design stage later, runs one spatial heuristic that cannot reach a pre-physical AIG, and reports a cross-state failure it attributes to the absence of exactly the domain-aware coarsening studied here.

After CTS-Bench, being first to coarsen in EDA cannot be claimed unqualified, and the claim made here is narrower. This thesis is the first to (i) benchmark multiple coarsening families, exact refinement-based, GNN-aware, and domain-specific, against a compression-matched random control for GNN training in EDA, (ii) do so on AIGs and optimizability regression, (iii) bring provably lossless compression into an EDA GNN setting, (iv) connect AIG-native equivalence reduction, structural hashing, FRAIG and MFFC decomposition, to the generic summarization framework, and (v) run a controlled three-protocol split comparison with per-design error distributions on this task, where the surveyed literature reports one protocol at a time (2.2.1). Spectral and hashing-based coarsening are surveyed in 2.2.3 but not run, and are no part of the claim. The eventual results are compared against SynthNet [Chowdhury et al., 2021], HOGA [Deng et al., 2024] and DeepGate4 [TODO, 2025] on the prediction task, and against CTS-Bench [Khadka et al., 2026] on the shape of the memory, speed and accuracy trade-off.

[TODO: more concise sentences but more thorough poitn by point where the related work is gapping in unseen IP, GNN scaling for AIGs]

[TODO: i think the ordering of these is off. gap at end. ml4eda first. idk seems off somehow]

Chapter 3

Methodology

3.1 Architecture

[TODO: make more concise also to evaluate what to put in the appendix and what to keep here write that as a permanent note so my thesis supervisor can see so not a ocmment] The encoder follows GNN+ [Luo et al., 2025]; hyperparameters come from the literature and from the tuning sweep of 3.4.2. One architecture is used for unreduced, sparsified, partitioned and summarized graphs alike, with two exceptions stated below: partitioning changes the pooling path (3.1.2), and the exact-compression track changes the input schema (3.1.5).

The encoder is an edge-aware graph convolutional network over AIGs. A node carries a one-hot type indicator over [constant, PI, AND, PO], so an AND gate is $[0, 0, 1, 0]$, and an edge carries a one-hot indicator of whether it is inverted. The one departure from that schema is the summarized track, where a node is a super-node and its vector counts the members it absorbed: a super-node holding 2 primary inputs, 15 AND gates and 1 primary output is $[0, 2, 15, 1]$, and a super-edge counts the inverted and non-inverted wires merged into it. A one-hot vector is the count vector of a single-member super-node, so both graph types occupy the same four-dimensional input space and are read by the same encoder weights (3.2.3). The remaining settings are given in 3.1 and derived in 3.1.1.

Table 3.1: Encoder configuration, used unchanged for unreduced, sparsified, partitioned and summarized graphs. The two exceptions are the pooling path under partitioning and the input schema of the exact-compression track, described in Sections 3.1.2 and 3.1.5.

Component	Setting
Node features	4-dimensional type indicator (member counts when summarized)
Edge features	2-dimensional inversion indicator
Positional encoding	Log logic level, projected to 32 dimensions
Input width	160 (128-dimensional node projection concatenated with the encoding), normalised over each graph before the first layer
Depth	4 message-passing layers
Hidden width	128, constant across layers
Activation	LeakyReLU in the encoder blocks, ReLU in the regression head
Feed-forward block	$128 \to 256 \to 128$
Normalisation	Layer normalisation in graph mode, bracketing the feed-forward block
Residuals	Skip connection added after message passing in every layer
Dropout	0.15 in the encoder blocks, 0.3 in the regression head
Degree normalisation	Disabled
Readout	Jumping knowledge summed over the 4 layers, with mean pooling
Regression head	$128 \to 64 \to 1$ with a terminal sigmoid

3.1.1 Encoder Formulation

Nothing in this subsection is a contribution of this thesis. The departures below are GNN+'s [Luo et al., 2025], each cited to the equation of that paper it comes from; they are written out because the reduction claims of 3.1.4 and 3.1.5 are claims about *this* forward pass, and cannot be checked against a bulleted configuration. The settings specific to this work are collected at the end of the subsection. The classic GCN layer [Kipf and Welling, 2017] that GNN+ modifies is

$$h_v^l = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\hat{d}_u \hat{d}_v}}\, h_u^{l-1} W^l\right). \tag{3.1}$$

Throughout, h_v^l is the representation of node v after layer l, e_{uv} the attribute of edge (u, v), σ the activation, $\|$ concatenation, and biases are suppressed for readability.

Edge-conditioned messages. Each edge attribute is projected by its own weight W_e^l and fused with the neighbour representation, so the message a node receives depends on the wire it arrived on:

$$m_v^l = \sum_{u \in \mathcal{N}(v)} c_{uv}\, \sigma\big(h_u^{l-1} W^l + e_{uv} W_e^l\big). \tag{3.2}$$

Placing σ on each message rather than on the aggregate follows the GNN+ reference implementation; its Eq. 6 writes the activation outside the sum. The per-edge coefficient c_{uv} is the one place (3.2) is configurable rather than fixed. With symmetric degree normalisation enabled it is the GCN coefficient $1/\sqrt{\hat{d}_u \hat{d}_v}$ of (3.1), computed once per edge when graphs are cached rather than at every forward pass; disabled, it is 1 and no edge-weight tensor is carried at all. It is disabled here. The same edge-weight channel is what the exact-compression track of 3.1.5 reuses: setting c_{uv} to the edge multiplicity w_{uv} scales the message *outside* σ, which is what makes one merged message equal the sum of the messages it replaces.

Normalisation, dropout, residual and feed-forward block. GNN+ adds four of its six techniques inside the layer block:

$$\tilde{h}_v^l = h_v^{l-1} + \mathrm{Drop}\big(\sigma(m_v^l)\big), \tag{3.3}$$

$$\bar{h}_v^l = \mathrm{Norm}\Big(\tilde{h}_v^l\Big), \tag{3.4}$$

$$h_v^l = \mathrm{Norm}\big(\bar{h}_v^l + \mathrm{FFN}(\bar{h}_v^l)\big), \tag{3.5}$$

$$\mathrm{FFN}(h) = \mathrm{Drop}\big(\mathrm{Drop}\big(\sigma(h W_1^l)\big)\, W_2^l\big), \tag{3.6}$$

with W_1^l: $128 \to 256$ and W_2^l: $256 \to 128$. The published layer carries one normalisation that (3.3) does not. Its Eqs. 7–9 normalise the aggregated message *before* the activation and before the residual is added, and its Eq. 10 closes the FFN by normalising that block's own residual sum; the GNN+ reference implementation keeps both and inserts a third ahead of the FFN, the bracketing arrangement of GraphGPS [Rampásek et al., 2022]. Only the message normalisation is dropped here. The two dropouts of (3.6) likewise follow the reference implementation, which Eq. 10 does not show.

Positional encoding. A per-node structural scalar, here the log-compressed logic level ℓ_v, is projected and concatenated to the node features rather than summed into them:

$$h_v^0 = \mathrm{GraphNorm}\big(x_v W_{\text{in}} \,\|\, \sigma(\ell_v W_{\text{PE}})\big)\,, \tag{3.7}$$

with $x_v \in \mathbb{R}^4$, $W_{\text{in}}\colon 4 \to 128$ and $W_{\text{PE}}\colon 1 \to 32$, so $h_v^0 \in \mathbb{R}^{160}$. GNN+ holds its concatenation at the model width, projecting the joined vector back down in Eq. 12 and narrowing the node projection beforehand in its implementation. Keeping all 160 dimensions here means the first layer maps $160 \to 128$ and is therefore the one layer whose residual in (3.3) is skipped, its input and output widths being unequal.

Configuration specific to this work. The encoder is GNN+; the following are this project's configuration of it, and none of them change the technique:

- **Degree normalisation off.** $c_{uv} = 1$ in (3.2), where GCN+ carries the $1/\sqrt{\hat{d}_u \hat{d}_v}$ of (3.1) into its own layer. Fan-in in an AIG is fixed by node type, so the coefficient, taken over fan-in and without the self-loop, is constant within each type pair and is zero on every edge leaving a primary input. GNN+ supplies no evidence either way: its ablations [Luo et al., 2025, Tables 5–6] cover the six techniques it adds and not the GCN coefficient it inherits, and its own edge-aware layer disables both that coefficient and the self-loop, so this setting agrees with the reference implementation where the implementation departs from Eq. 6. The normalised variant is implemented and reachable through the model configuration, not exposed as a training flag.
- **No self-loop.** Aggregation in (3.2) runs over $\mathcal{N}(v)$ without the $\cup\{v\}$ of (3.1); a node's own state re-enters through the residual of (3.3) only.
- **LeakyReLU for** σ in the encoder, and layer normalisation in graph mode for Norm, taking statistics over all nodes of one graph, rather than the batch normalisation used throughout the GNN+ ablations.
- **Level-based positional encoding** in (3.7), a single learned projection of log logic depth, where GNN+ uses a Random Walk Structural Encoding (RWSE). Level comes from a single topological pass, where RWSE costs one sparse matrix product per walk step on graphs of this size and adds the walk length to a tuning budget already committed to the reduction parameters.
- **Readout and head.** GNN+ reads out from the last layer alone; here the layer outputs are summed first (jumping knowledge over all $L = 4$ layers), then mean-pooled and passed through a bounded head:

$$h_G = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \sum_{l=1}^{L} h_v^l, \tag{3.8}$$

$$\hat{y} = \mathrm{Sigmoid}\big(\mathrm{Drop}_{0.3}(\mathrm{ReLU}(h_G W_3))\, W_4\big)\,, \tag{3.9}$$

with $W_3\colon 128 \to 64$ and $W_4\colon 64 \to 1$. The terminal sigmoid matches the $[0, 1]$ range of the optimizability target, and is the reason the head uses ReLU and a heavier dropout ($p = 0.3$) than the $p = 0.15$ of the encoder blocks.

3.1.2 Partitioned Inputs

Partitioning is the one reduction family that requires an architectural change. Graphs are partitioned offline and the edges spanning partitions are removed, which leaves the node set intact but disconnects the graph into k components. Pooling directly to the graph level would then average over components the encoder never allowed to communicate, so the readout is made hierarchical:

- GNN encoder (applied on intra-partition edges only)
- Mean pooling from node representations to partition representations
- Mean pooling from partition representations to a single graph representation
- Prediction head applied to the graph representation to produce the output $\hat{y}$

Feature normalization is also made partition-aware: the input GraphNorm normalizes within each partition rather than within each graph, so statistics do not leak across a boundary the message passing itself cannot cross.

3.1.3 Sparsified Inputs

Sparsification alters only the data pipeline: a sparsified AIG preserves the feature schema of the original, so the encoder of 3.1 ingests it unmodified.

Reductions are stored as node and edge masks applied at load time rather than materialised into the corpus, so a single corpus serves every sparsification setting. Edge masks (random edge dropout, spanning forest) leave the node count unchanged, whereas node masks (PageRank pruning, AND-gate-only) reduce it. The node budget governing batch assembly (3.4.2) is therefore measured on the post-mask count.

3.1.4 Summarized Inputs

Summarized graphs are also fed to the unmodified encoder, which is possible only because the merge preserves the input schema as a superset: node and edge features become member counts rather than one-hot indicators, and a super-node containing a single node reproduces its original feature row exactly (3.2.3). Positional encodings are pooled by member minimum, so a super-node enters the circuit at the level of its earliest member.

A full graph is consequently a valid input to a summary-trained model without preprocessing, being a graph of size-1 super-nodes. The cross-state inference of RQ5 therefore evaluates one set of weights on both representations.

3.1.5 Encoder Variant for Exact Colour Refinement

Colour refinement at count-cap ∞ (3.2.3) is exact rather than approximate, but the guarantee holds only under a modified encoder.

When k nodes merge, parallel edges collapse into one super-edge whose message must equal the sum of the k it replaces. Multiplicity w_{uv} carried as an edge attribute enters σ in (3.2), which does not decompose over sums. Setting $c_{uv} = w_{uv}$ instead scales the message outside σ and gives equality. Edge attributes are therefore dropped, and inversion moves onto the nodes: fan-in is fixed by node type in an AIG (zero for constants and primary inputs, two for AND gates, one for primary outputs), so a node's type and its count of inverted incoming edges determine which fan-ins are inverted, up to order. Which specific fan-in is inverted is lost, which a graph-level target does not need.

Coarsening to depth d equal to the number of message-passing layers then yields outputs identical to the uncoarsened graph [Bollen et al., 2023]: compression that costs no accuracy by construction, and the positive control for RQ5 (3.4.1). The variant pays for this in schema compatibility. A full graph must be converted before this encoder can be queried on it, whereas the other methods support cross-state inference with no such step.

[TODO: go over this when comparins to exact sumamrizaiotn related work]

3.1.6 Baselines for Optimizability Prediction

RQ1 is a comparative claim and needs a reference. Two groups supply one: trivial predictors, which fix what a given R^2 is worth on this label, and three published circuit models ported to this task. A third group of standard graph-level encoders (GCN, GraphSAGE, GIN) was considered and may be added if time permits.

Trivial Predictors

Three predictors, each fitted on the validation split and scored on test, so that every baseline is scored on data it has not seen. Two are constant,

$$\hat{y}_i^{\text{mean}} = \bar{y}_{\text{val}}, \qquad \hat{y}_i^{\text{med}} = \text{med}(y_{\text{val}}) \,, \tag{3.10}$$

and the third is an ordinary least squares fit on graph size alone,

$$\hat{y}_i^{\text{size}} = \text{clip}_{[0,1]}(\beta_0 + \beta_1 \log_{10} |\mathcal{V}(G_i)| + \beta_2 \log_{10} |\mathcal{E}(G_i)|) \,. \tag{3.11}$$

Equation (3.10) fixes the zero point of (3.29). The test-set mean is the least-squares optimal constant, so any other constant scores $R^2 \leq 0$, with equality only at $\bar{y}_{\text{val}} = \bar{y}_{\text{test}}$. Equation (3.11) is the weakest non-trivial hypothesis about the task, that optimizability is a function of size alone, and any claim that the encoder reads circuit *structure* has to clear it.

[TODO: evaluate which of these i will actulaly do shoudl be the base scores the means and all that for comparatability and compared to random and CIs] [TODO: is ranodmly intialized GNN a baseline? i think so but i need to check the literature]

Adapting Published Circuit Models

SynthNet [Chowdhury et al., 2021], HOGA [Deng et al., 2024] and DeepGate4 [TODO, 2025] are introduced in 2.2.1. None was published for this task, so each had to be adapted, and what follows states those adaptations rather than the models.

Shared porting procedure. Each paper supplies an encoder g_ϕ carrying an AIG to node states $\{h_v\}_{v \in \mathcal{V}(G)}$, together with a head supervising that paper's own target. The encoder is reused unmodified and the head is replaced, so every port has the form

$$\hat{y}(G) = \text{Sigmoid}\big(\text{MLP}\big(\pi\big(\{h_v\}_{v \in \mathcal{V}(G)}\big)\big)\big) \,, \tag{3.12}$$

with π a permutation-invariant readout and the terminal sigmoid matching the $[0, 1]$ range of the target of 2.1.6. Only π, the head and the input adapter differ across the three, and the trunk of (3.12) is in every case the published one.

Whether π is inherited or chosen differs by model, and the distinction bears on what each comparison licenses. SynthNet already predicts a graph-level quantity, so π is its own concatenated maximum and mean pooling. HOGA classifies nodes and DeepGate4 pools over cones; neither defines a whole-circuit readout, so for both π is mean pooling and is this work's choice, not the paper's.

Adaptations required by a graph-level target. Three, each a consequence of the target being one scalar per circuit under one fixed script:

- **The recipe branch is removed.** SynthNet predicts $\hat{y}(G, s)$, conditioning on a synthesis recipe s through a separate encoder. Training here fixes the script, so s is constant across the corpus and that branch emits one vector for every sample, contributing a bias to the head and nothing else. It is dropped rather than left to learn that constant.
- **Batches are bounded by nodes, not graphs.** A graph-level readout requires all of $\mathcal{V}(G)$ in one forward pass, so a graph cannot be split across batches, and neither published batch size transfers: HOGA minibatches nodes drawn from one graph, DeepGate4 minibatches cones. Batches are assembled to a node budget B and paired with an accumulation count a, and it is the product that matters. One graph contributes one label regardless of its size, so the number of losses averaged per update is $a \cdot \mathbb{E}[\, B/|\mathcal{V}(G)|\,]$, held near the primary model's value. Budget and accumulation are tuned and reported as a pair.
- **Activations are recomputed rather than retained.** DeepGate4 attends over a virtual edge set $\bar{\mathcal{E}}_k = \{(u, v) : \text{dist}(u, v) \leq k\}$, which at the published radius $k = 8$ measures roughly 180 edges per circuit node against an AIG fanin of at most two. Each of its attention layers materialises a message tensor over $\bar{\mathcal{E}}_k$, so recomputing them during the backward pass, which leaves outputs and gradients unchanged, is what makes the baseline runnable at all.

Corrections to the released implementations. Two departures are corrections rather than adaptations, and are stated because they change published behaviour. HOGA's released attention normalises its scores over the head axis rather than the key axis, so the weights never form a distribution over hop neighbours; the port normalises over keys, following Eq. 5 of the paper. Its optional reverse-propagation branch applies the forward adjacency to the reverse features, which duplicates the forward branch rather than reversing it, and the port applies the reverse adjacency. A third difference is a convention, not an error:

OpenABC-D orients edges toward the primary inputs and this corpus orients them the other way, so each node summarises its fanout cone under one convention and its fanin cone under the other. Both orientations are run and the pair reported. **[TODO: i will provavly not report both just the proper one the correct one that was originally used in paper]**

Comparability. Only the data pipeline is shared, since a different split would make the comparison meaningless. Hyperparameters, optimizer, loss and schedule take each paper's published values. Settings a paper leaves unpublished are recorded as assumptions in A.3.

Scores printed in those papers are a different matter and cannot be placed beside the ones here. OpenABC-D standardises its targets within each design d, so its reference spread is the within-design sum of squares where the denominator of (3.29) is the total,

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{here}} = \underbrace{\sum_i \left(y_i - \bar{y}_{d(i)}\right)^2}_{\text{upstream}} + \sum_d n_d \left(\bar{y}_d - \bar{y}\right)^2 . \quad (3.13)$$

The two differ by the between-design term, which on a design-disjoint split (3.4.2) is the variation the task is about. Only the sign of the score transfers (4.1.4). **[TODO: do i need this?]**

[TODO: this shoudl really be mostly on how i adapted my baselines ot the task. so recipe was not inputted i cut off that branch]

3.2 Graph Reduction

This section describes the reduction methods evaluated in this thesis. Their objective is to relieve the memory bottleneck that prevents training on unreduced AIGs and to shorten training time, while preserving the structural features on which accurate optimizability prediction depends.

3.2.1 Partitioning

Partitioning splits a graph into blocks small enough to process, keeping every node and discarding only the edges that cross blocks. A k-way partition of an AIG $G = (\mathcal{V}, \mathcal{E})$ is a map $\pi \colon \mathcal{V} \to \{1, \dots, k\}$ with blocks $\mathcal{V}_i = \pi^{-1}(i)$, and the reduced graph keeps exactly the edges internal to a block,

$$R_\pi(G) = \big(\mathcal{V},\, \mathcal{E} \backslash \mathcal{E}_{\text{cut}}(\pi)\big), \qquad \mathcal{E}_{\text{cut}}(\pi) = \{\, (u,v) \in \mathcal{E} : \pi(u) \neq \pi(v) \,\} \quad (3.14)$$

the disjoint union of the k induced subgraphs. Node retention is $\eta_{\mathcal{V}} = 1$ by construction, so the only quantity a partitioner controls is the edge retention $\eta_{\mathcal{E}} = 1 - |\mathcal{E}_{\text{cut}}(\pi)|/|\mathcal{E}|$ of (2.13). All blocks of a graph are kept and enter the model together; the hierarchical pooling path that reads them is described in 3.1.2.

The number of blocks is set per graph from a target block size s,

$$k(G) = \min\Big\{ k_{\max},\ \max\big\{ k_{\min},\ \lfloor |\mathcal{V}|/s \rfloor \big\} \Big\}, \quad (3.15)$$

with $s = 10{,}000$ and $[k_{\min}, k_{\max}] = [2, 32]$, so blocks hold roughly s nodes until the cap binds. The four methods below receive the same $k(G)$ on every graph and differ only in the assignment π, which is what makes them directly comparable.

Random Hashing

Each node is assigned a block independently and uniformly at random, $\pi(v) \sim \text{Uniform}\{1, \dots, k\}$, from a fixed seed. The assignment ignores the edge set entirely, so each edge survives with probability $1/k$ and

$$\mathbb{E}\big[\eta_{\mathcal{E}}(R_\pi(G))\big] = \frac{1}{k}, \quad (3.16)$$

independently of the graph. This is the floor of the family: a structure-aware method justifies its cost only by retaining more than this fraction. Blocks are balanced only in expectation, but the deviation is negligible at the block sizes implied by (3.15).

METIS

METIS [Karypis and Kumar, 1998] approximates the balanced minimum-cut partition

$$\min_\pi \big|\mathcal{E}_{\text{cut}}(\pi)\big| \quad \text{subject to} \quad \max_i |\mathcal{V}_i| \le (1+\varepsilon) \left\lceil \frac{|\mathcal{V}|}{k} \right\rceil , \quad (3.17)$$

a problem that is NP-hard, with a multilevel heuristic: the graph is coarsened by successive matchings, the coarsest graph is partitioned directly, and the partition is projected back through the levels with local refinement at each step. It operates on the undirected projection of the AIG, so edge direction plays no part in where the cuts fall. The objective also counts every cut edge equally: severing a connection between adjacent levels costs the same as severing an edge that bridges half the circuit's depth. That is precisely the assumption the domain-informed variant below modifies.

Level Slicing

Nodes are ordered by topological level (2.5) and the ordering is cut into k blocks of equal cardinality. Writing $\text{rank}(v) \in \{0, \dots, |\mathcal{V}| - 1\}$ for the position of v in a stable sort by $\ell(v)$,

$$\pi(v) = 1 + \left\lfloor \frac{k \cdot \text{rank}(v)}{|\mathcal{V}|} \right\rfloor . \quad (3.18)$$

Balance is exact by construction, and blocks are contiguous in depth: $\ell(u) < \ell(v)$ implies $\pi(u) \le \pi(v)$. Because every edge runs from a lower level to a strictly higher one, an edge is cut exactly when its endpoints straddle one of the $k-1$ block boundaries. The local directed structure of each depth band therefore survives intact; the price is that every path from the inputs to the outputs crosses every boundary and is severed $k - 1$ times.

Span-Weighted METIS

The domain-aware adaptation changes only the objective of METIS. Each edge $e = (u, v)$ has a span

$$\text{sp}(e) = \ell(v) - \ell(u) \ge 1, \qquad w(e) = 1 + \gamma \cdot \text{sp}(e), \quad (3.19)$$

the number of levels it crosses, and the weighted cut $\sum_{e\in\mathcal{E}_{\text{cut}}(\pi)} w(e)$ is minimised under the same balance constraint as (3.17), with $\gamma = 10$. A long edge is a shortcut between distant levels, and cutting it removes gates from the fanin cones (2.6) of everything downstream, which is exactly the damage the uniform objective is indifferent to. Under (3.19) the cost of a cut grows linearly with the length of the chain it breaks, so the minimiser pushes cuts toward span-one edges and the longest causal chains stay whole.

[TODO: make sure the methods mentioned in related work are defined formulaically there and here is jus the adaptations i had ot make or how they are applied]

3.2.2 Sparsification

Sparsification reduces graph density by selectively removing edges or nodes while attempting to preserve overall structural properties [Hashemi et al., 2024]. Two of the four methods below act on the edge set alone, replacing $\mathcal{E}$ with a subset $\mathcal{E}' \subseteq \mathcal{E}$ and leaving $\eta_{\mathcal{V}} = 1$. The other two select a node subset $\mathcal{V}' \subseteq \mathcal{V}$ and return the induced subgraph $R(G) = G[\mathcal{V}']$, so every edge with an endpoint outside $\mathcal{V}'$ is removed as well. That difference determines which retention figure is meaningful for each method, and it is why 2.1.5 insists on reporting node and edge retention separately rather than as one compression number.

Random Edge Dropout

This naive baseline uniformly drops a fixed random fraction of edges across the graph, acting as a control for structural degradation. Each edge draws an independent uniform variate and survives when the draw clears the drop rate p,

$$\mathcal{E}' = \{\, e \in \mathcal{E} : U_e \geq p \,\}, \qquad U_e \overset{\text{i.i.d.}}{\sim} \text{Unif}[0,1], \tag{3.20}$$

so each edge is kept independently with probability $1-p$ and $\mathbb{E}[\eta_{\mathcal{E}}] = 1-p$. Random edge removal is established as a per-epoch regulariser for deep Graph Neural Network (GNN) training [Rong et al., 2020]. Here the mask of (3.20) is instead drawn once and fixed, which makes it a reduction rather than a regulariser. The drop rate is a freely tunable compression parameter, which is what makes this method usable as a matched-compression control against the parameter-free methods below. The sweep sets $p = 0.3$, for a measured edge retention of 69.7%.

AND-Gate Retention

This domain-specific method removes the circuit interface and keeps the logic. In the node role notation of (2.3),

$$\mathcal{V}' = \{\, v \in \mathcal{V} : \tau(v) \notin \{\text{PI}, \text{PO}\} \,\}, \tag{3.21}$$

so all Primary Input (PI) and Primary Output (PO) nodes are dropped with every edge that touches them, and the internal AND network survives intact. The rationale is that the PI/PO boundary is fixed by the circuit's interface and cannot be restructured by synthesis, so the AND network carries the optimizable structure. The method is parameter-free: its compression is whatever fraction of the graph the interface happens to occupy (measured 82.1% node and 73.3% edge retention), not a value that can be dialled.

Random Spanning Forest

This method extracts a sparse connectivity backbone. Let $\tilde{G}$ be the undirected projection of the AIG, and assign each edge an independent weight $w_e \sim \text{Unif}[0,1]$. The method keeps the minimum-weight spanning forest

$$F = \operatorname*{arg\,min}_{F' \in \mathfrak{F}(\tilde{G})} \sum_{e \in F'} w_e, \qquad |\mathcal{E}(F)| = |\mathcal{V}| - c, \tag{3.22}$$

where $\mathfrak{F}(\tilde{G})$ is the set of spanning forests of $\tilde{G}$ and c the number of connected components, computed with Kruskal's algorithm [Kruskal, 1956]. The weights are almost surely distinct, so each draw has a unique minimiser and the randomised weights select a random forest rather than a deterministic one. A forest contains no cycles. Every reconvergent path therefore loses an edge, while connectivity is kept intact. Because F is computed on $\tilde{G}$, edge direction plays no part in which edges survive. That is a real loss of information on a DAG, and one reason to expect this method to damage the predictive signal.

The edge count in (3.22) also makes the method parameter-free: $|\mathcal{V}| - c$ edges is a property of the graph rather than a setting, and the measured edge retention of 58.1% follows from the density of the corpus.

This method replaced a stretch-bounded spanner [Peleg and Schäffer, 1989], which was implemented and abandoned. A t-spanner keeps a subgraph $H \subseteq \tilde{G}$ with $d_H(u,v) \leq t \cdot d_{\tilde{G}}(u,v)$ for every node pair, so an edge can be discarded only when an alternative path of length at most t survives. Spanners exploit dense short-range redundancy, and AIGs have little of it: the reconvergence they are rich in runs through long paths, so at stretch $t = 3$ the randomised construction of Baswana and Sen [2007] returned the graph nearly unchanged. That negative result is worth reporting: it is a small, concrete instance of the thesis's general claim that generic graph machinery does not transfer to AIGs unexamined.

PageRank Node Pruning

This centrality-based approach scores every node by PageRank and retains only the highest-scoring fraction, implicitly dropping all edges incident to the removed low-centrality nodes. The score is the stationary solution of

$$\text{pr}(v) = \frac{1-\alpha}{|\mathcal{V}|} + \alpha \sum_{u \in \text{fi}(v)} \frac{\text{pr}(u)}{|\,\text{fo}(u)|}, \tag{3.23}$$

computed on the directed graph with damping factor $\alpha = 0.85$ [Brin and Page, 1998, Langville and Meyer, 2003], with fanin and fanout as in (2.2) and the mass of the POs, whose fanout is empty, redistributed uniformly [Langville and Meyer, 2003]. The kept set $\mathcal{V}'$ collects the $\lceil \kappa\,|\mathcal{V}| \rceil$ nodes of highest score.

The retention fraction κ is a parameter, which is what allows this method to be matched against AND-gate retention: $\kappa = 0.8$ was chosen to sit at that method's measured 82.1% node retention, making the two a directly comparable domain-informed / generic pair (4.4.1).

[TODO: the retention values and such go in the end when evaluating i think.] [TODO: i originally did .3 i

think for random edge dropout since it was close to the and gate only... need jsutification for all these hyperparameters for all the reduction methods]

3.2.3 Summarization and Coarsening

These methods compress the graph by merging nodes into super-nodes. Unlike the other two families, which only delete, summarization must decide what a merged node *is*, so the shared rewrite that turns a clustering into a graph is described first (3.2.3), and each method below then reduces to the single question of which nodes it groups together.

Three methods are run, spanning domain-specific, general state of the art, and exact, so that RQ4 has something to compare. A fourth, random merging, is a control rather than a contender; it is what makes matched-compression comparison possible. Table 3.2 lists the set.

Table 3.2: Summarization methods.

Method	Role	Compression
Domain-specific coarsening (TBD)	domain-specific	method-dependent
Colour refinement	adapted, exact	fixed
ConvMatch	GNN-aware SOTA	target ratio
Random within-type	control / matching	exact

Merge Map and Super-Node Schema

Every method produces only a *clustering*: an assignment of each node to a group. One shared rewrite turns that clustering into a graph, so that methods differ in what they merge and in nothing else. Given a clustering, the rewrite:

- sets each super-node's features to the *member type counts* $[\#\text{const}, \#\text{PI}, \#\text{AND}, \#\text{PO}]$;
- remaps every edge to its endpoints' clusters, discards the resulting self-loops, and sums the inversion counts of parallel super-edges so that a super-edge records how many normal and how many inverted connections it stands for;
- pools the positional encoding by member *minimum*, so a super-node sits at the level of its earliest member;
- records the number of discarded intra-cluster edges, and the preserved primary input and output counts, as graph attributes.

Schema compatibility. The count representation is a strict superset of the one-hot representation: a super-node with one member reproduces its original feature row exactly. An unreduced AIG is therefore already a valid input in the summarized schema, without conversion. This is the property that makes RQ5 testable with a single set of weights, and it is a design constraint on every method rather than a convenience: a merge that produced features outside this space would put summary and full graphs in different input spaces and make cross-state inference undefined.

Boundary preservation. No method merges a primary input or primary output into a super-node with a different role. The PI/PO boundary is the circuit's fixed interface, and dissolving it changes what the graph represents rather than merely coarsening it.

Domain-Specific Coarsening

The AIG-native slot. Two candidates are implemented; which one is run is not yet decided, and the decision waits on measured compression and retention on the real corpus.

- **Cone coarsening.** Merges along two independent axes: chains of single-fanout gates contract into one super-node, up to a capped chain length, reducing circuit *depth*; and gates within a bounded band of topological levels that share a common immediate *post*-dominator merge together, compressing parallel width while leaving critical-path depth untouched, so the level-based positional encoding stays exact. The band width and the chain cap are the only compression controls any domain-specific candidate offers.
- **MFFC skeleton coarsening.** Contracts each *maximal fanout-free cone* into one super-node: every AND gate whose fanout lies entirely inside one cluster joins that cluster, iterated to a fixed point. What survives is the multi-fanout sharing skeleton plus the fixed PI/PO interface. It is parameter-free, so its compression is a property of the circuit and cannot be dialled.

The two are related rather than rival: chain contraction is the capped, chain-shaped special case of MFFC absorption, which additionally captures cones whose interior reconverges.

MFFC coarsening invites an obvious objection, and the answer is the thesis claim in miniature. MFFCs are prime targets for synthesis rewriting, since refactoring collapses and re-expresses one MFFC at a time, so merging them might be expected to destroy exactly the signal the model must read. The claim here is the opposite, and it is falsifiable: intra-cone wiring is what local rewriting is *free to change*, so it is noise with respect to the label, whereas the sharing structure that *constrains* what rewriting can achieve is what survives the merge.

Both candidates preserve acyclicity by the same argument: a cluster is absorbed only into the single cluster receiving all of its outgoing edges, which cannot close a cycle in a DAG. For cone coarsening this holds at band width zero, where the width axis merges only within a level; wider bands give up both the guarantee and the exactness of the level encoding.

Graded Colour Refinement

The exact anchor of the study. Nodes are merged when they agree under d rounds of colour refinement, with d set equal to the encoder's message-passing depth so that merged nodes are exactly those the network cannot distinguish. A count-cap c interpolates between two named endpoints [Bollen et al., 2023]: at $c = \infty$ refinement distinguishes nodes by the full multiset of neighbour colours and the merge is *lossless* for a summing

GNN; at $c = 1$ it sees only the set of neighbour colours, which is bisimulation: coarser, cheaper, and lossy for a counting model.

Two AIG adaptations are required. Node colours are initialised from the four-way type and the level encoding rather than uniformly, so nodes at different depths never share a class and the pooled positional encoding of a super-node is exact. **Edge inversion is treated as a distinct relation** so that a normal and an inverted connection never contribute to the same colour class. Without this, logically distinct subgraphs merge. Refinement runs backward from the primary outputs by default; direction is a parameter, and it changes both how much compresses and which structure survives.

At $c = \infty$ this method is orthogonal to the optimization being predicted by construction: it removes only what the model provably cannot see, so it cannot erase the label. That is what makes it the positive control for RQ5 rather than merely another contender, and realising the guarantee end to end requires the schema of 3.1.5.

The obvious risk is that structural hashing has already removed the easy equivalences (2.1.3), since every corpus graph is strashed. What this method finds is therefore *residual* redundancy beyond one hop, and how much of it exists is itself a reportable statistic about AIGs.

Convolution Matching

The strongest general-purpose competitor: ConvMatch [Dickens et al., 2024] merges nodes that are equivalent with respect to the *graph convolution operation* itself rather than with respect to a graph property. As for every other method, merges never cross node types. The restriction is imposed on the candidate set, so the convolution objective itself is unmodified.

It is the bar the domain-specific method must clear, and it is the right bar precisely because it is GNN-aware but domain-blind: if an AIG-specific method beats it, the domain claim is earned against a method optimising the same objective, not against a strawman.

Its behaviour on a deep DAG with inverted edges is untested in the literature, and one property found here explains why it may differ from the published setting. Its merge cost is an unweighted sum over the convolution output, so merging two low-degree nodes scores better than merging two genuine convolution twins of higher degree, regardless of similarity. Convolution equivalence therefore decides *between* pairs of comparable degree, not across them, which on a graph with the degree profile of an AIG biases merging toward the input frontier.

Random Within-Type Merging

Uniformly random nodes of the same type are merged until the node count reaches a target. Merges are restricted to one node type so that the comparison isolates *which* nodes are merged rather than confounding it with whether the interface survives, matching the boundary guarantee of every other method.

This serves two purposes, and the second is the important one.

As a **naive floor**, it answers the question a reader will ask first: if a sophisticated method does not beat random merging at the same compression, its sophistication bought nothing. It is also the exact counterpart of random edge dropout in the sparsification family, so both halves of the study have a floor constructed the same way.

As a **matching instrument**, it is what makes RQ4 answerable at all. Compression is a free parameter for only some methods: ConvMatch takes a target ratio directly, cone coarsening offers only coarse integer knobs, and MFFC coarsening and colour refinement have no compression parameter whatsoever. There is therefore no general way to make two *real* methods meet at the same ratio. Random merging reaches any target exactly by construction, so each method is instead compared against a random control run at that method's own achieved compression. The paired design replaces a matching problem that has no solution.

[TODO: completely look at and redo by hand comment out all this text and lets start with the basic concept i am plannign to implement. keep the text but commented out]

3.3 Data

[TODO: lets]

3.3.1 Source Circuits

The corpus is built from 55 source circuits, drawn from three collections and summarised in 3.3. The characteristics of each design, measured on its untransformed base AIG, are given in 3.4.

Twenty-nine are the hardware IP designs of OpenABC-D [Chowdhury et al., 2021], which collects them in turn from OpenCores, IWLS, the MIT Lincoln Laboratory CEP and OpenPiton. They span bus and peripheral control (`spi`, `simple_spi`, `i2c`, `sasc`, `ss_pcm`, `usb_phy`, `wb_dma`, `wb_conmax`, `ac97_ctrl`, `pci`, `mem_ctrl`, `ethernet`, `vga_lcd`), cryptography (`aes`, `aes_xcrypt`, `aes_secworks`, `des3_area`, `sha256`), signal processing (`fir`, `iir`, `dft`, `idft`, `jpeg`), and processors and SoC fragments (`tv80`, `tinyRocket` `bp_be`, `picosoc`, `fpu`, `dynamic_node`).

All 47 non-synthetic designs entered the pipeline as BENCH netlists from the OpenABC-D distribution, obtained from the NYU UltraViolet repository record (https://ultraviolet.library.nyu.edu/records/mw6q2-a8p15) linked in the project's code repository (https://github.com/NYU-MLDA/OpenABC). Its bench directory carries the classical benchmarks of the following paragraph alongside the 29 IP designs the dataset paper documents. Each netlist is converted to an AIG by a single ABC pass that reads it and applies structural hashing, which is the only preprocessing applied before the transformation of 3.3.2.

Eighteen more are classical combinational benchmarks that OpenABC-D does not include: the eight arithmetic circuits of the EPFL suite [Amarú et al., 2015] (`div`, `hyp`, `log2`, `max`, `multiplier`, `sin`, `sqrt`, `square`), four ISCAS-85 circuits [Brglez and Fujiwara, 1985] (`c1355`, `c5315`, `c6288`, `c7552`), and six MCNC/LGSynth circuits [Yang, 1991] (`apex1`, `bc0`, `dalu`, `i10`, `k2`, `mainpla`). These extend the corpus into deep arithmetic, which OpenABC-D deliberately excludes in favour of larger and functionally more di-

verse IP. The effect on the depth distribution is pronounced. Median depth across the 47 designs outside this suite is 31 levels and the deepest of them is `fpu` at 819, whereas the EPFL circuits reach 4,373 (`div`), 5,059 (`sqrt`) and 24,802 (`hyp`). The three deepest circuits in the corpus therefore all enter through this collection, and they set the bound on the precomputed depth encoding of 3.3.5.

The remaining eight are synthetic circuits, named `128`, `256`, `512`, `1024`, `2048`, `4096`, `8192` and `16384` after their *primary input count* rather than their size: `16384` has 16,384 inputs but 131,072 AND gates, so the name understates the circuit by an order of magnitude. They are randomly sampled AIGs, produced by the random AIG generator of `mockturtle`, part of the EPFL logic synthesis libraries [Soeken et al., 2018]. The generator takes a number of primary inputs, a number of AND gates and a seed, and grows the network one gate at a time. It draws two operands uniformly at random from the signals built so far, namely the constant, the primary inputs and every gate already created, inverts each with probability one half, and adds the resulting AND gate to the pool. Draws that structurally hash to an existing node or collapse to a trivial case do not count towards the gate budget and are redrawn, so every gate is distinct. Once the budget is met, every node still without a fanout is made a primary output. The gate budget is eight times the input count in every one of the eight, from 128 inputs / 1,024 gates up to 16,384 inputs / 131,072 gates, doubling at each step. The number of primary outputs is not a parameter but a consequence of the sampling and lands between 37% and 39% of the gate count (398 for the smallest, 49,307 for the largest): it is the share of nodes never drawn as an operand.

The generation script itself was not kept, so this account is reconstructed from the eight AIGs rather than from a recipe, and they determine it tightly. Across all eight there is not one duplicated or trivial AND gate, the gate count is exactly eight times the input count, and the primary outputs are exactly the fanout-free nodes together with the unused constant, which the generator emits as the first output, a detail of that implementation and of no other. What is *not* recoverable is the seed. The consequences of that are taken up in 3.4.4.

That construction is worth stating precisely, because it explains how these eight behave later. A network whose fanins are drawn uniformly at random from the whole pool, with duplicates rejected as they are created, arrives already structurally hashed and with almost none of the repeated local subfunctions that `rewrite`, `refactor` and `resub` exist to collapse. Their near-zero optimizability (3.3.4) is therefore a property of how they were sampled rather than a measurement failure, and it is the largest circuits, where the pool is widest and collisions rarest, that sit closest to an exact fixed point.

[TODO: if in table text could mayeb be reduced? comment out the original text and rewrite more concise outline style]

The license column of 3.3 records each suite's own terms, but that is not the term that governs the copies actually used, because the 18 classical benchmarks were not obtained from their original distributions. Every non-synthetic netlist entered the corpus through the OpenABC-D release, so those copies were received under its BSD 3-Clause license. The EPFL suite's MIT license applies in addition, and the ISCAS-85 and MCNC circuits attach no formal terms of their own, leaving the BSD 3-Clause license of the redistributing dataset as the only written term on the copies used. Any release of the corpus is therefore governed by BSD 3-Clause attribution requirements, retaining the MIT notice for the EPFL circuits (5.6).

Table 3.3: Where the 55 source designs come from, and under what terms. The license column gives each suite's own terms; the ISCAS-85 and MCNC circuits predate modern license practice and carry no formal terms. The terms governing the copies actually used are stated in the text.

Source	Designs	License
OpenABC-D [Chowdhury et al., 2021]	29	BSD 3-Clause
EPFL combinational suite [Amarú et al., 2015]	8	MIT
ISCAS-85 [Brglez and Fujiwara, 1985]	4	none stated
MCNC / LGSynth [Yang, 1991]	6	none stated
Synthetic (this work)	8	n/a
	55	

The provenance is the point here, not the inventory, because the design-level split (3.4.2) rests on the assumption that these 55 are genuinely distinct circuits rather than variants of one another. Mostly they are: three different sources, four benchmark traditions, and no design derived from another. The assumption is not perfect, and the exceptions should be named rather than assumed away. The three AES variants implement the same cipher; `dft` and `idft` are forward and inverse transforms of one structure, as are `fir` and `iir` to a lesser degree; and the eight synthetic circuits differ from each other only in scale. A split by design does not guarantee that such a family lands entirely on one side of it, so a small amount of family-level similarity can cross the train/test boundary even though no individual circuit does. This is a weaker form of leakage than the one design-level splitting exists to close, being a structural family resemblance rather than the same circuit in two optimization states, but it bounds how strongly an unseen-design result can be read and is taken up in 5.4.

Scale spans nearly three orders of magnitude, which is what forces the node-budget batching of 3.4.2. The base AIGs range from 403 AND nodes (`ss_pcm`) to 245,046 (`dft`) at a median of 11,464, and depth ranges from 8 levels to 24,802 at a median of 35. Transformation and optimization move these figures, so the bounds the preprocessing pipeline records over the full corpus are higher still, at 366,040 AND nodes and a depth of 24,972.

Stats

[TODO: mention all the depths, nodes distributions and etc. for the chosen starting circuits. rename this to something more formal]

3.3.2 Dataset Generation

Random Structural Transformation

Each source design is expanded by applying random synthesis recipes to it and retaining every intermediate result, following the construction of OpenABC-D [Chowdhury et al., 2021]. A

Table 3.4: Characteristics of the 55 source circuits before any transformation. Primary inputs (PI), primary outputs (PO), AND nodes (N), edges (E), inverted edges (I) and depth (D), measured on the base AIG of each design. Every AND node has two fanins and every output one, so $E = 2N + PO$ throughout. Rows are grouped by function and ordered by size within each group, and colour repeats that grouping: Communication and bus protocol, Controller, Cryptography, Signal processing, Processor and SoC, Arithmetic, Combinational logic, Synthetic random.

Design	Source	PI	PO	N	E	I	D	Function
ss_pcm	OpenABC-D	104	90	403	896	462	8	Single slot PCM
usb_phy	OpenABC-D	132	90	487	1,064	513	10	USB PHY 1.1
sasc	OpenABC-D	135	125	613	1,351	788	9	Simple asynchronous serial controller
simple_spi	OpenABC-D	164	132	930	1,992	1,084	12	MC68HC11E based SPI interface
i2c	OpenABC-D	177	128	1,169	2,466	1,188	15	Bidirectional serial bus protocol
spi	OpenABC-D	254	238	4,219	8,676	5,524	35	Serial peripheral interface
wb_dma	OpenABC-D	828	702	4,587	9,876	4,768	29	Wishbone DMA/bridge
pci	OpenABC-D	3,429	3,157	19,547	42,251	25,719	29	PCI controller
wb_conmax	OpenABC-D	2,122	2,075	47,840	97,755	42,138	24	Wishbone Conmax
ethernet	OpenABC-D	10,731	10,422	67,164	144,750	86,799	34	Ethernet IP core
ac97_ctrl	OpenABC-D	2,339	2,137	11,464	25,065	14,326	11	Wishbone AC97 controller
mem_ctrl	OpenABC-D	1,187	962	18,092	37,146	16,307	36	Wishbone memory controller
vga_lcd	OpenABC-D	17,322	17,063	105,334	227,731	141,037	23	Wishbone VGA/LCD controller
des3_area	OpenABC-D	303	64	4,971	10,006	4,686	30	DES3 encrypt/decrypt
sha256	OpenABC-D	1,943	1,042	15,816	32,674	18,459	76	SHA-256 hash
aes	OpenABC-D	683	529	28,925	58,379	20,494	27	AES (LUT-based)
aes_secworks	OpenABC-D	3,087	2,604	40,778	84,160	45,391	42	AES-128 (simple)
aes_xcrypt	OpenABC-D	1,975	1,805	45,840	93,485	36,180	43	AES-128/192/256
fir	OpenABC-D	410	351	4,558	9,467	5,696	47	FIR filter
iir	OpenABC-D	494	441	6,978	14,397	8,596	73	IIR filter
jpeg	OpenABC-D	4,962	4,789	114,771	234,331	146,080	40	JPEG encoder
idft	OpenABC-D	37,603	37,419	241,552	520,523	317,210	43	Inverse discrete Fourier transform
dft	OpenABC-D	37,597	37,417	245,046	527,509	322,206	43	Discrete Fourier transform
tv80	OpenABC-D	636	361	11,328	23,017	11,653	54	TV80 8-bit microprocessor
dynamic_node	OpenABC-D	2,708	2,575	18,094	38,763	23,377	33	OpenPiton NoC architecture
fpu	OpenABC-D	632	409	29,623	59,655	37,142	819	OpenSPARC T1 floating point unit
tinyRocket	OpenABC-D	4,561	4,181	52,315	108,811	67,410	80	32-bit tiny RISC-V core
bp_be	OpenABC-D	11,592	8,413	82,514	173,441	109,608	86	BlackParrot RISC-V execution engine
picosoc	OpenABC-D	11,302	10,797	82,945	176,687	106,737	43	SoC with PicoRV32 RISC-V
c6288	ISCAS-85	32	32	2,337	4,706	4,132	121	16×16 multiplier
max	EPFL	512	130	2,865	5,860	3,629	288	Maximum
sin	EPFL	24	25	5,416	10,857	6,085	226	Sine
square	EPFL	64	128	18,485	37,098	23,439	251	Square
sqrt	EPFL	128	64	24,618	49,300	36,581	5,059	Square root
multiplier	EPFL	128	128	27,062	54,252	32,086	275	Multiplier
log2	EPFL	32	32	32,060	64,152	36,784	445	Base-2 logarithm
div	EPFL	128	128	57,247	114,622	87,301	4,373	Divider
hyp	EPFL	256	128	214,335	428,798	277,636	24,802	Hypotenuse
c1355	ISCAS-85	41	32	512	1,056	658	30	32-bit error-correcting circuit
apex1	MCNC	45	45	1,578	3,201	1,082	15	PLA-derived logic
bc0	MCNC	26	11	1,592	3,195	1,958	32	PLA-derived logic
c5315	ISCAS-85	178	123	1,613	3,349	1,743	39	ALU and selector
dalu	MCNC	75	16	1,735	3,486	1,963	36	Datapath ALU
c7552	ISCAS-85	207	107	2,198	4,503	2,692	31	ALU and control
i10	MCNC	257	224	2,273	4,770	2,323	59	Combinational logic
k2	MCNC	45	45	2,290	4,625	1,784	23	PLA-derived logic
mainpla	MCNC	27	54	5,346	10,746	7,172	39	PLA-derived logic
128	Synthetic	128	398	1,024	2,446	1,024	13	Random AIG, 128 inputs
256	Synthetic	256	773	2,048	4,869	2,038	14	Random AIG, 256 inputs
512	Synthetic	512	1,572	4,096	9,764	4,122	17	Random AIG, 512 inputs
1024	Synthetic	1,024	3,085	8,192	19,469	8,216	16	Random AIG, 1024 inputs
2048	Synthetic	2,048	6,195	16,384	38,963	16,484	15	Random AIG, 2048 inputs
4096	Synthetic	4,096	12,231	32,768	77,767	32,906	16	Random AIG, 4096 inputs
8192	Synthetic	8,192	24,624	65,536	155,696	65,302	17	Random AIG, 8192 inputs
16384	Synthetic	16,384	49,307	131,072	311,451	130,909	18	Random AIG, 16384 inputs

recipe is a sequence of 20 transformations drawn independently and uniformly from seven area-oriented passes: balancing, and rewriting, refactoring and resubstitution each in a plain and a zero-cost variant, the latter accepting replacements that leave node count unchanged. The same 200 recipes, the first 200 of the 1,500 released with OpenABC-D and all of them distinct, are applied to every design, so recipe identity is not confounded with design identity.

The AIG is written after *every* pass rather than only at the end, so a single recipe contributes 20 graphs, one per prefix of the recipe, plus one further graph after the structural-choice pass that follows the transformation sequence: 21 in total. Together with the unmodified design itself this gives $1+200\times21 = 4{,}201$ base graphs per design, and $55 \times 4{,}201 = 231{,}055$ overall. The recipes continue into a technology-mapping tail, which does not execute because no standard-cell library is loaded in this flow, so no graph beyond the structural-choice pass is produced. The per-design count is verified rather than assumed, and generation is treated as failed unless exactly $4{,}201$ AIGs are present. The pass alphabet, the recipe format and the unexecuted tail are listed in A.1.

Randomised transformation rather than plain replication is what makes this corpus a learning problem at all. Replicating a design would yield identical graphs carrying identical labels, adding weight but no information. A random recipe prefix instead yields a circuit that is *functionally* equivalent to its source, since every pass is equivalence-preserving, but structurally different, and, critically, differently optimizable: how much a synthesis script can still remove depends on which rewriting the graph has already absorbed. The corpus therefore varies along exactly the axis the label measures while holding the function fixed.

That is also what makes the design-level split (3.4.2) necessary rather than merely conservative. The $4{,}201$ graphs derived from one design are not independent samples: they share a Boolean function, a primary-input interface, and a large amount of common structure, and successive prefixes of one recipe differ by a single pass. A random split over graphs would place near-siblings of every test graph into training, and the resulting score would measure recall of a design the model had already seen rather than generalization. Splitting on the design partitions the corpus along the one boundary this construction never crosses.

[TODO: TODO: step-21 choice networks are not plain AIGs; the node count understates them in 43 of 200 cases. this was a mistake i dont want to retrain can this be added in limitation insteead do imention this here? i will just pul lit from inference values at least]

Optimization

Four synthesis scripts were applied during dataset generation: `Orchestrate`, `Deepsyn`, `Syn4` and `C2RS`. They differ in how they search: some apply a fixed sequence of rewriting, refactoring and balancing passes, while others explore seeded or scored variations. Command templates and per-algorithm parameters are given in A.

Only `Orchestrate` is used for training and evaluation in this thesis (1.2.6); the remaining three exist on disk and are the most direct extension available (6.3.1).

3.3.3 Tiered Dataset Structure

The corpus has two tiers of *stored* graphs. Every stored graph is a training example; the optimized results used to label them are not themselves retained, for the reason given in 3.3.3.

Tier 0: Base Graphs

This originates from 55 designs randomly transformed into 231,055 base graphs (Tier-0).

Tier 1: Single-Algorithm Optimization

Applying the target synthesis algorithms to these base graphs yields exactly 231,055 primary (Tier-1) samples per algorithm. For the Orchestrate prediction task three of them contribute samples (`Deepsyn`, `Syn4` and `C2RS`), while `Orchestrate` itself does not: an Orchestrate-produced graph relabelled by Orchestrate is precisely the sequential step that 3.3.3 stops at, so it is a label source rather than a sample.

Labelling and Tier Depth

Labelling a graph means running the target script on it and recording how many nodes it removed (3.3.4). The optimized *output* of that run is a graph too, so the construction could be iterated indefinitely; it is stopped after one step, and the outputs of the labelling runs on tier-1 graphs are discarded rather than promoted to a third tier.

Two consequences are worth stating, because both shape the results.

First, the two tiers are not independent of each other. Optimizing a tier-0 graph with algorithm S produces both that graph's label under S *and* the corresponding tier-1 graph: one synthesis run serves both purposes. The tier-1 set is therefore exactly the set of labelling outputs of tier 0.

Second, this is what bounds how optimizable the corpus is. A tier-1 graph has already been through one full synthesis script, so what Orchestrate can still remove from it is small, though how small depends far more sharply on *which* script ran first than on the tier itself (3.6), to the point that the tier ordering inverts and tier 1 carries the higher mean target. That decomposition is taken up in 3.3.6. Roughly half the evaluation corpus has an optimizability of exactly zero (3.4). That is a property of the construction rather than a defect, but every R^2 in 4 has to be read against it (3.3.4).

3.3.4 Target Label: Optimizability

Definition

For an AIG G and synthesis algorithm S, the optimizability target is the relative node reduction $y(G)$ of (2.14), formulated as a regression task, and is not restated here. The bound motivates

the terminal sigmoid in the prediction head (3.1): the model cannot predict outside the achievable range.

Note what this does and does not measure. It is a *relative* quantity, so it is comparable across circuits of different sizes, which is what makes a single model over a heterogeneous corpus meaningful, but it is also therefore harder to predict than an absolute count, since the model must infer both what will be removed and what the graph started with. It is an area proxy and says nothing about delay or power (1.2.6).

Label Distribution

The target is heavily concentrated near zero (mean 0.020, standard deviation 0.053, and just under half of all graphs at exactly zero), and what determines it is the synthesis script a graph came from rather than the circuit or the tier. Both facts constrain how the results can be read, and both are set out with the measurements in 3.3.6 and 3.3.6 rather than repeated here.

3.3.5 Graph Representation

Node and Edge Features

Each AIG is represented as an attributed graph carrying one-hot node types (constant, primary input, AND, primary output), a one-hot edge inversion flag (regular connection or inverted signal), and a topological depth encoding. The AIGER files are read and traversed with AIGVERSE [Walter, 2025], an interface to the same EPFL logic synthesis libraries [Soeken et al., 2018].

Positional Encoding Precomputation

Topological levels are computed once, offline, and stored on the graph; the model reads the stored attribute rather than recomputing it. This matters for more than speed. Levels are computed on the *unreduced* graph and then carried through reduction, so a partitioned or sparsified graph retains the depth information of the circuit it came from even where the edges that establish that depth have been cut. A runtime encoding would instead describe the reduced graph's own topology, which is a different and, for this task, less informative quantity.

3.3.6 Dataset Analysis

This subsection characterises the corpus the model is actually trained and evaluated on: its scale, its structure, the shape of the target, and how the two evaluation splits differ from one another. Several decisions made elsewhere in this chapter rest on the distributions reported here rather than on argument: the graph-scale bound of 3.4.2, the node-budget batching of 3.4.2, the terminal sigmoid of 3.1. Every R^2 in 4 has to be read against them.

One scope caveat applies throughout, and it is not a small one. Except for 3.8, which covers every stored graph, what follows is measured on the *evaluation splits*, 96,430 test graphs over six designs and 80,525 validation graphs over five, rather than over the full corpus of 3.3.3. The splits are design-disjoint samples of the same construction, so the structural statistics can be read as representative; the label statistics cannot, because which designs land in a split changes them substantially (3.3.6).

Scale and Structure

Graph size spans nearly three orders of magnitude within the evaluation splits alone: a median of 15,977 nodes against a maximum of 232,715 (3.1, 3.5). This is the distribution that forces node-budget batching rather than a fixed graph count per batch (3.4.2); at a fixed batch size the largest graphs in the corpus are four orders of magnitude more expensive than the smallest.

Structurally the graphs are far more uniform than they are in size (3.2, 3.7). AND gates and the constant account for 82.1% of nodes on average and the primary-input/output interface for the remaining 17.9%, and the graphs are sparse and near-tree-like at 1.75 edges per node. The AND-gate share is not a descriptive statistic here but a mechanism: the `and_gate_only` sparsification of 3.2.2 drops exactly the interface nodes, so 82.1% *is* its node retention, and the spread on that share (63.3% to 99.9% across graphs) is why its compression varies between circuits rather than being a dial that can be set.

Measured Target Distribution

The target is extremely concentrated near zero (3.3, 3.5). Pooled over the test split the mean optimizability is 0.020 with a standard deviation of 0.053, and 73.4% of graphs sit below $y = 0.005$. Just under half, 48.8%, are *exactly* zero: Orchestrate removes nothing at all.

Two consequences follow, and both shape how the results chapter can be read. First, R^2 is a ratio against the target's own variance, and that variance is small, so a respectable RMSE is entirely compatible with a near-zero or negative R^2; the baseline diagnosis of 3.1.6 turns on precisely this. Second, with a point mass of this size at zero, a constant predictor is a strong baseline, and any model has to beat it on the minority of graphs where something is actually removable.

Where that point mass sits is not uniform (3.4). It concentrates in the smallest circuits, which are already minimal, and in the synthetic designs: the design `16384` contributes $16{,}080$ test graphs, a sixth of the split, with a mean target of 0.0000, a maximum of 0.0003 and 92.4% of graphs at exactly zero. Orchestrate is at a fixed point on it, for the reasons given in 3.3.1. The joint distribution of size and label (3.5) is the entire signal available to the size-only baseline of 4.1.1, and it shows why that baseline is not trivial: graph size is largely a proxy for "is this optimizable at all".

Determinants of the Target

The strongest single determinant of the target is not the circuit but the synthesis script that produced the graph (3.6). A tier-1 graph has already been optimized once, and how much Orchestrate can still remove depends overwhelmingly on which script ran first. Pooled over both evaluation splits, Syn4-derived graphs have a mean optimizability of 0.046 with only 11.7% at exactly zero, while Deepsyn- and C2RS-derived graphs average 0.0022 and 0.0011 with 54.4% and 52.1% at zero. That is a

twenty- to forty-fold difference in the target, driven by a property of the label pipeline rather than of the circuit.

This is also what explains an apparent anomaly in 3.5: tier-1 graphs have a *higher* mean target (0.023) than tier-0 graphs (0.010), although 3.3.3 argues that a graph which has already been through a full synthesis script should have less left to remove. The elevation is entirely attributable to one of tier 1's three sources. On the test split the three source means are 0.0015 (C2RS), 0.0025 (Deepsyn) and 0.0654 (Syn4), whose average is 0.023, exactly the tier-1 figure. Excluding the Syn4 third, tier 1 averages 0.0020, five times *below* tier 0. The tier is therefore the weaker cut of the data, and the source script should be reported alongside it wherever tier-wise results appear.

Composition of the Evaluation Splits

Under a design-level split the test set is a handful of whole circuits rather than a random sample (3.7), so every pooled test metric is an average over six designs and their particular mix. With one of those six contributing a sixth of the graphs at a constant-zero target, the pooled figures are sensitive to composition in a way that a random split would have hidden.

Validation and test are correspondingly not exchangeable (3.8, 3.6). They are disjoint circuit sets, and they differ on every axis that matters: 26.2% of validation graphs sit at $y = 0$ against 48.8% of test, the test maximum target is more than twice the validation maximum (0.274 against 0.114), and the validation graphs are larger at the median (29,399 nodes against $15{,}977$). The divergence is sharpest for the Syn4 subset, where not one validation graph has a target of exactly zero against 21.4% of the corresponding test graphs. This is expected under a design-level split rather than a defect, but it is the direct explanation for the validation-to-test gap in 4.28, and it bounds how far validation-based model selection can be assumed to transfer.

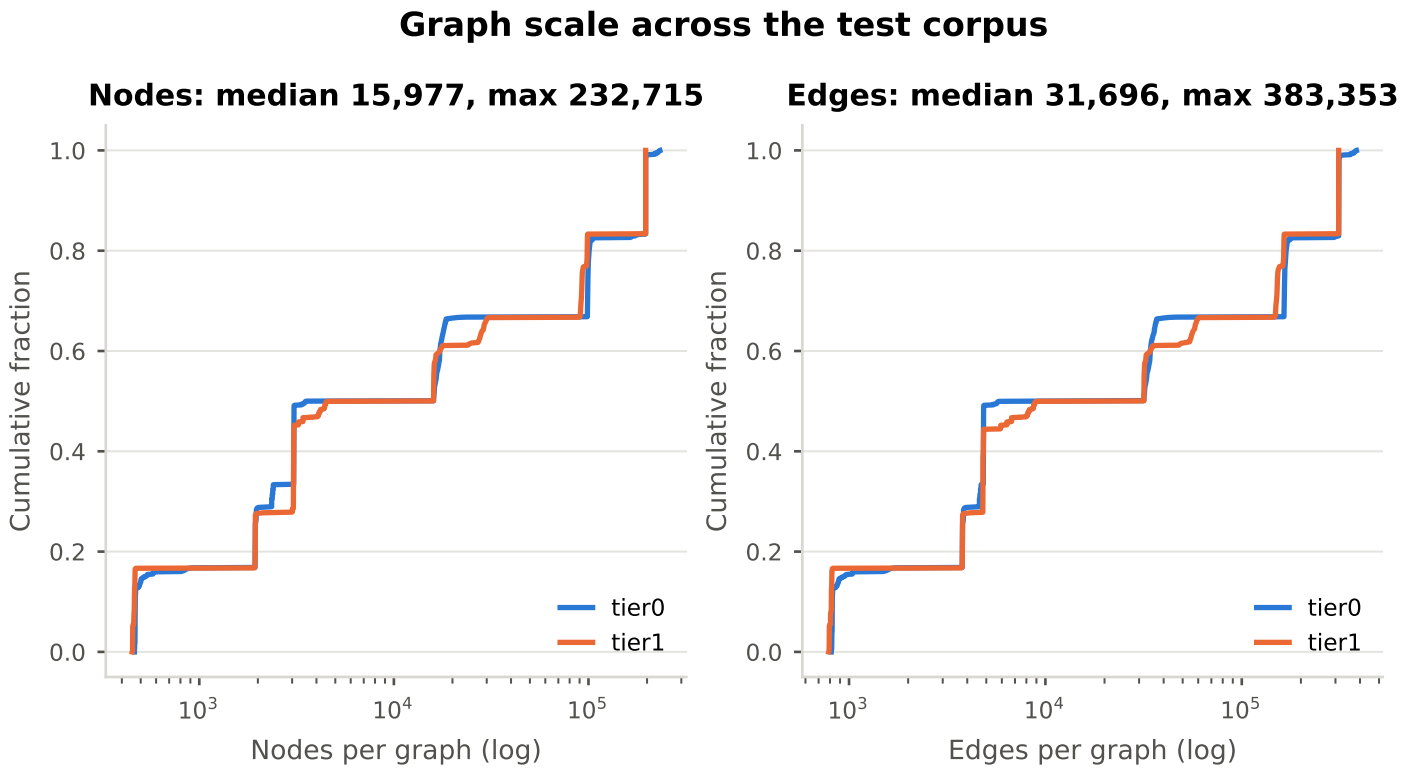

Figure 3.1: Node and edge count distributions across the evaluation corpus, by tier. This is the distribution behind the graph-scale bound of 3.4.2 and the node-budget batch sizes of 3.4.2.

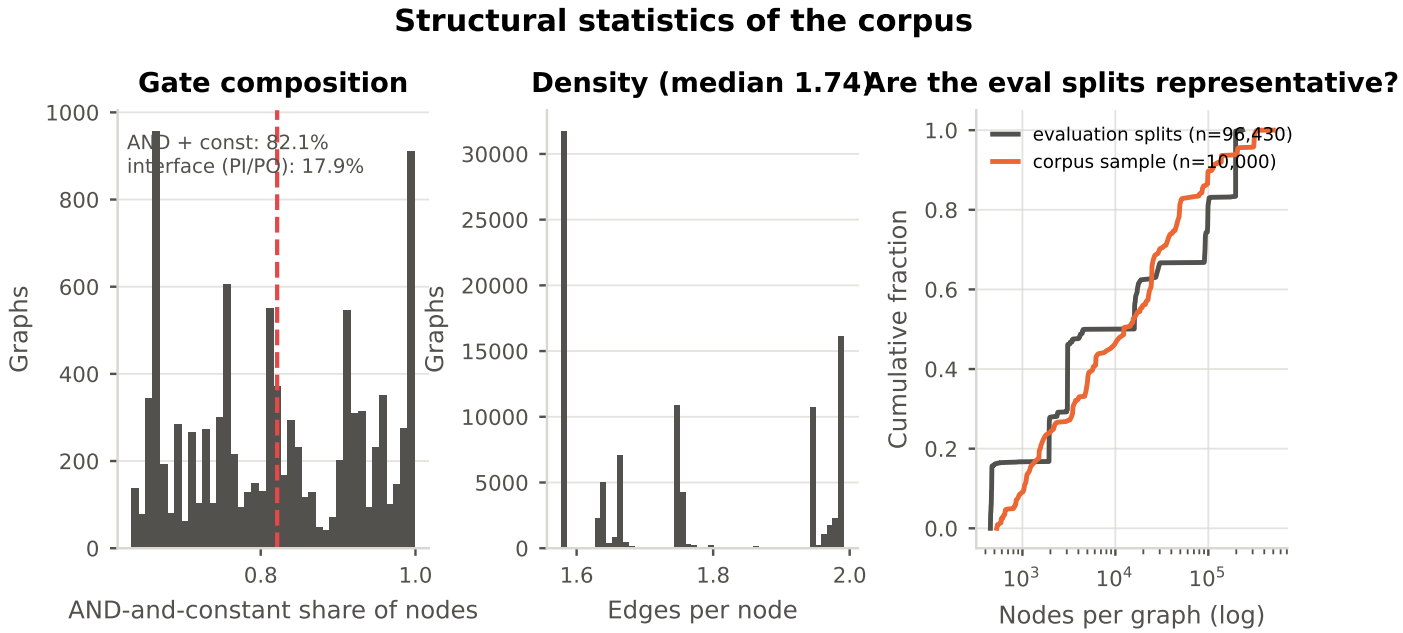

Figure 3.2: Structural statistics recoverable without a pass over the graph cache. The AND-gate share is read off the AND-gate-only node masks, which drop exactly the primary inputs and outputs: 82.1% of nodes are AND gates or constants and 17.9% are interface. That figure *explains* that method's compression rather than merely accompanying it (3.2.2). The right panel checks that the evaluation splits are representative of the wider corpus sample on size.

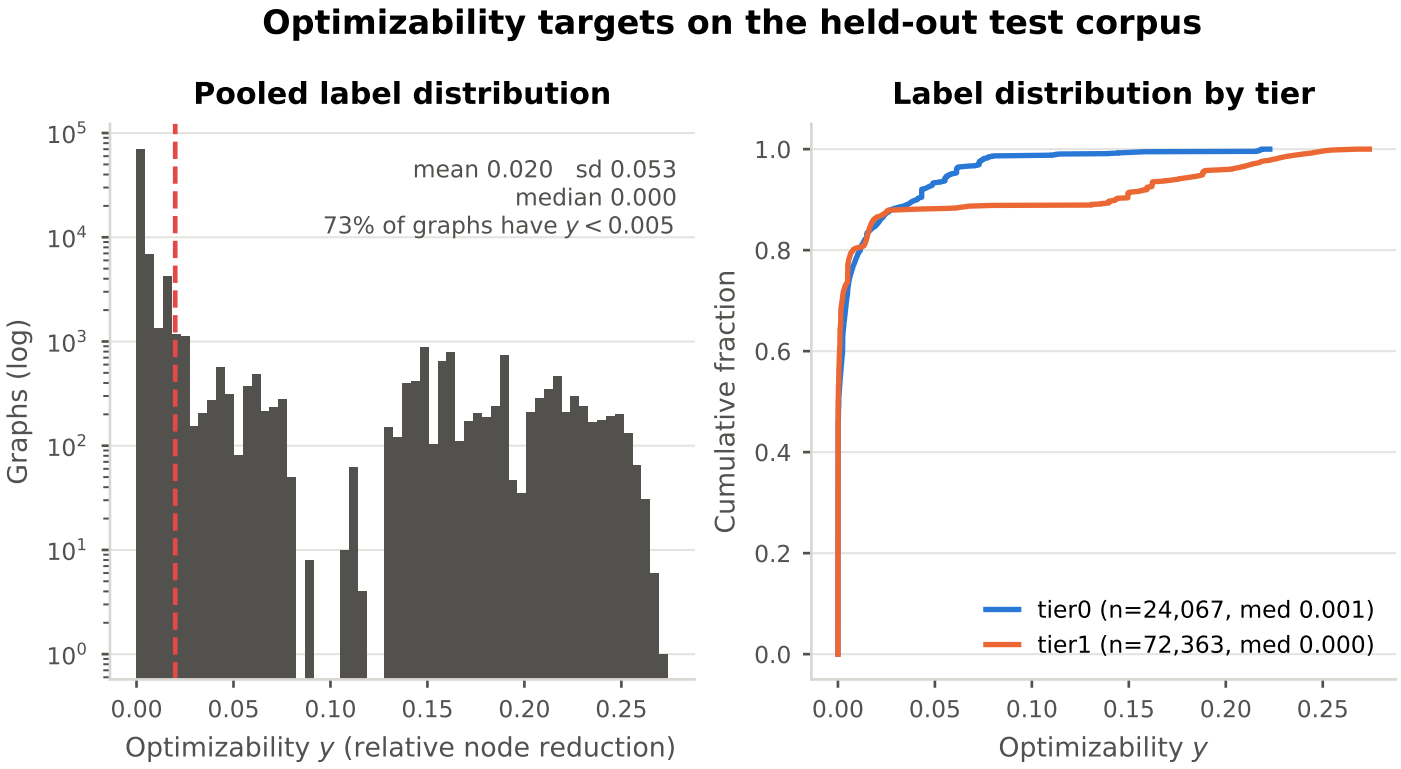

Figure 3.3: Distribution of the optimizability target on the held-out test corpus, pooled (log counts) and per tier. The label is concentrated near zero: the great majority of graphs sit below $y = 0.005$. This is the variance every R^2 in 4 is measured against, and it is why a respectable RMSE is compatible with a near-zero or negative R^2.

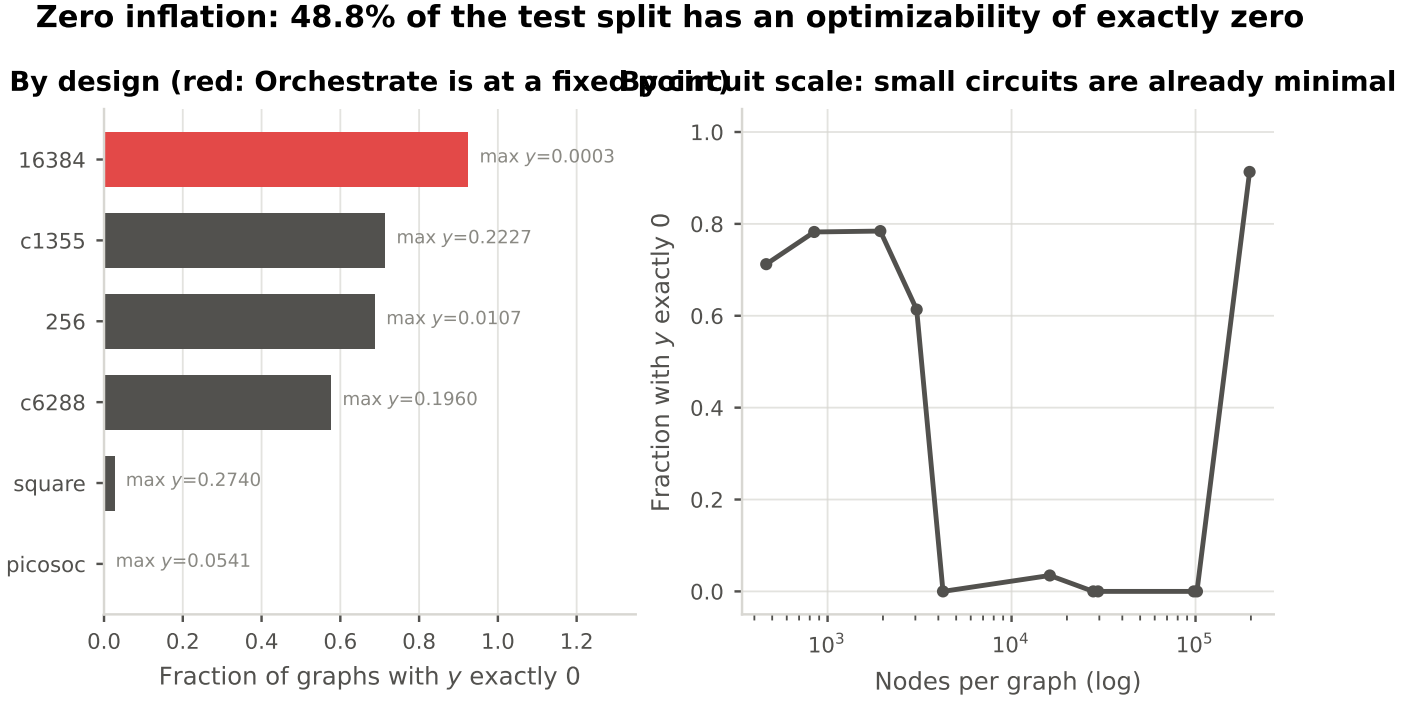

Figure 3.4: Where the point mass at exactly zero sits. Roughly half the test split has an optimizability of exactly zero, concentrated in particular designs (16384 and 8192 reach a maximum y of 0.0003) and in the smallest circuits, which are already minimal. This is the figure the size-only baseline of 4.1.1 has to be read against: graph size is largely a proxy for "is this optimizable at all".

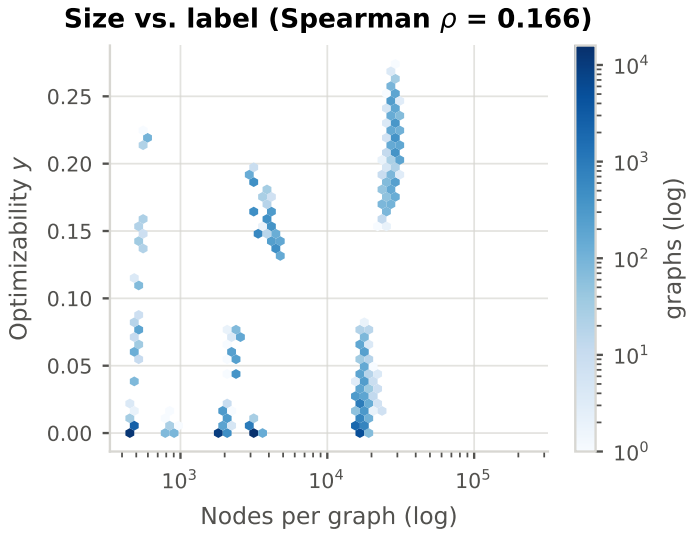

Figure 3.5: Joint density of graph size and optimizability. This is the entire signal available to the size-only baseline of 4.1.1.

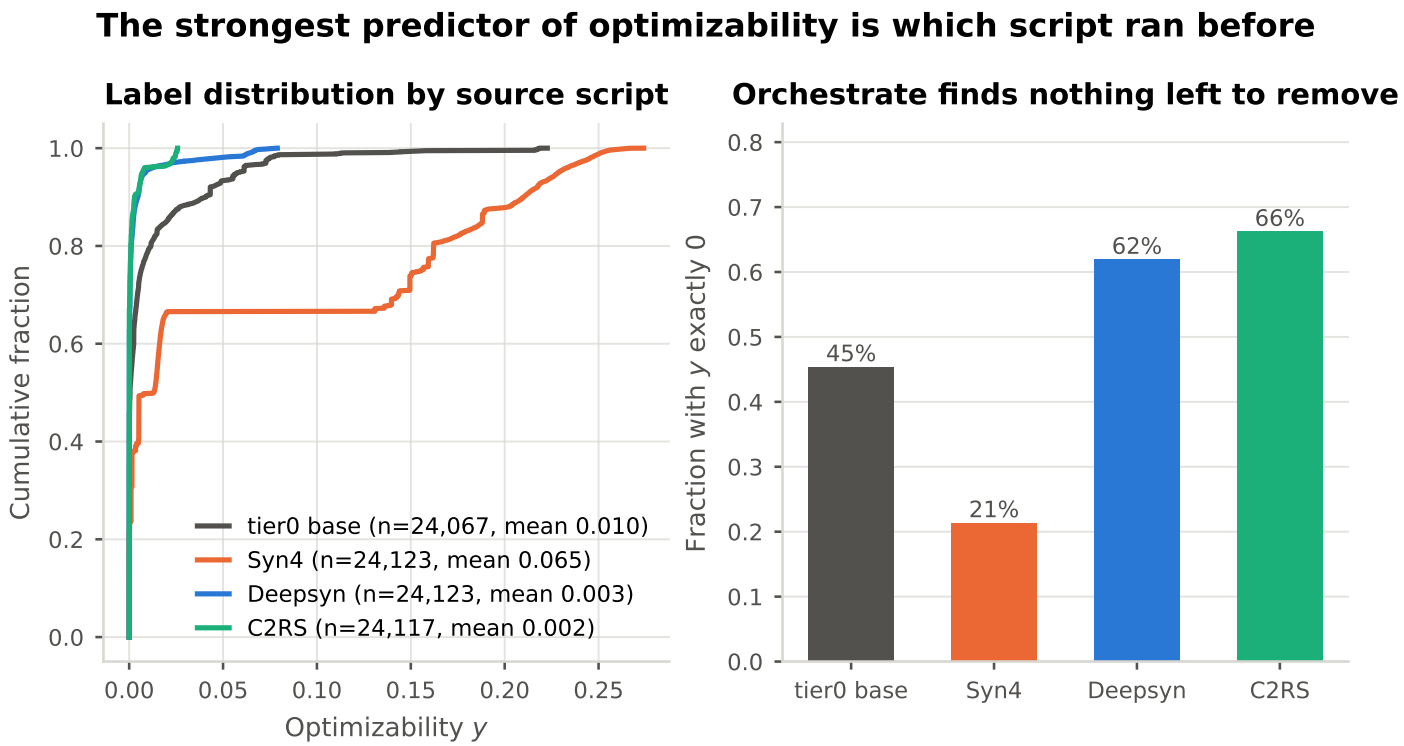

Figure 3.6: What the target mostly measures. A tier-1 graph is a base circuit already optimized by one of three scripts, and the label is what Orchestrate can still remove after that. Orchestrate finds almost nothing left on a C2RS- or Deepsyn-derived graph (52–54% score exactly zero) and a great deal on a Syn4 one (11.7% zero, mean y 0.046). The source script, not the circuit, is the strongest single determinant of the target.

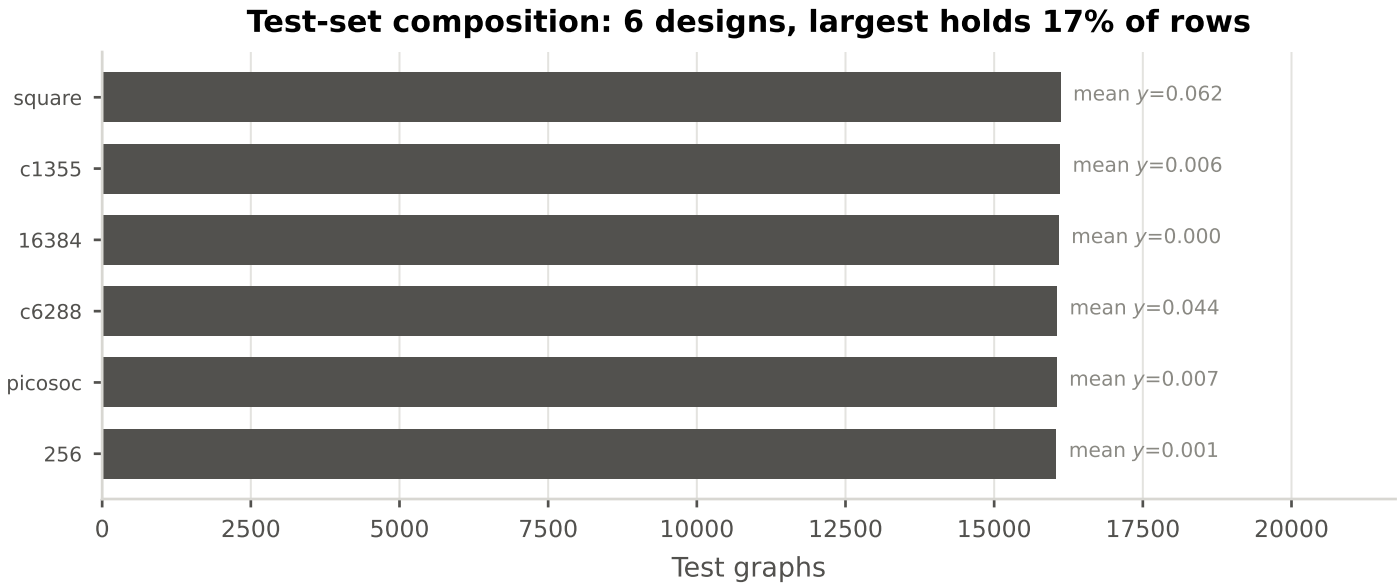

Figure 3.7: How the design-disjoint test set is composed. Under a design-level split the test set is a handful of whole circuits rather than a random sample, so every pooled test metric is an average over these designs and their mix.

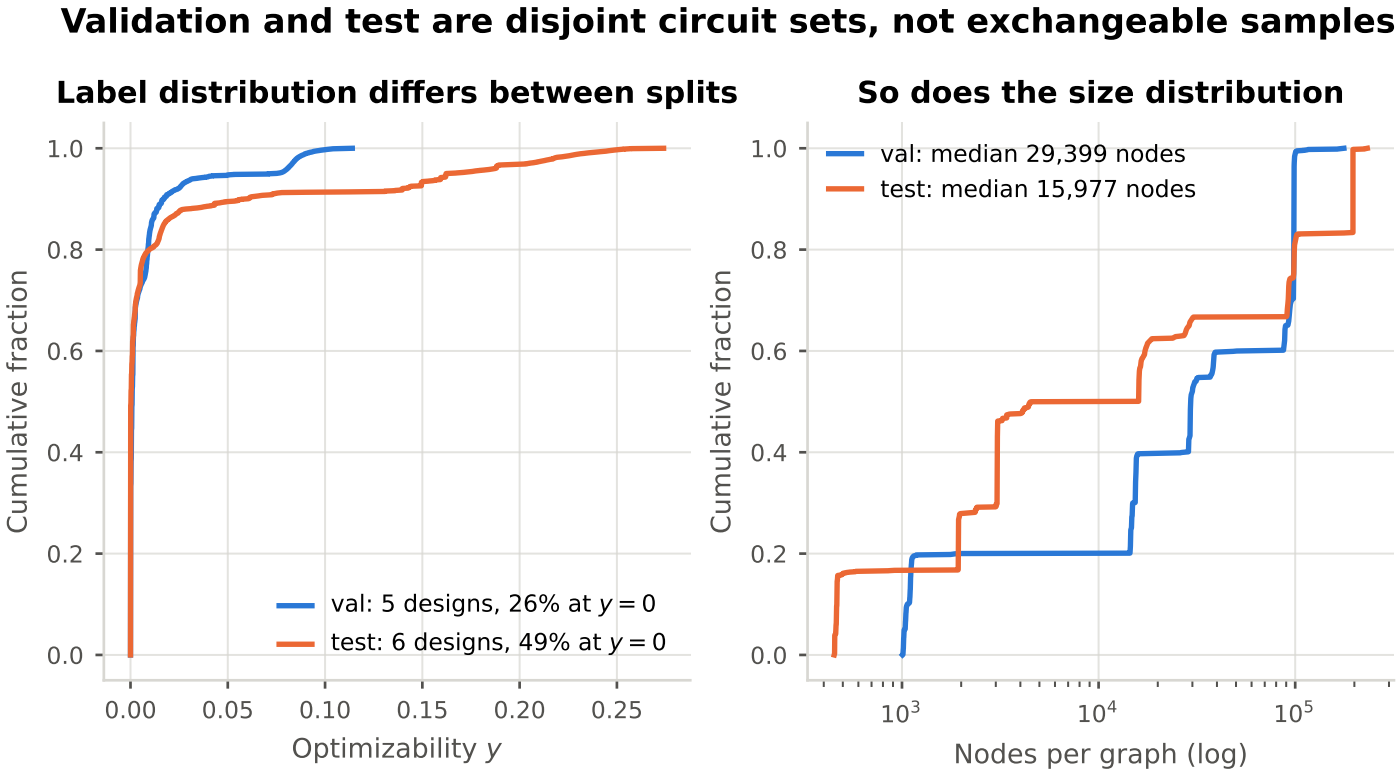

Figure 3.8: Validation and test are disjoint circuit sets, not exchangeable samples: 26% of validation graphs sit at $y = 0$ against 49% of test, and the test maximum y is more than twice the validation maximum. This is expected under a design-level split (3.4.2) and is the direct explanation for the validation-to-test gap in 4.28; it also bounds how much validation-based model selection can be assumed to transfer.

Table 3.5: Held-out test corpus. The label is concentrated near zero, which is the denominator every R^2 in this chapter is measured against.

Tier	Graphs	Designs	Median nodes	Max nodes	Mean y	SD y	$y < 0.005$
tier0	24067	6	3559	232715	0.010	0.025	0.717
tier1	72363	6	15977	196764	0.023	0.059	0.739
pooled	96430	6	15977	232715	0.020	0.053	0.734

Table 3.6: Composition of the evaluation corpus over the two stored tiers. Validation and test differ substantially in label and size distribution, which is expected under a design-level split and is the direct explanation for the validation-to-test gap.

Split	Designs	Graphs	tier0	tier1	Median nodes	Mean y	$y = 0$	Max y
val	5	80525	20108	60417	29399	0.008	0.262	0.114
test	6	96430	24067	72363	15977	0.020	0.488	0.274

Table 3.7: Structural corpus statistics. The AND-gate share is recovered from the AND-gate-only node masks over 10,000 graphs: that method drops exactly the primary inputs and outputs, so its node retention is the AND-and-constant share. This is what fixes that method's compression, rather than a parameter that could be dialled.

Statistic	Mean	SD	Min	Max
AND gates and constants (share of nodes)	0.821	0.114	0.633	0.999
Primary inputs and outputs (share of nodes)	0.179	0.114	0.001	0.367
Edges per node	1.752	0.166	1.577	1.992

Table 3.8: Whole-corpus statistics by tier.
NOT YET GENERATED. Run `data/creation/corpus_tier_stats.py -latex` *on the cluster, where the per-graph ML CSV lives, and write the result to* `media/tables/corpus_tiers.tex`.

3.3.7 Held-Out Hyperparameter Tuning Subset

A smaller subset of 50,000 graphs, balanced between the two stored tiers, was used exclusively for the initial hyperparameter tuning and then excluded from the rest.

[TODO: make much more concise. keep full text commented out.]

3.4 Experiments

3.4.1 Research Design

The manipulated variable is the reduction method applied to the training data; the encoder, dataset, splits, label, optimizer and evaluation protocol are held constant, so that differences in outcome are attributable to the reduction. The reference point is the full-graph model of RQ1, and every reduced configuration is reported relative to it rather than in isolation.

Two hypotheses are stated in advance (1.2.3, 1.2.4): H1, that path-contracting summarization retains more than edge-removing sparsification at matched compression, because contraction shortens propagation paths where deletion only lengthens them; and H2, that the domain-informed adaptations tested here are too simple to win outright, in which case the informative outcome is what the null result says about where the predictive signal lives. **[TODO: done really need to restate hypithesis more so say how this addresses each RQ or how i iwll address them]**

[TODO: this paragraph seems unnecessary and wdym known]

Positive control. The design includes one configuration whose outcome is known in advance. Colour refinement at count-cap ∞ on the exact-compression track is provably lossless for this encoder (3.1.5), so a model trained on those coarsened graphs must score essentially identically to the full-graph baseline, both on reduced graphs and when queried on full ones. This is what makes a poor cross-state result interpretable: without it, RQ5 cannot distinguish "models trained on reduced graphs do not generalize" from "the evaluation path is wrong". Any deviation of this configuration from the baseline is a defect to be found, not a finding to be reported.

The experiments. Four experiments implement the design, and they do not map one-to-one onto the five research questions (3.9). RQ4 is a re-analysis of the Phase 3 runs under a paired comparison rather than a separate experiment, which is why it costs no additional training; RQ5 reuses the Phase 3 checkpoints and adds only an evaluation pass. The ordering is a dependency chain rather than a preference: Phase 3 cannot start until Phase 2 has produced the cached reduction artifacts, and Phase 4 needs Phase 3's checkpoints.

Phase 1 establishes both the predictive reference point and the baseline hardware cost (peak memory, epoch time) that every reduced configuration is measured against. Phase 2 requires no training and determines which methods are viable at corpus scale at all. Phase 3 places the reduced configurations against the Phase 1 reference on a Pareto front of predictive degradation versus memory and speed. Phase 4 queries models trained exclusively on reduced graphs with unreduced ones, testing structural generalization.

Table 3.9: The four experimental phases, what each answers, and what each costs. Only Phases 1 and 3 consume training compute.

Phase	Answers	What is run	Training co
1	RQ1	Full-graph model and baselines	one run per
2	RQ2	Offline reduction precompute, profiled	none
3	RQ3, RQ4	One model per reduced configuration	one run per
4	RQ5	Phase 3 checkpoints on full graphs	none (evalu

Limits of the design. The comparison is within one encoder family, one synthesis algorithm and one bounded graph scale. It can rank reduction methods against each other under those conditions; it cannot establish that the ranking transfers to another architecture, another target algorithm, or industrial-scale circuits. Section 5.4 treats each in turn. **[TODO: dont discuss limits outside of limitatoin this much in my opinion]**

3.4.2 Setup

Data Splitting

An 80/10/10 split by volume, partitioned *by design*: all graphs derived from a given source circuit fall in the same split. This is the strong choice and it is deliberate. Because the corpus is built by transforming and then repeatedly optimizing a small number of source designs, a random split would place a base graph in training and its own optimized descendant in test: structurally near-identical circuits in different optimization states. The model could then score well by recognising the design rather than by reading its structure. Splitting by design removes that path entirely, at the cost of making the task harder: the test set contains circuits the model has never seen in any state.

Two alternative strategies are implemented and available (split by optimization recipe, and a plain random split). They are not used for the reported results; they are run once, at the primary configuration, as the protocol-sensitivity experiment RQ1a (4.1.6). The three protocols differ in what they hold out, and each corresponds to an evaluation setting already established in the literature (Table 3.10).

Graph Scale Bounds

The corpus is bounded at 366,040 nodes per AIG. This intermediate scale is what makes RQ1 possible: a single unreduced graph fits in accelerator memory, so the full-graph baseline every other result is measured against can actually be trained. Larger circuits would remove the reference point and leave only reduced configurations to compare with each other.

The bound is a scoping decision with a cost, and it is the limitation closest to the motivation: the argument for this work

Table 3.10: The three train/test protocols implemented in this work, ordered from leakiest to strictest. Only the design-disjoint protocol is used for the reported results; the other two are run once each at the same configuration to quantify how much of a reported score is design recognition rather than structural reading.

Protocol	Held out	Established as	Role here
Random (per-row)	Nothing. Individual graphs are assigned independently, so a base graph and its own optimized descendant may straddle the split.	The common default in circuit representation-learning evaluations.	Ablation
Recipe-disjoint	Whole synthesis recipes: every tier-0 step of a held-out recipe, plus its tier-1 and tier-2 descendants, across all designs. Circuits stay seen; the recipe does not.	OpenABC-D Variant 1 [Chowdhury et al., 2021]; LOSTIN transductive [Wu et al., 2022].	RQ1a
Design-disjoint	Whole source circuits: every graph derived from a held-out design, in any optimization state, is confined to one split.	OpenABC-D Variant 2 [Chowdhury et al., 2021]; LOSTIN inductive [Wu et al., 2022].	**Headline**

invokes circuits of millions to billions of nodes, and the evaluation does not reach them (5.4.2).

Dynamic Batching

Graphs in this corpus vary in size by orders of magnitude, so a fixed batch size in graphs gives wildly varying memory use and eventually fails on the largest circuits. Batches are instead assembled to a maximum *node* budget by greedy bin-packing, which bounds activation memory regardless of which graphs are drawn. The batch plan is computed once and cached, since replanning over a corpus this size is expensive.

Two budgets are used and they are deliberately distinct. Training uses the smaller of the two; evaluation, which stores no activations or gradients, uses a substantially larger one. Keeping them separate rather than raising a single shared value avoids invalidating the batch plan the trained checkpoints were produced under.

Constraint on efficiency measurements. The evaluation budget must be identical across all configurations, because peak memory and throughput both depend on it. Any change requires re-running every configuration; a mixed set of budgets makes the RQ2 comparison meaningless rather than merely noisy. This is stated here because it constrains how the results in 4.2 may be read and combined.

One irreducible peak remains that no budget can lower: a graph larger than the budget still forms a batch of its own, since a graph-level target cannot be split across batches. The largest circuit in the corpus therefore sets a floor on peak memory for every configuration that does not reduce node counts.

Model Configuration

An Edge-Aware Graph Convolutional Network (GCN+) is trained to predict the optimizability of a circuit when subjected to the `Orchestrate` synthesis script. The architecture is described in 3.1 and is held fixed across all reduction configurations; the baselines it is compared against are described in 3.1.6.

Hyperparameter Tuning

Hyperparameters were selected by an Optuna [Akiba et al., 2019] sweep on the held-out balanced subset of 3.3.7, which is then excluded from all subsequent training and evaluation. Tuning on a subset that later appeared in training or test would let the reported scores absorb information from the tuning process; excluding it entirely costs 50,000 graphs and removes the question.

Tuning was performed once, on the full-graph configuration, and the resulting values are held fixed for every reduced configuration. This is the conservative choice for the comparison being made: re-tuning per reduction method would confound "this reduction retains more signal" with "this reduction received more tuning effort". It does mean reduced configurations are evaluated at hyperparameters chosen for a different input distribution, which if anything biases against them. That is worth stating explicitly, since it bounds the strength of any negative result about reduction.

Training Configuration

Smooth L1 loss with $\beta = 0.01$, chosen well below the default because the target occupies a narrow band within $[0, 1]$ and the default threshold would place nearly every residual in the quadratic regime, making the loss effectively squared error. Optimization uses AdamW with learning rate 3×10^{-4} and weight decay 3.96×10^{-3}, a linear warmup over the first 10,000 steps, and `ReduceLROnPlateau` thereafter (factor 0.5, patience 2, floor 10^{-6}). Training stops early on validation loss with patience 3. Gradients are clipped at 1.0.

Two settings affect the measurements rather than the model and must therefore be disclosed with them: `torch.compile` is enabled, which changes every timing figure, and mixed precision is used on GPU, which changes every memory figure. Neither is varied across configurations, so comparisons remain valid; absolute numbers are only meaningful together with these settings.

[TODO: all these specific hyperparemeters like train split etc. how can i best display this more concisely idk if it should really be takign up this much space in my methodology or if this is more appendix info]

Hardware and Software Environment

All experiments ran on the Snellius national supercomputer: H100 80 GB GPU nodes for training and evaluation, and CPU-only

nodes for the offline reduction precompute, which is embarrassingly parallel over graphs and does not benefit from a GPU.

Two measured characteristics of the workload are worth recording, because they explain the shape of the RQ2 results rather than merely describing the setup. First, memory scales close to linearly with nodes and edges per batch, the encoder's own parameters being negligible at this width and depth, which is why a node budget bounds memory effectively at all. Second, message passing is latency-bound on gather and scatter operations rather than compute-bound: at full device occupancy the floating-point pipelines and memory bandwidth both sit far below saturation. The consequence for RQ2 is that a larger batch amortises per-batch overhead but does not deliver a proportional speedup, and that reductions which remove *edges* should help throughput more than their node compression alone would suggest. **[TODO: shoudl this be here or in reproducability?]**

[TODO: keep full text commented out. make more concise for this turn in.]

3.4.3 Evaluation Metrics

Four groups of metrics are reported. Accuracy and ranking are reported together and never separately: on a coarsened EDA graph, CTS-Bench [Khadka et al., 2026] observed the mean absolute error barely moving while R^2 fell below zero. Absolute error alone can look healthy after the model has lost all discriminative power, which is exactly the failure a reduction study is most at risk of missing.

Notation. Let $\mathcal{D} = \{G_i\}_{i=1}^{N}$ be an evaluation set, $y_i = y(G_i) \in [0, 1]$ the optimizability target of 2.1.6, f_θ the trained encoder and R the reduction operator, with $R = \mathrm{id}$ for the unreduced baseline. Residuals are written $r_i = \hat{y}_i - y_i$, and $\bar{y} = N^{-1} \sum_i y_i$ is the target mean over $\mathcal{D}$.

Evaluation protocol. Each trained configuration is evaluated in two passes over the test set, differing only in what the same weights are given as input:

$$\hat{y}_i^{\text{match}} = f_\theta(R(G_i)) \,, \tag{3.24}$$

$$\hat{y}_i^{\text{full}} = f_\theta(G_i) \,. \tag{3.25}$$

The matched pass (3.24) sees graphs under the reduction the model was trained with, and the full-graph pass (3.25) sees unreduced graphs. The matched pass answers RQ3. The pair together answers RQ5, since the difference between the passes isolates the cost of the structural shift from the cost of the reduction itself.

Predictive Accuracy

The training objective is the smooth L1 loss with transition point $\beta = 0.01$,

$$\ell_\beta(r) = \begin{cases} r^2/(2\beta), & |r| < \beta, \\ |r| - \beta/2, & \text{otherwise}, \end{cases} \tag{3.26}$$

reported on validation and test for comparability with the training curves. Since the target occupies $[0, 1]$ while $\beta = 0.01$, all but the smallest residuals fall in the linear branch of (3.26): outside a band of one percentage point the objective behaves as a mean absolute error. Accuracy proper is reported as

$$\mathrm{MAE} = \frac{1}{N} \sum_{i=1}^{N} |r_i| \,, \tag{3.27}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} r_i^2}, \tag{3.28}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N} r_i^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2}. \tag{3.29}$$

MAE and RMSE are on the label's own scale, so $\mathrm{RMSE} = 0.05$ is five percentage points of optimizability. By the power mean inequality $\mathrm{MAE} \le \mathrm{RMSE}$, with equality exactly when every $|r_i|$ is identical, so the ratio $\mathrm{RMSE}/\mathrm{MAE}$ measures how much a few large errors carry the average. Equation (3.29) divides the same squared error by the target's own spread: $R^2 = 0$ is the score of the constant predictor $\hat{y}_i = \bar{y}$, and $R^2 < 0$ a model worse than that constant. Because the denominator is N times the sample variance of y within $\mathcal{D}$, R^2 is computed for validation and test only, never per training batch, whose variance is not a meaningful reference for the spread of the target.

Persisted predictions and stratified re-scoring. The evaluation sweep writes every per-graph prediction to disk, not only the aggregates, so every metric above can be recomputed on any subset of the test set from the same forward pass. This matters because the pooled label is severely skewed: roughly half the test graphs have $y_i = 0$ exactly and about three quarters have $y_i < 0.005$, so a pooled R^2 is dominated by a mass of near-identical labels and can look respectable while saying nothing about the graphs a practitioner would ask about. Each accuracy claim is therefore repeated on three subsets beside the pooled figure: graphs from designs the synthesis script can still change (two designs sit at a fixed point, with largest observed optimizability 0.0003, so they contribute no variance for R^2 to explain, only noise for it to divide by); graphs with $y_i > 0$; and graphs derived from a single upstream script, which holds constant how much optimization the input received before Orchestrate saw it. The strata are applied uniformly to every configuration rather than invoked where they happen to help.

Ranking Quality

Spearman rank correlation between predicted and true optimizability across the test set, that is the Pearson correlation of the midrank vectors,

$$\rho = \frac{\mathrm{cov}(\mathrm{rk}(\hat{y}), \mathrm{rk}(y))}{\sigma_{\mathrm{rk}(\hat{y})}\, \sigma_{\mathrm{rk}(y)}}. \tag{3.30}$$

Equation (3.30) is unchanged by any strictly increasing map applied to $\hat{y}$, which is why it is reported alongside the error metrics rather than in place of them: a model whose predictions have collapsed toward $\bar{y}$, the failure mode a reduction study should expect, holds ρ while RMSE and R^2 degrade. Ranking is also closer to use. A practitioner deciding where to spend synthesis effort needs the ordering of circuits, not calibrated values.

The metric ranks *circuits* under one fixed script, not *scripts* for one circuit. The latter is the script-selection task the motivation invokes and this thesis does not attempt (1.2.6).

Efficiency and Resource Metrics

Peak device memory, training step and epoch wall-clock time, and inference throughput in graphs per second. These come from three instruments running under three different batching regimes, and which instrument produced a number determines what that number may be compared against, because 3.4.2 makes budget uniformity a precondition for reading the efficiency results and that precondition applies to only one of the three.

Inference throughput and peak memory are measured during the evaluation sweep, under the fixed evaluation node budget of 3.4.2, identical across every configuration. Comparing configurations evaluated at different budgets would be invalid rather than merely noisy.

Training step time and step memory come from a separate benchmark that processes one graph per batch, deliberately using no budget at all: a node budget packs a different number of graphs into a batch for every configuration, so per-batch aggregates would average over incomparable batch compositions. With one graph per batch, each reduced graph is instead matched to its unreduced counterpart by graph identity, which is what the paired statistics of 3.4.3 require. The price is that these are per-graph costs, a controlled comparison between configurations rather than an estimate of wall-clock training cost.

Epoch wall-clock time is recorded by the training loop itself under the training node budget, the smaller of the two. It is comparable across configurations and against nothing else here.

Memory quantities reported. Peak *allocated* memory is the high-water mark of live tensors and the closest measure of the model's actual demand. Peak *reserved* memory is what the caching allocator has taken from the driver, is never smaller, and reflects allocation history and fragmentation as much as demand. Allocated is the primary figure, but reserved sets the out-of-memory boundary and is recorded alongside it. Both are taken against a *memory floor* m_{floor} (parameters, optimizer state and device context, measured after warmup with no graph resident), so the marginal cost of the graph itself is

$$\Delta m = m_{\text{peak}} - m_{\text{floor}}. \tag{3.31}$$

Δm rather than m_{peak} is the quantity a reduction can move: m_{floor} is fixed by the architecture and identical across configurations, so savings quoted against m_{peak} would be diluted toward zero by an additive constant having nothing to do with the reduction.

Reduction Quality Metrics

The node and edge retention ratios $\eta_{\mathcal{V}}$ and $\eta_{\mathcal{E}}$ of (2.13) are reported separately rather than collapsed into one compression figure, together with the offline wall-clock cost of computing the reduction.

The offline cost matters independently of the training saving. Let c_R be the cost of computing and storing one graph's reduction, and t_{id} and t_R the per-graph step cost on the unreduced and the reduced graph. One epoch visits each graph once, so over n epochs reducing is cheaper than not reducing once

$$n > n^\star = \frac{c_R}{t_{\text{id}} - t_R}. \tag{3.32}$$

Reporting the amortisation threshold $n^\star$ rather than a raw offline cost states the trade-off in the terms that decide it, and makes the failure case explicit: (3.32) is undefined for $t_R \geq t_{\text{id}}$, so a reduction that does not shorten a step never amortises, however cheap it is to compute. The reduced graph is stored once and reloaded, so n accumulates over every epoch of every run that reuses the artifact.

Effective receptive field. Hypothesis H1 claims that coarsening helps by contracting paths, and that claim needs a measurement of its own. For a node v, let

$$\mathcal{C}_k(v) = \{u \in \mathcal{V} : u \rightsquigarrow v \text{ along a directed path of length at most } k\} \tag{3.33}$$

be its k-hop fanin cone, and let $m_u = \mathbf{1}^\top x_u$ be the member count of node u, supplied directly by the count-based feature schema of 3.1 and equal to 1 on an unreduced graph. The metric is the member-weighted mean cone size over a node set $\mathcal{S}$ sampled from $\mathcal{V}(G)$,

$$\bar{c}_k(G) = \frac{1}{|\mathcal{S}|} \sum_{v \in \mathcal{S}} \sum_{u \in \mathcal{C}_k(v)} m_u, \tag{3.34}$$

evaluated at $k = L$, the number of encoder layers, before and after reduction. The weighting is what makes the comparison meaningful: counting super-nodes rather than their members would make $\bar{c}_L$ fall under every coarsening by construction, while weighting by m_u counts original gates in both states. H1 predicts $\bar{c}_L(R(G)) > \bar{c}_L(G)$ under summarization and $\bar{c}_L(R(G)) \leq \bar{c}_L(G)$ under sparsification. Unlike a DAG longest-path metric, (3.34) is well defined for every method, not only for those that guarantee an acyclic quotient.

Statistical Treatment

Efficiency comparisons are made pairwise per graph against the full-graph baseline. For each test graph the saving in an efficiency quantity q, either marginal memory Δm or step time, is taken relative to that graph's own baseline measurement,

$$\delta_i = 100 \left(1 - \frac{q_R(G_i)}{q_{\text{id}}(G_i)}\right), \tag{3.35}$$

and $\{\delta_i\}$ is summarised by its mean with a 95% percentile bootstrap interval over $B = 2000$ resamples, together with a two-sided Wilcoxon signed-rank test of $H_0\colon \operatorname{med}(\delta) = 0$. Both configurations are measured on the same graph, so the pairing in (3.35) is by construction, and the savings are skewed across a corpus whose graph sizes span orders of magnitude, which is why a rank test replaces a paired t-test.

Accuracy comparisons have no equivalent treatment, and this is a real weakness rather than an omission of detail. Each configuration is trained once, so there is no run-to-run variance against which to judge whether a difference is real. This bears

directly on RQ4, whose expected effect is small: either the RQ4 pairings are re-run across several seeds, or the accuracy differences must be reported as point estimates and read with corresponding caution (5.4.1).

[TODO: make much more concise. keep full text commented out. epoch wall time is not useful since i varied between experimetns i have benchmarking at the end for througput per rgaph etc.] [TODO: mention briefly which measurements answer which question]

3.4.4 Reproducibility

Split assignment, the randomised reduction methods, and model initialisation are each seeded at a fixed value; the deterministic reduction methods produce identical output by construction. Seed values and the full experimental configuration are listed in A.4.

Seeding makes a single run repeatable, not robust. Each configuration is trained once, so differences between configurations are point estimates carrying no run-to-run variance, a limitation treated in 3.4.3.

Reduction is precomputed offline, and the artifacts behind every reported experiment are released as masks over the original graphs (3.4.4). Reproducing a result therefore does not require re-running the reduction stage.

Numerical Determinism

[TODO: this seems like overkill. never seen it in another paper feel free to prove me wrong]

Code and Data Availability

Chapter 4

Results

[TODO: check these against current RQs]

4.1 Baseline Predictive Performance on Full AIGs (RQ1)

4.1.1 Comparison Against Naive Baselines

4.1.2 Predictive Accuracy Metrics

Tabular presentation of Smooth L1 Loss, RMSE, and R^2 for the Edge-Aware GCN+ predicting Orchestrate optimizability, on a test set of full AIGs.

4.1.3 Ranking Efficacy

Spearman correlation coefficients.

4.1.4 Baseline Model Outcomes

4.1.5 Error Analysis

4.1.6 Protocol Sensitivity: How Much of the Score Is Design Recognition? (RQ1a)

4.1.7 Cost of the Full-Graph Baseline

Three quantities characterise the cost of training on unreduced graphs, and together they form the reference against which every reduction in RQ2 is read: time per optimisation step and per epoch, peak allocated device memory, and inference throughput in graphs per second.

4.2 Efficiency Profile of Graph Reduction Techniques (RQ2)

4.2.1 Graph Reduction Offline Profile

- Statistical summary of node and edge reduction ratios achieved by Partitioning, Sparsification, and Summarization, reported separately per node and per edge (2.1.5).
- Computational wall-clock time required to execute these reduction algorithms offline, and the amortisation threshold each implies: how many training runs the cached artifact must serve before the reduction pays for itself.

4.2.2 Training Memory Dynamics and Compute Efficiency

Memory is reported as peak allocated device memory per configuration, in absolute terms and as a fraction of device capacity. Compute efficiency is reported as device utilisation alongside memory-bandwidth activity, which together distinguish a configuration that is compute-bound from one that is waiting on memory.

4.2.3 Throughput and Training Speedup

- Comparison of `train_step_time_s_step` and `train_step_tim` against the full-graph baseline.
- Inference throughput improvements (graphs processed per second) over the baseline.

4.2.4 Efficiency Ranking Across Methods

4.3 Predictive Retention and Performance Trade-offs (RQ3)

4.3.1 Accuracy Degradation

Grouped bar charts comparing the RMSE and R^2 of models trained and tested on reduced graphs against the RQ1 full-graph baseline.

4.3.2 Ranking Preservation

4.3.3 Accuracy Loss Attributable to Reduction

4.3.4 The Pareto Front (Trade-Off Analysis)

Scatter plots visualizing Accuracy (y-axis) vs. Memory/Speed Gains (x-axis) to identify the optimal graph reduction technique, bridging RQ2 and RQ3.

4.4 Value of Domain-Informed Adaptation (RQ4)

4.4.1 Matched-Compression Pairings

4.4.2 Retention Gap at Matched Compression

4.4.3 Structural Interpretation

4.4.4 Adequacy of the Heuristics Tested

4.5 Cross-Structural Generalization (RQ5)

4.5.1 Reduced-to-Full Inference Accuracy

Smooth L1 Loss, RMSE, and R^2 metrics from testing a model (which was trained exclusively on reduced AIGs via partitioning, sparsification, or summarization) directly on normal, full-sized AIGs.

4.5.2 Zero-Shot Ranking

Spearman correlation.

4.5.3 Matched-State vs. Cross-State Drop-Off

Comparison measuring the performance drop-off between matched-state testing (Reduced $\rightarrow$ Reduced) vs. cross-state testing (Reduced $\rightarrow$ Full).

4.5.4 Generalization by Reduction Family

4.5.5 Inference Cost

4.6 Summary of Findings

4.7 Figure and Table Gallery (unsorted, to be filed)

4.7.1 RQ1: Baseline Predictive Performance

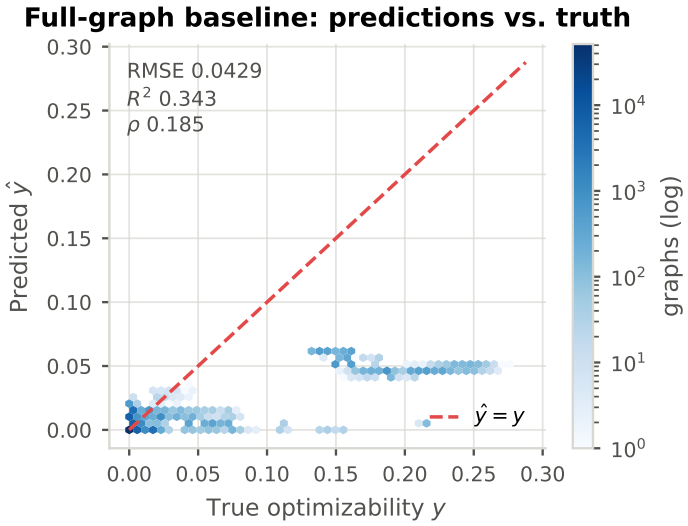

Figure 4.1: Predicted against true optimizability for the full-graph baseline on the test split.

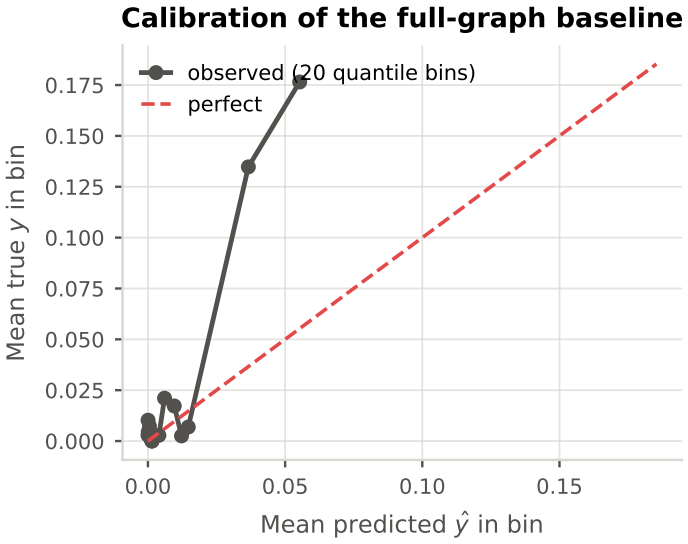

Figure 4.2: Binned reliability of the full-graph baseline. A model that has collapsed toward the label mean shows here as a flat curve regardless of how healthy its RMSE looks.

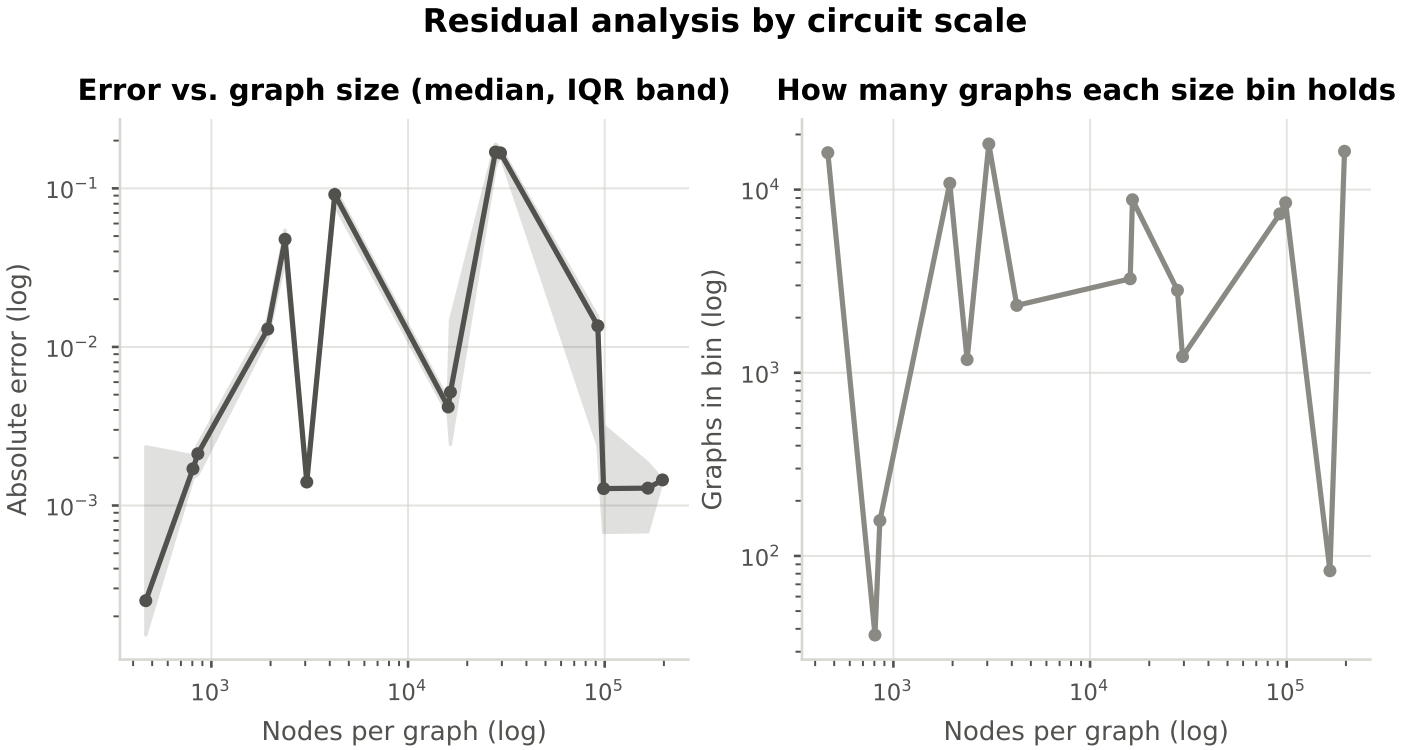

Figure 4.3: Absolute error against graph size (median and interquartile band), with the number of graphs per size bin alongside. Sets the expectation for the reduction chapters: a reduction acting hardest on large graphs starts from whatever position this plot shows.

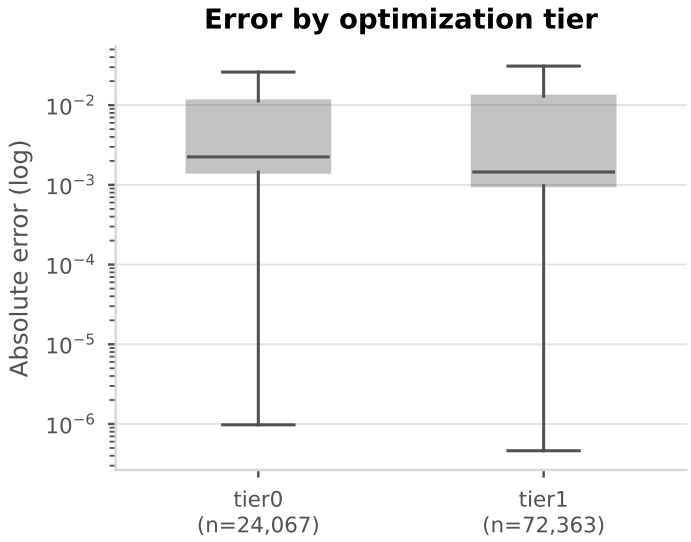

Figure 4.4: Absolute error by tier. Tiers differ by construction, since a graph already optimized once has less left to remove, so a difference here is expected and its absence would itself be a finding about the generation pipeline.

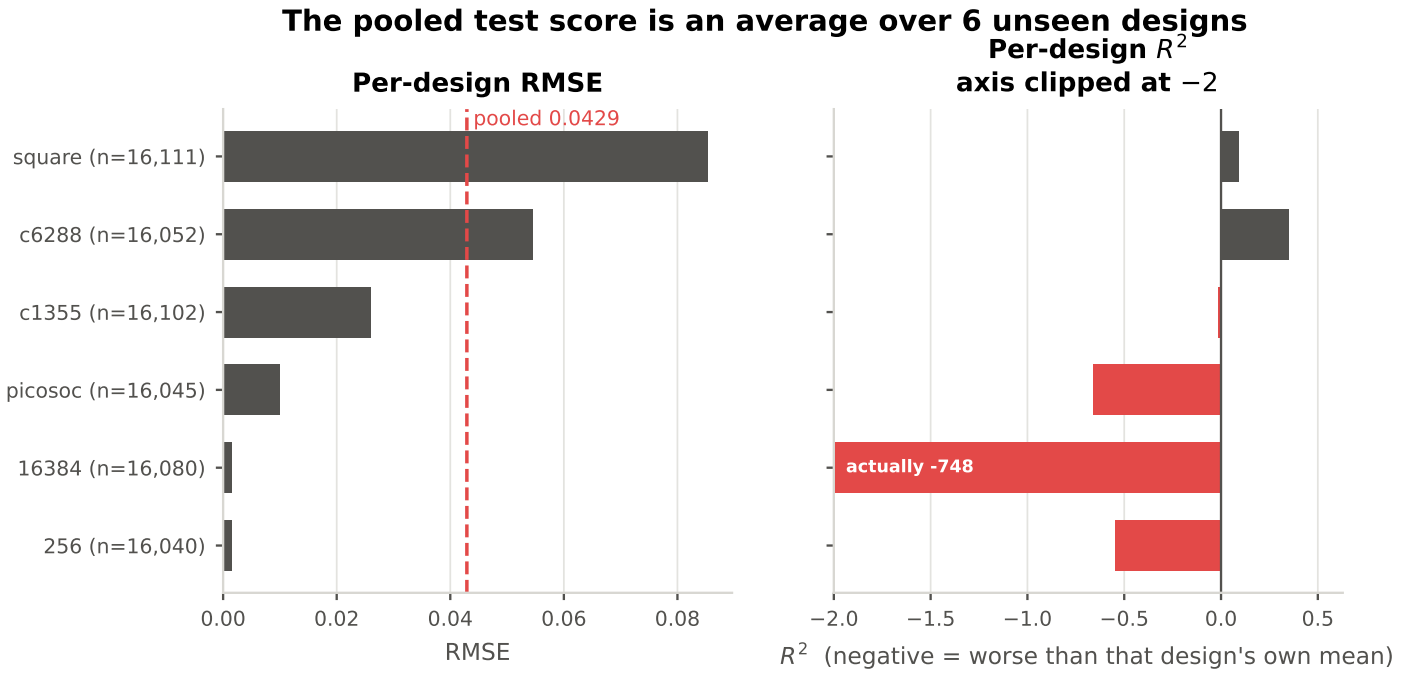

Figure 4.5: Per-design RMSE and R^2 on the design-disjoint test set. The spread across designs is very wide and R^2 is strongly negative on some of them, which means the pooled score describes the mix of designs at least as much as it describes the model. Feeds 4.1.5 and 4.1.6.

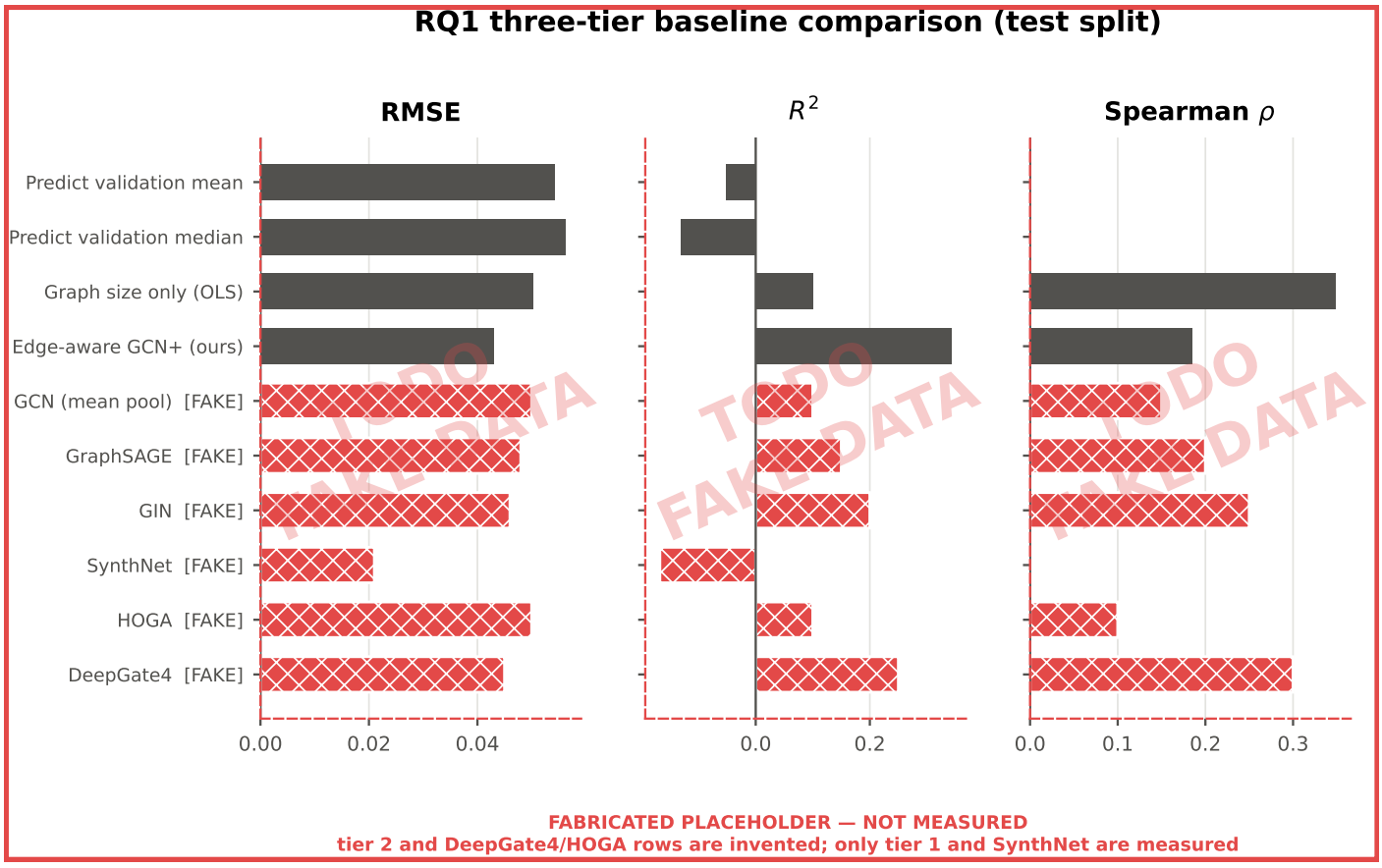

Figure 4.6: **PARTLY FABRICATED.** The RQ1 baseline comparison. The trivial predictors (fitted on validation and scored on test) and the encoder are measured. The standard encoders under identical training and the DeepGate4/HOGA rows are invented placeholders; see `src/analysis/fake_data.py`. Note that the size-only regressor outranks the encoder on Spearman correlation.

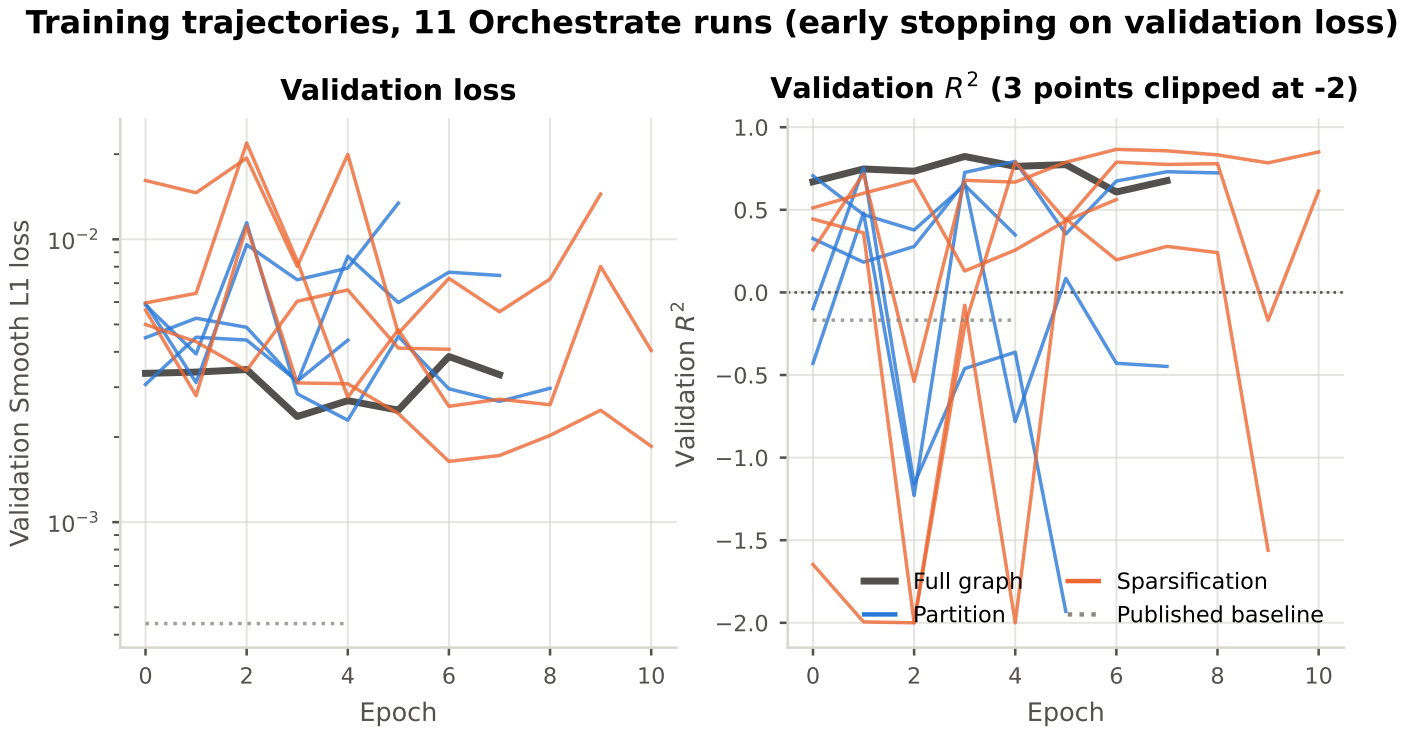

Figure 4.7: Validation loss and R^2 per epoch for every Orchestrate training run, coloured by reduction family. Validation R^2 swings by several units between consecutive epochs on some configurations, which bounds how much weight any single-seed comparison can carry (3.4.3).

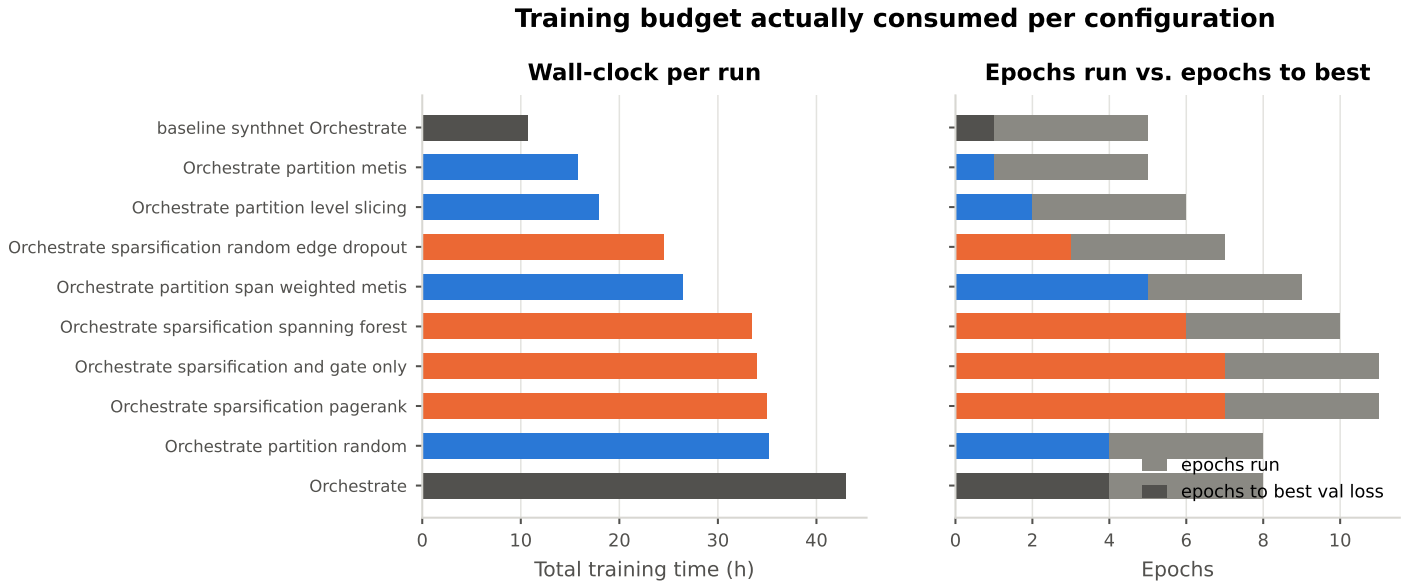

Figure 4.8: Wall-clock and epochs actually consumed per configuration, against epochs to best validation loss. Two runs of the same nominal budget are not the same amount of compute if one stopped at epoch 4 and the other at 10.

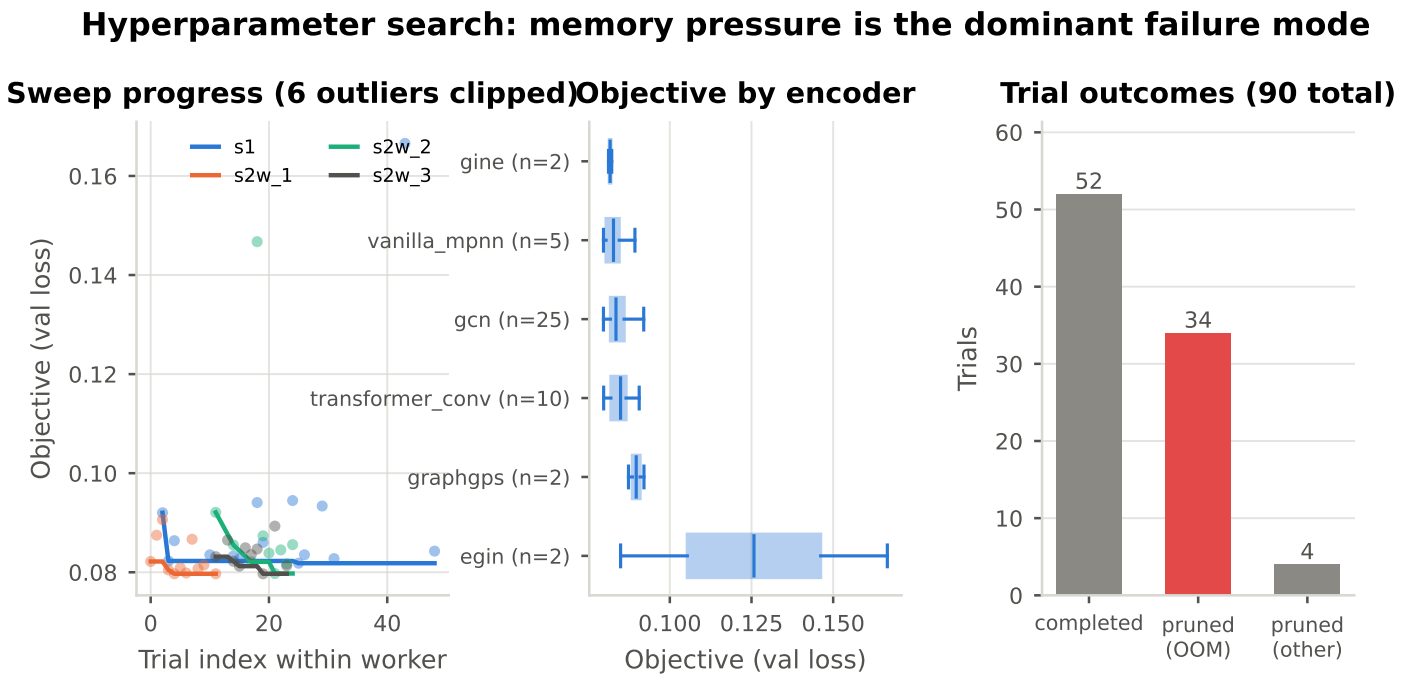

Figure 4.9: The hyperparameter sweep that fixed the architecture: progress per worker, objective by encoder, and trial outcomes. Memory pressure is the dominant failure mode: a large share of trials were pruned on memory exhaustion rather than on their objective, which is the hyperparameter-search face of the bottleneck this thesis is about. Reconstructed from the worker logs; the study databases were lost with the scratch workspaces.

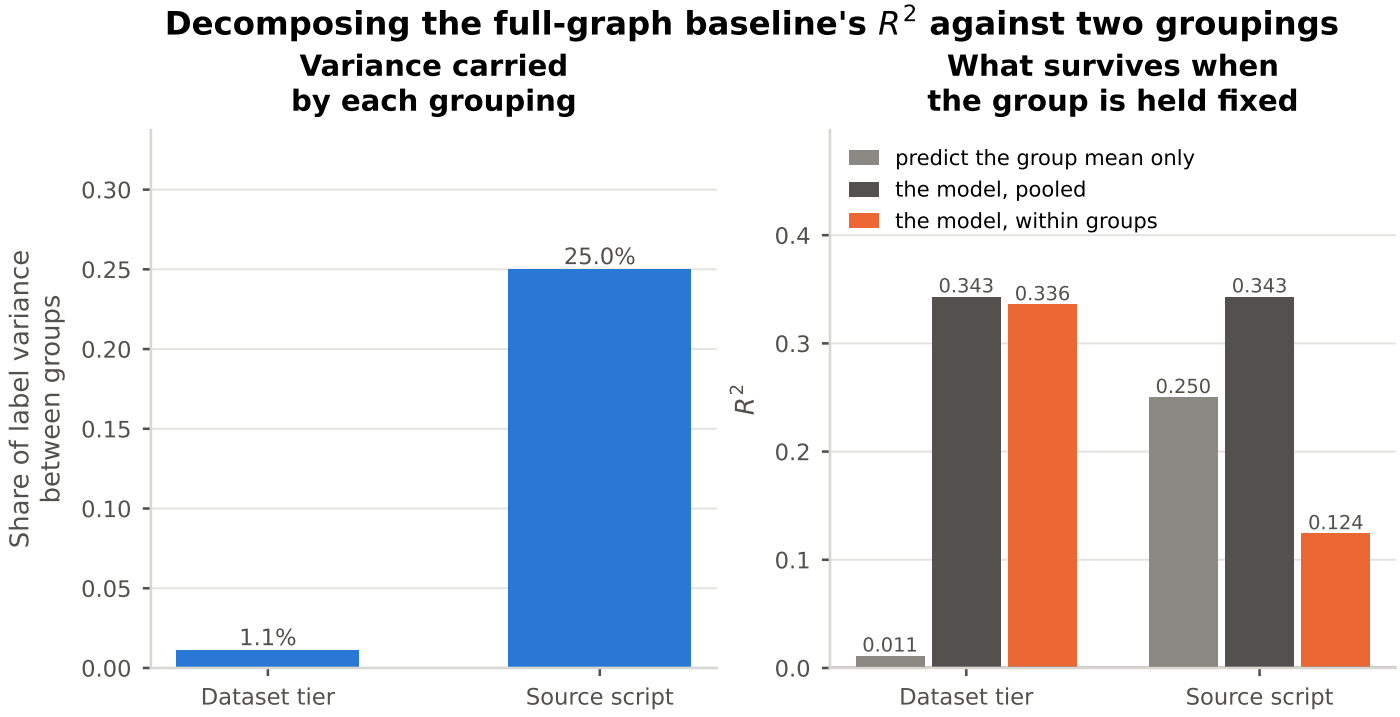

Figure 4.10: How much of the full-graph baseline's $R^2 = 0.343$ is reachable without learning anything about an individual circuit. Grouping by dataset tier carries only 1.1% of the label variance, so the tier is not a confound. Grouping by *source script* carries 25.0%: a predictor that outputs nothing but each group's mean scores $R^2 = 0.250$, i.e. roughly three-quarters of the model's reported figure. Held against within-group variance the model retains $R^2 = 0.124$. That residual is the part attributable to reading circuit structure, and it is the number 4.1 should defend.

Table 4.1: RQ1 three-tier baseline comparison on the design-disjoint test split. Trivial predictors are fitted on validation and scored on test. Upstream R^2 figures are NOT comparable: OpenABC-D z-scores its targets per design, which removes between-design variance from the denominator. Only the sign transfers.

Model	Tier	RMSE	MAE	R^2	Spearman
Predict validation mean	1: trivial	0.054	0.023	-0.052	–
Predict validation median	1: trivial	0.056	0.020	-0.131	–
Graph size only (OLS)	1: trivial	0.050	0.024	0.101	0.348
Edge-aware GCN+ (ours)	This work	0.043	0.017	0.343	0.185
[TODO/FAKE] GCN (mean pool)	2: standard encoders	0.050	0.030	0.100	0.150
[TODO/FAKE] GraphSAGE	2: standard encoders	0.048	0.029	0.150	0.200
[TODO/FAKE] GIN	2: standard encoders	0.046	0.028	0.200	0.250
[TODO/FAKE] SynthNet	3: published models	0.021	0.012	-0.168	0.000
[TODO/FAKE] HOGA	3: published models	0.050	0.030	0.100	0.100
[TODO/FAKE] DeepGate4	3: published models	0.045	0.027	0.250	0.300

Rows marked [TODO/FAKE] are invented. Train GCN / GraphSAGE / GIN under the headline pooling, head and budget (src/shell/train.sh with –encoder_name), and finish the DeepGate4 and HOGA ports on the baselines/openabc-synthnet-hoga branch. SynthNet is the only published baseline that has produced a number (train_baseline_synthnet_Orchestrate, val R2 = -0.168); the HOGA run crashed at epoch 0.

Table 4.2: Per-design metrics on the design-disjoint test set. A pooled score over a design-level split is an average over this handful of whole circuits, not over a random sample.

Design	Graphs	Mean y	RMSE	MAE	R^2	Spearman
256	16040	0.001	0.001	0.001	-0.548	0.493
16384	16080	0.000	0.001	0.001	-747.953	0.043
picosoc	16045	0.007	0.010	0.006	-0.657	0.265
c1355	16102	0.006	0.026	0.006	-0.014	0.331
c6288	16052	0.044	0.055	0.038	0.347	0.539
square	16111	0.062	0.085	0.049	0.091	0.884

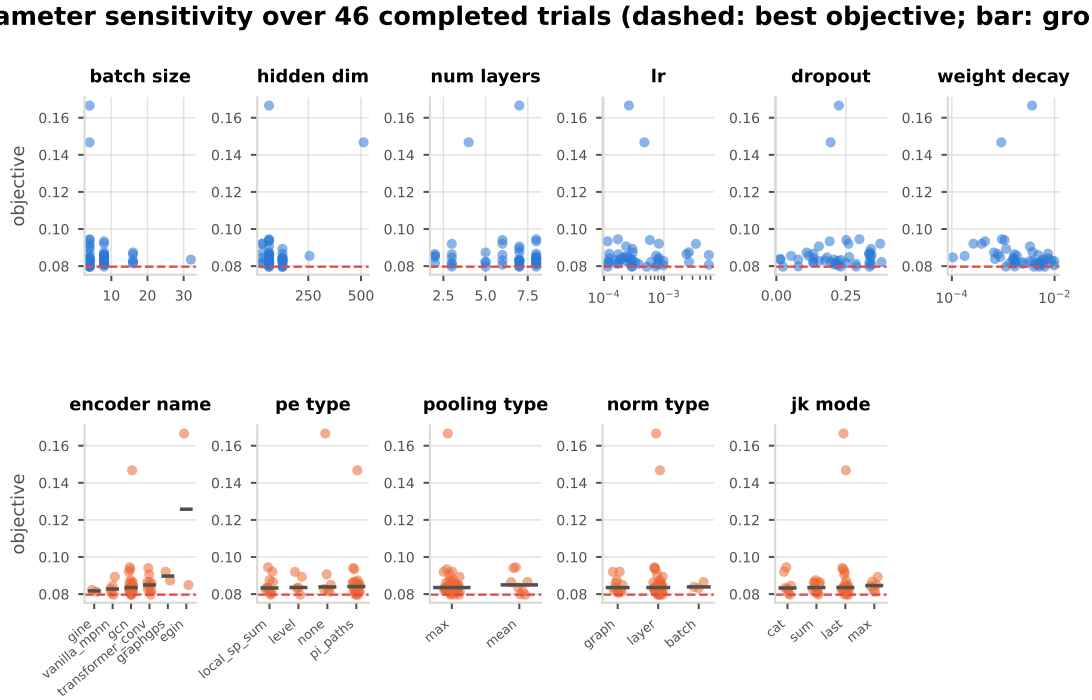

Figure 4.11: Objective against each searched hyperparameter over the 52 completed trials. This is a coarse sensitivity read rather than an importance analysis: Optuna concentrates sampling on promising regions and the sampler state was lost with the scratch workspaces, so it supports “which settings never produced a good trial” and not “which parameter matters most”.

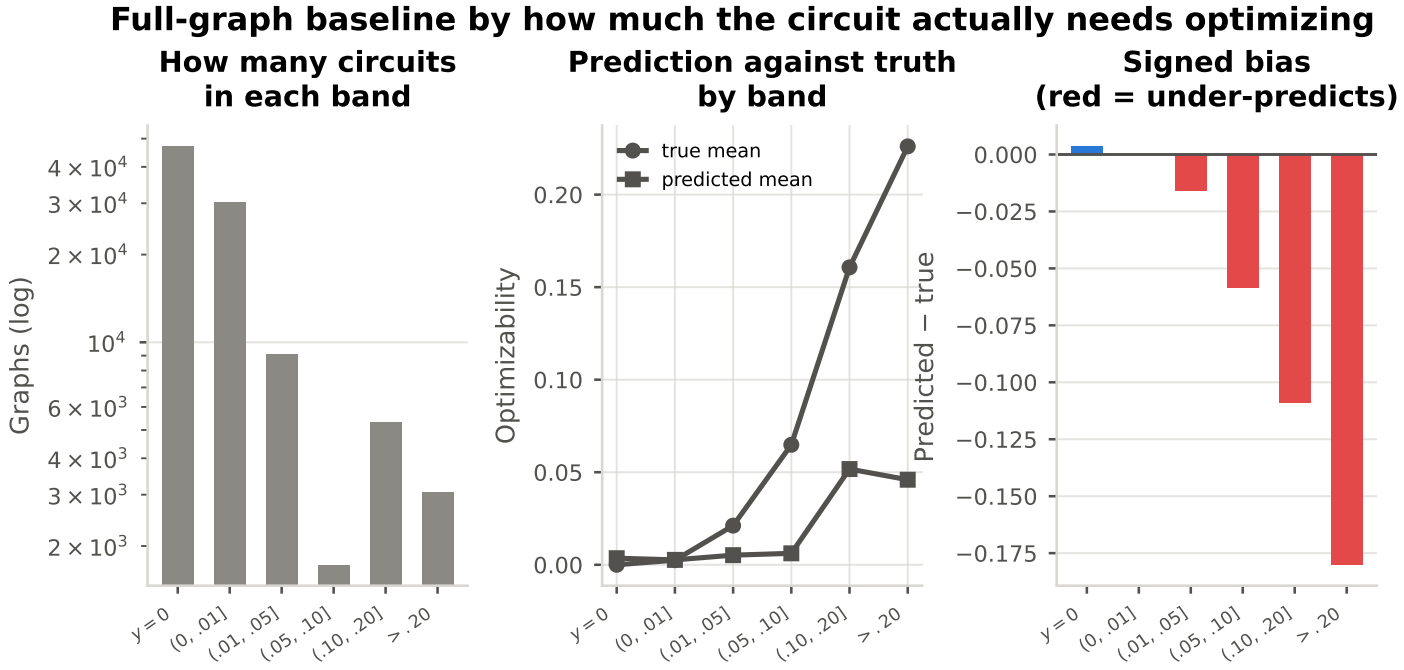

Figure 4.12: The full-graph baseline as a function of how much the circuit actually needs optimizing. The model tracks the truth on the two lowest bands, which hold 80% of the corpus, and then flattens: at a true optimizability above 0.20 it predicts 0.046 against a true mean of 0.226, an absolute error of 0.180. It has largely learned the label's marginal distribution rather than the task.

Table 4.3: Final configuration, as recorded in the W&B run config of the headline run. Fields the run did not log are marked rather than omitted.

Hyperparameter	**Value**
Encoder	gcn
Hidden dim	128.0
Layers (message passing)	4 (from config.py)
Positional encoding	level
PE dim	32.0
Pooling	mean
Head dropout	0.3
Learning rate	0.0003
Min learning rate	1e-06
Weight decay	0.00396
Warmup steps	10000.0
Scheduler patience	2.0
Node input dim	4.0
Edge attr dim	2.0
Seed	not recorded

4.7.2 RQ1a: Protocol Sensitivity

Table 4.4: RQ1a protocol sensitivity. Identical encoder, budget and seed; only the split changes. Random is the common default in circuit representation-learning evaluations, recipe-disjoint is OpenABC-D Variant 1, design-disjoint is Variant 2 and is what this thesis reports.

Protocol	RMSE	R^2	Spearman	Inflation vs. design-disjoint
[TODO/FAKE] Random (per-row)	0.020	0.900	0.900	0.557
[TODO/FAKE] Recipe-disjoint	0.030	0.700	0.750	0.357
Design-disjoint (ours)	0.043	0.343	0.185	0.000

Rows marked [TODO/FAKE] are invented. IN FLIGHT: the –split_by random and –split_by recipe training runs are on the cluster now (the design-disjoint run already exists). Identical encoder, budget and seed; only the split changes. When they land, run src/test.py for each, delete this block and rerun analysis.make_all.

4.7.3 RQ2: Reduction Efficiency

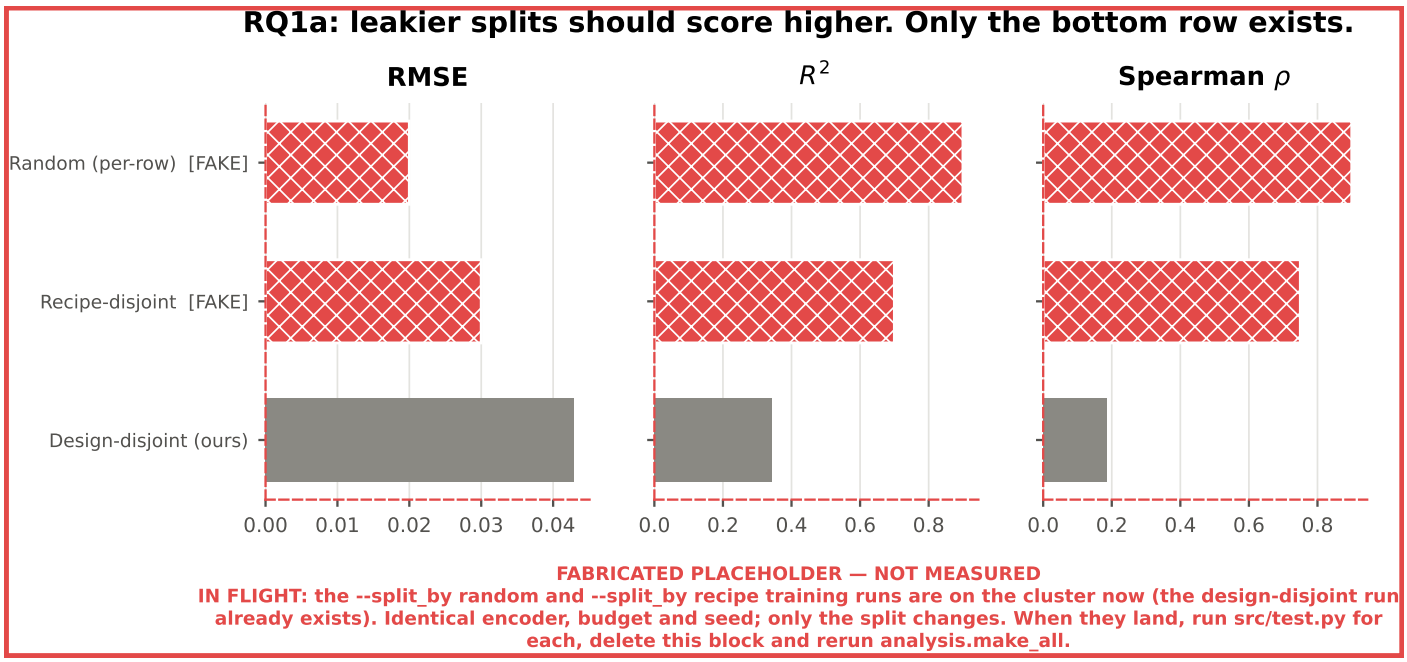

Figure 4.13: **MOSTLY FABRICATED.** How much of the score is design recognition. Only the design-disjoint row is measured; the random and recipe-disjoint rows are placeholders for two training runs that have not been made. Until they exist, the comparison against published numbers obtained under leakier protocols cannot be made quantitative.

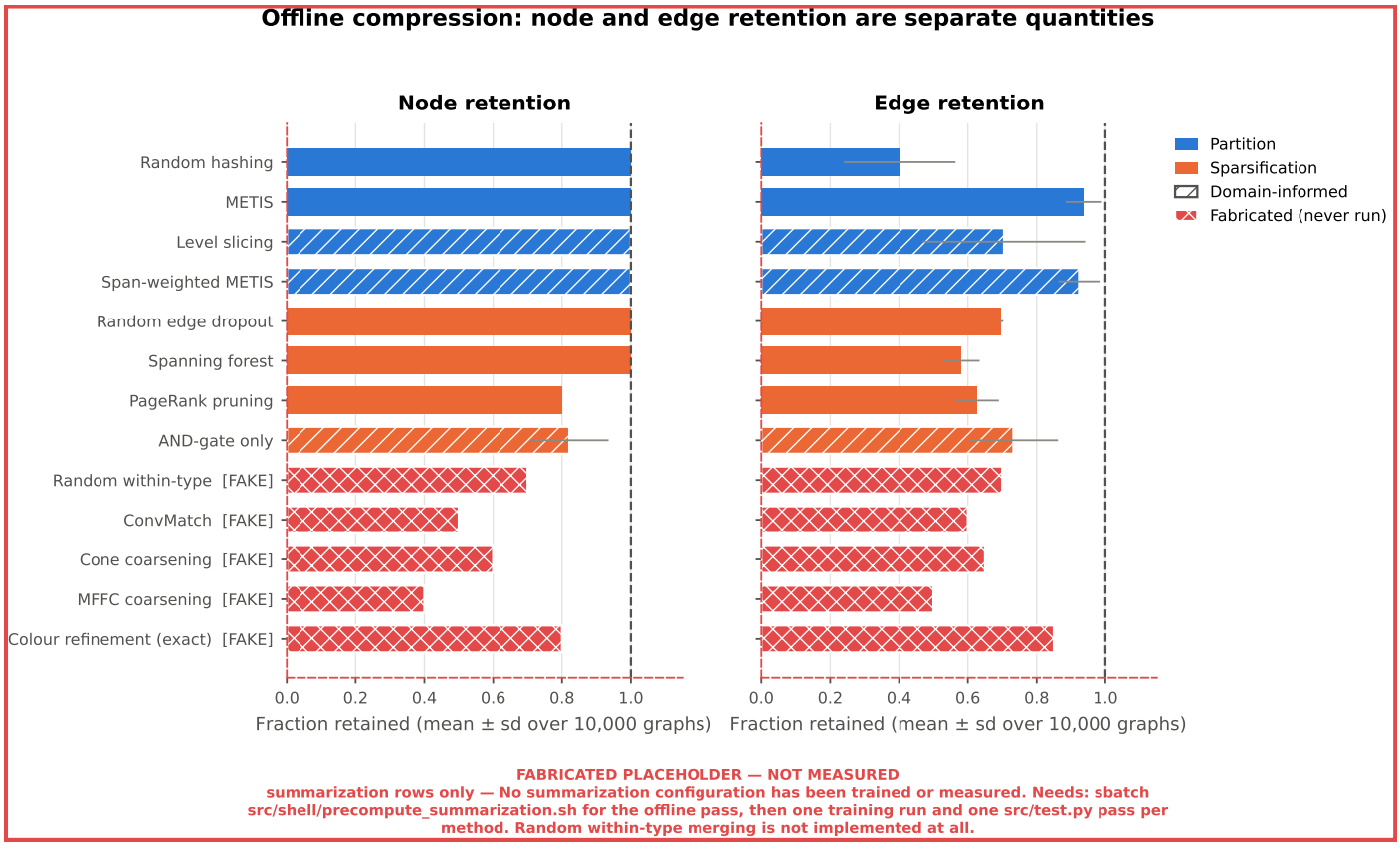

Figure 4.14: **PARTLY FABRICATED.** Offline compression, reported separately for nodes and edges. Partitioning keeps every node by construction, so a single "compression" figure would hide the whole distinction (2.1.5). The summarization rows are invented.

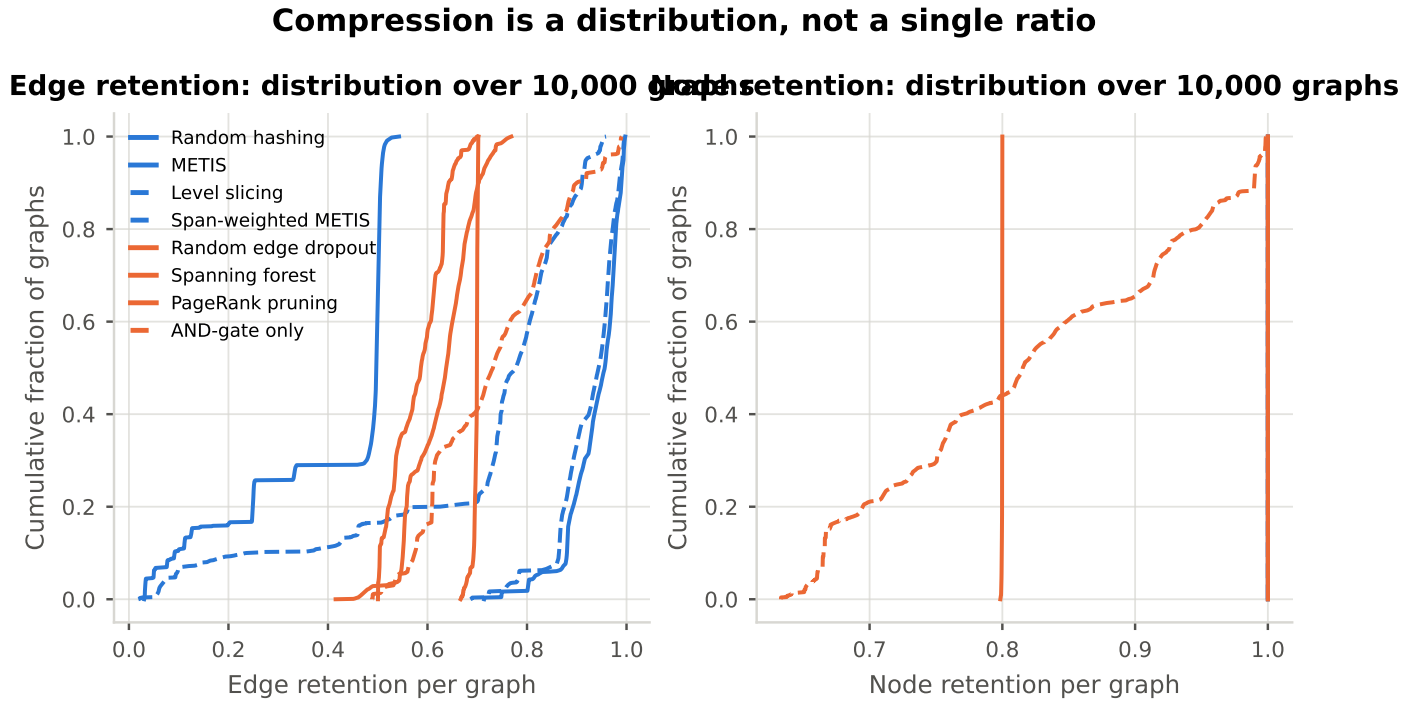

Figure 4.15: Per-graph retention over 10,000 graphs. Compression is a distribution rather than a ratio: the parameter-free methods' compression is a property of each circuit, and PageRank's node retention is nearly a point mass at its configured 0.8 while AND-gate-only's is not, which is what makes the pairing between them approximate rather than exact.

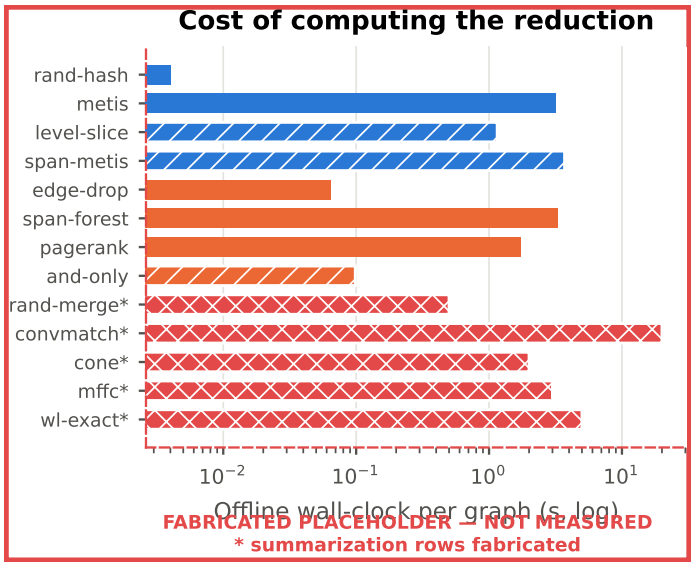

Figure 4.16: **PARTLY FABRICATED.** Wall-clock cost of computing each reduction, per graph. Spans two orders of magnitude across methods.

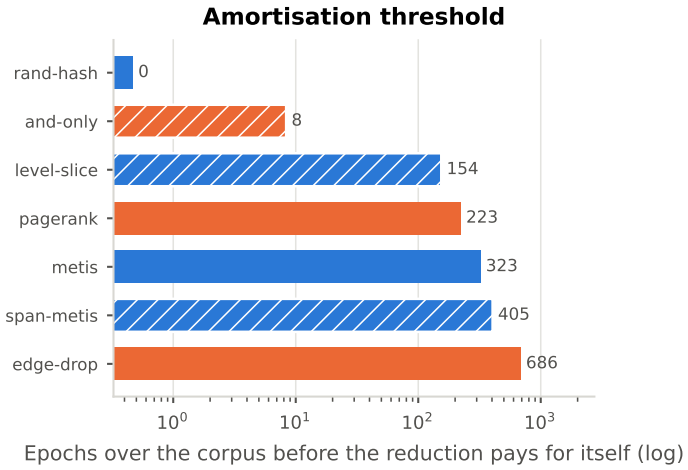

Figure 4.17: How many epochs over the corpus a cached reduction must serve before its offline cost is repaid by the training time it saves. This is the form 3.4.3 asks the offline cost to be reported in, rather than as a raw number of seconds.

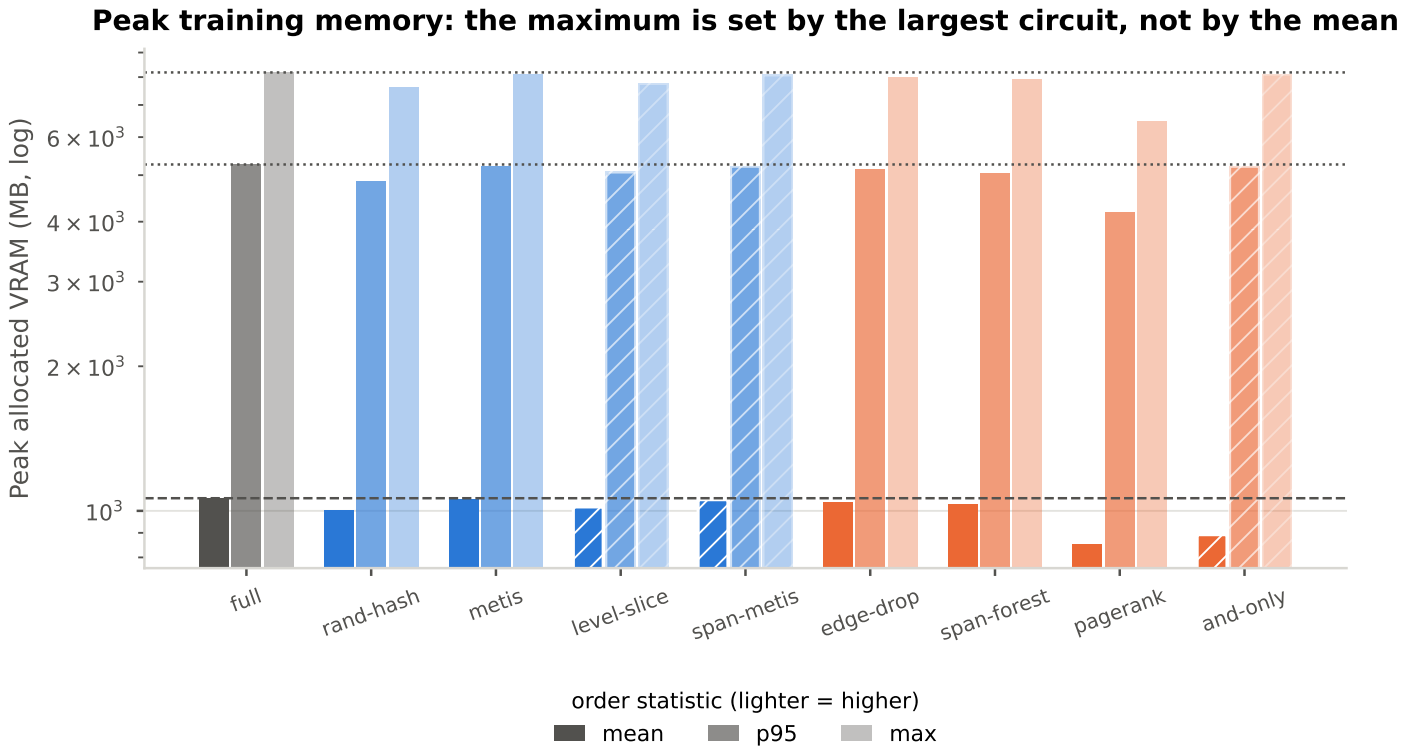

Figure 4.18: Peak allocated device memory per configuration at three order statistics. The mean, p95 and max disagree by an order of magnitude, and that gap is the point: a graph larger than the batch budget forms a batch of its own, so the maximum is floored by the largest circuit in the corpus no matter what the reduction did to the average one.

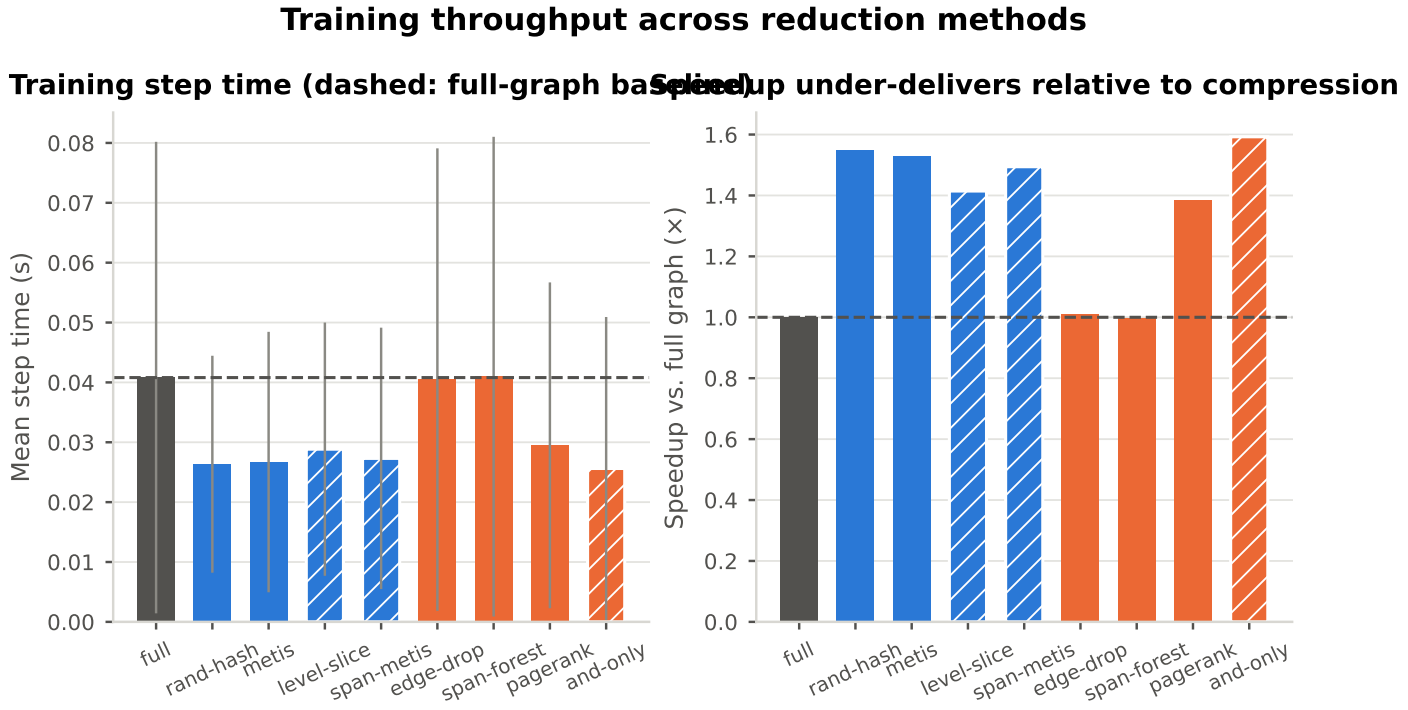

Figure 4.19: Mean step time and speedup against the full-graph baseline. Speedup under-delivers relative to compression because the kernels are latency-bound on message-passing gather/scatter (3.4.2), so removing work does not translate proportionally into wall-clock.

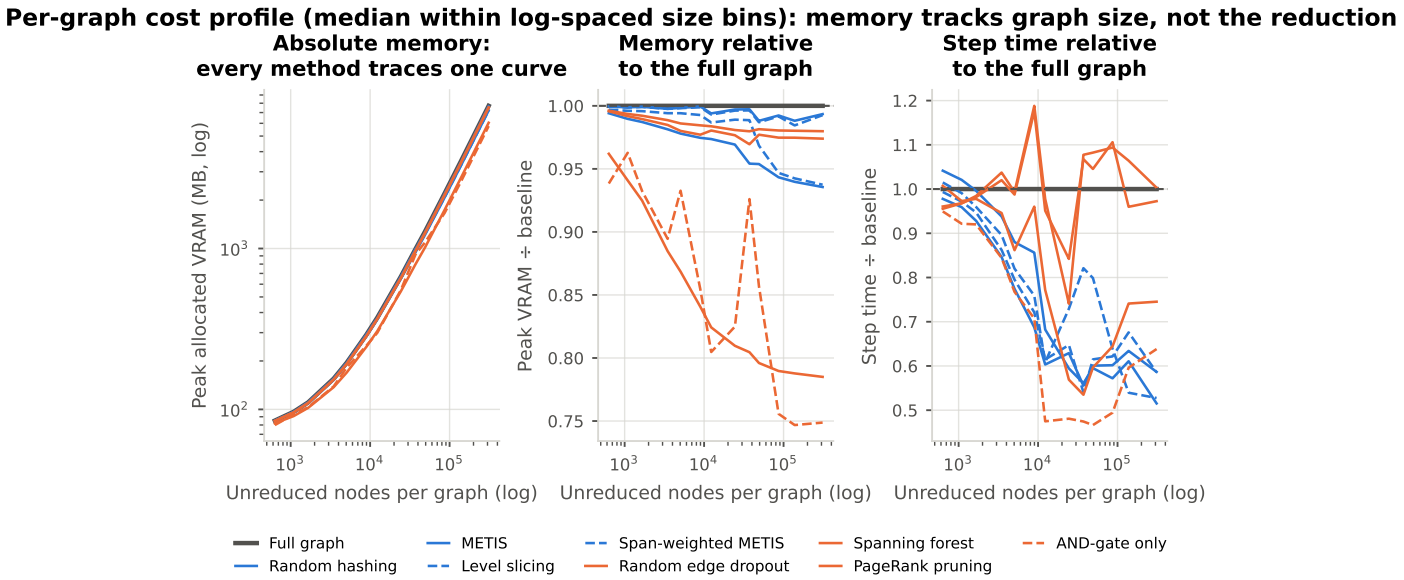

Figure 4.20: Peak memory and step time as a function of graph size. In absolute terms every method traces the same memory curve, because memory tracks the graph rather than the reduction, so the two ratio panels are where any difference between methods is visible at all. Reporting a single peak per configuration would conflate the reduction's effect with which graphs happened to land in the peak batch.

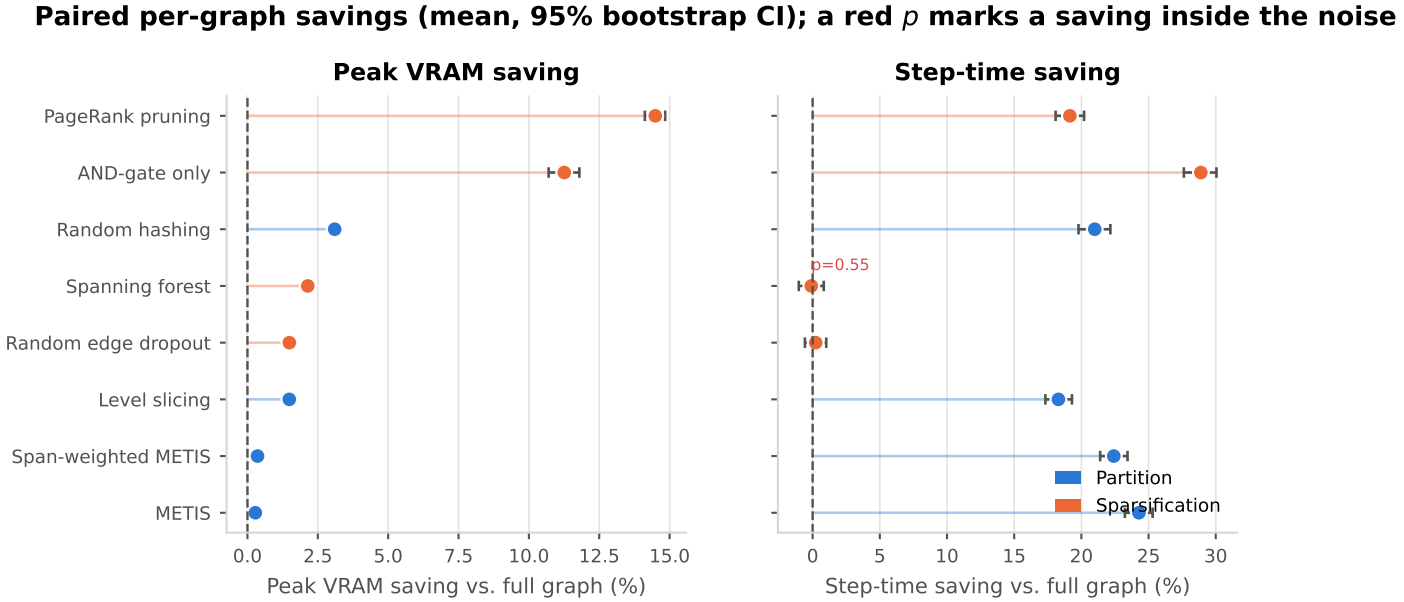

Figure 4.21: Per-graph peak-memory and step-time savings against the baseline, paired by graph id, with 95% percentile bootstrap intervals. A red p-value marks a Wilcoxon signed-rank test that fails to separate the saving from noise: spanning forest's step-time saving is inside the noise, despite removing 42% of edges.

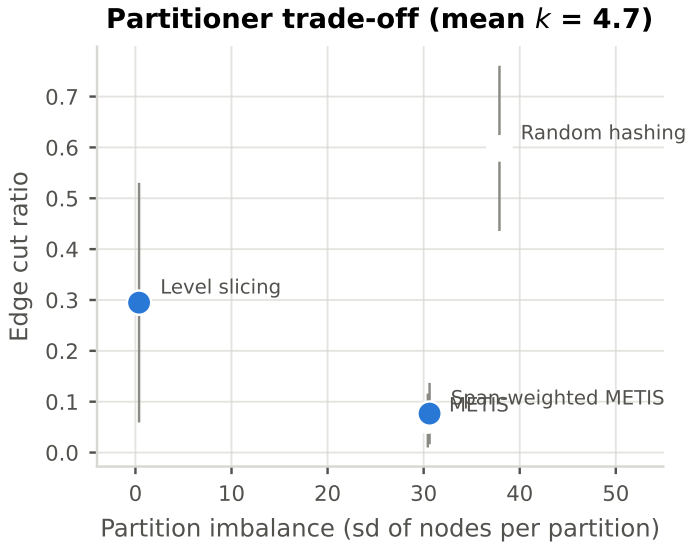

Figure 4.22: Edge cut against partition imbalance for the four partitioners at the same k. Since all four keep every node and differ only in which edges they cut, this plane is their entire comparison.

Table 4.5: RQ2 efficiency profile. Node and edge retention are reported separately because several methods preserve node counts exactly. Savings are per-graph means paired against the full-graph baseline.

Method	Family	Kind	Node ret.	Edge ret.	Offline (s/graph)	Step time (s)	VRAM mean (MB)	VRAM saving (%)	Time saving (%)
Random hashing	Partition	Generic	1.000	0.402	0.004	0.026	1002.217	3.100	20.996
METIS	Partition	Generic	1.000	0.937	3.196	0.027	1056.507	0.279	24.284
Level slicing	Partition	Domain	1.000	0.705	1.151	0.029	1019.120	1.487	18.291
Span-weighted METIS	Partition	Domain	1.000	0.923	3.701	0.027	1055.088	0.357	22.412
Random edge dropout	Sparsification	Generic	1.000	0.697	0.064	0.040	1040.969	1.489	0.229
Spanning forest	Sparsification	Generic	1.000	0.581	3.309	0.041	1031.674	2.142	-0.101
PageRank pruning	Sparsification	Generic	0.800	0.627	1.739	0.029	850.969	14.492	19.145
AND-gate only	Sparsification	Domain	0.821	0.733	0.099	0.026	892.163	11.257	28.893
[TODO/FAKE] Random within-type	Summarization	Generic	0.700	0.700	0.500	0.030	800.000	–	–
[TODO/FAKE] ConvMatch	Summarization	Generic	0.500	0.600	20.000	0.028	700.000	–	–
[TODO/FAKE] Cone coarsening	Summarization	Domain	0.600	0.650	2.000	0.026	750.000	–	–
[TODO/FAKE] MFFC coarsening	Summarization	Domain	0.400	0.500	3.000	0.024	600.000	–	–
[TODO/FAKE] Colour refinement (exact)	Summarization	Domain	0.800	0.850	5.000	0.032	900.000	–	–

Rows marked [TODO/FAKE] are invented. No summarization configuration has been trained or measured. Needs: sbatch src/shell/precompute_summarization.sh for the offline pass, then one training run and one src/test.py pass per method. Random within-type merging is not implemented at all.

Table 4.6: Per-graph savings against the full-graph baseline, paired by graph id, with percentile bootstrap intervals and a Wilcoxon signed-rank test on the paired deltas. Savings are not normally distributed, so bare means would be misleading.

Method	Paired graphs	VRAM saving (%)	95% CI low	95% CI high	Time saving (%)	95% CI low	95% CI high	Wilcoxon p (time)
PageRank pruning	1000	14.492	14.121	14.842	19.145	18.081	20.203	0.000
AND-gate only	1000	11.257	10.702	11.794	28.893	27.627	30.054	0.000
Random hashing	1000	3.100	2.984	3.216	20.996	19.785	22.157	0.000
Spanning forest	1000	2.142	2.080	2.202	-0.101	-1.031	0.826	0.549
Random edge dropout	1000	1.489	1.451	1.526	0.229	-0.574	1.012	0.001
Level slicing	1000	1.487	1.383	1.596	18.291	17.322	19.300	0.000
Span-weighted METIS	1000	0.357	0.336	0.378	22.412	21.396	23.435	0.000
METIS	1000	0.279	0.262	0.297	24.284	23.237	25.303	0.000

Table 4.7: The four partitioners keep every node and differ only in which edges they cut, at the same k. That makes them the cleanest domain-informed / generic pairs in the thesis.

Partitioner	Kind	Mean k	Edge cut	Edge cut sd	Imbalance (sd nodes)	Max part. nodes	Offline (s)
Random hashing	Generic	4.662	0.598	0.162	37.891	6382	0.004
METIS	Generic	4.662	0.063	0.053	30.433	6370	3.196
Level slicing	Domain	4.662	0.295	0.236	0.378	6334	1.151
Span-weighted METIS	Domain	4.662	0.077	0.060	30.616	6371	3.701

4.7.4 RQ3: Predictive Retention and Trade-offs

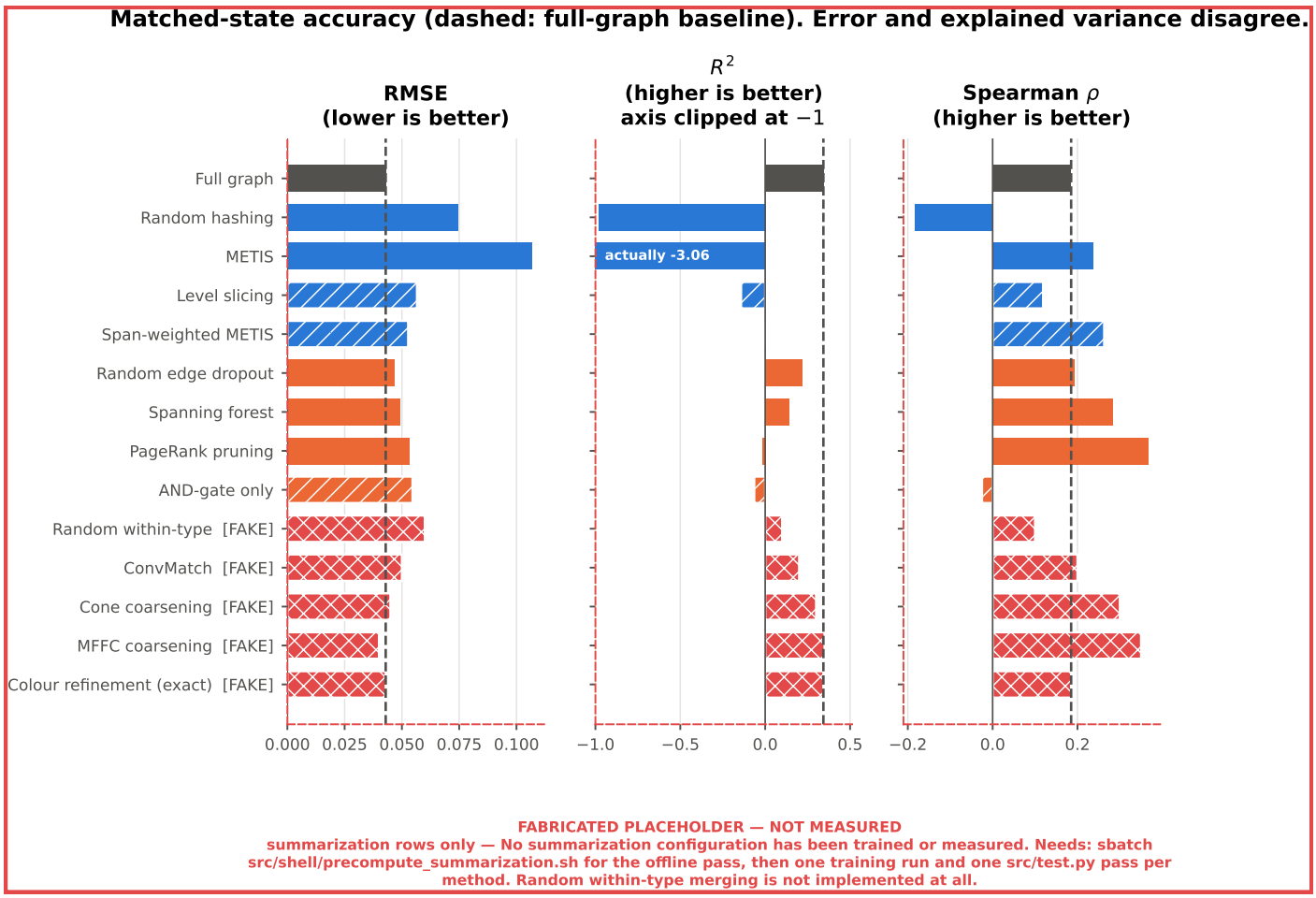

Figure 4.23: **PARTLY FABRICATED.** RMSE, R^2 and Spearman under the reduction each model was trained with, against the full-graph baseline. The three metrics disagree about the ranking, which is exactly the failure mode 3.4.3 exists to catch. The summarization rows are invented.

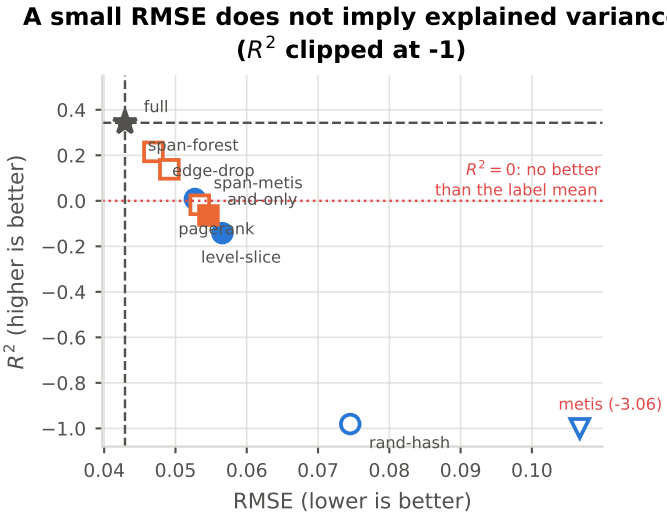

Figure 4.24: RMSE against R^2, one point per configuration. Several configurations hold RMSE close to the baseline while R^2 falls below zero. This is the same dissociation Khadka et al. [2026] report on a coarsened EDA graph, and the reason error and explained variance are never reported separately here.

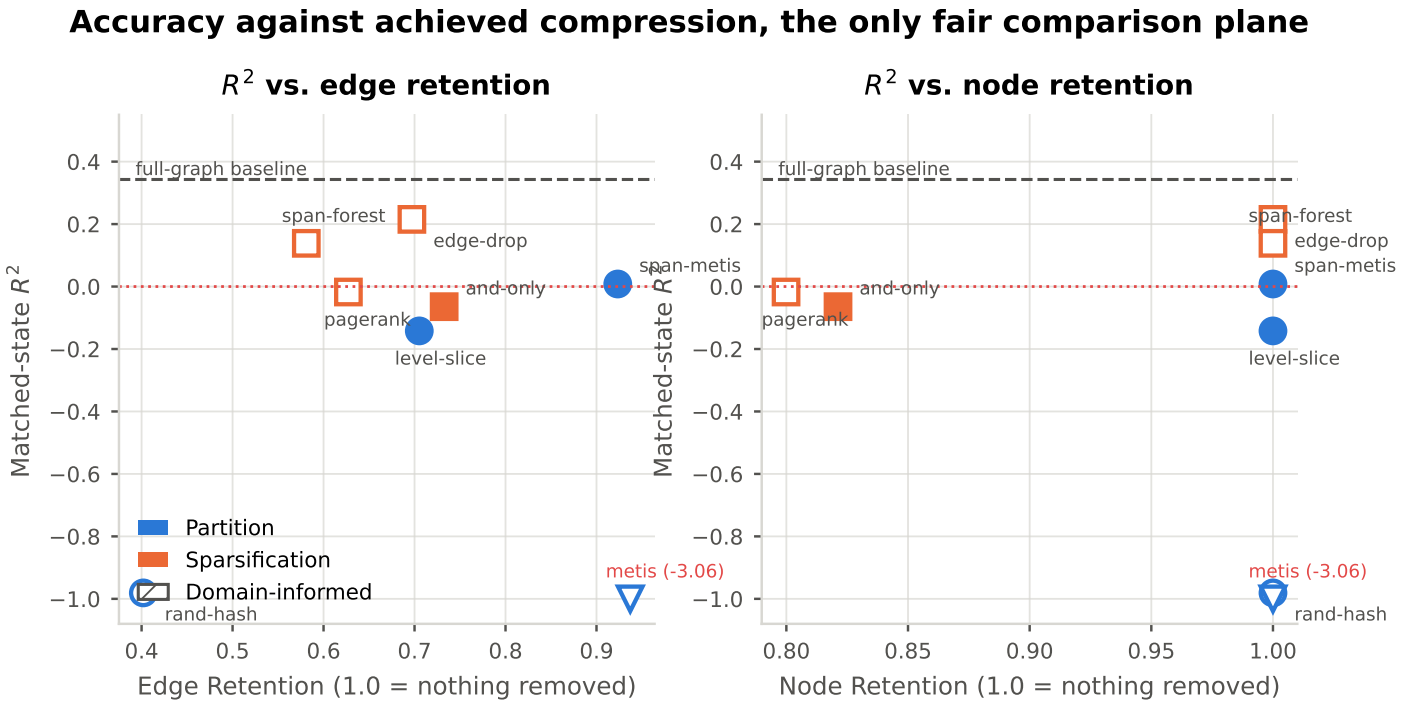

Figure 4.25: Matched-state R^2 against edge and node retention. Without this plane, "which method is best" confounds a genuinely better method with one that simply reduced less (4.3.3).

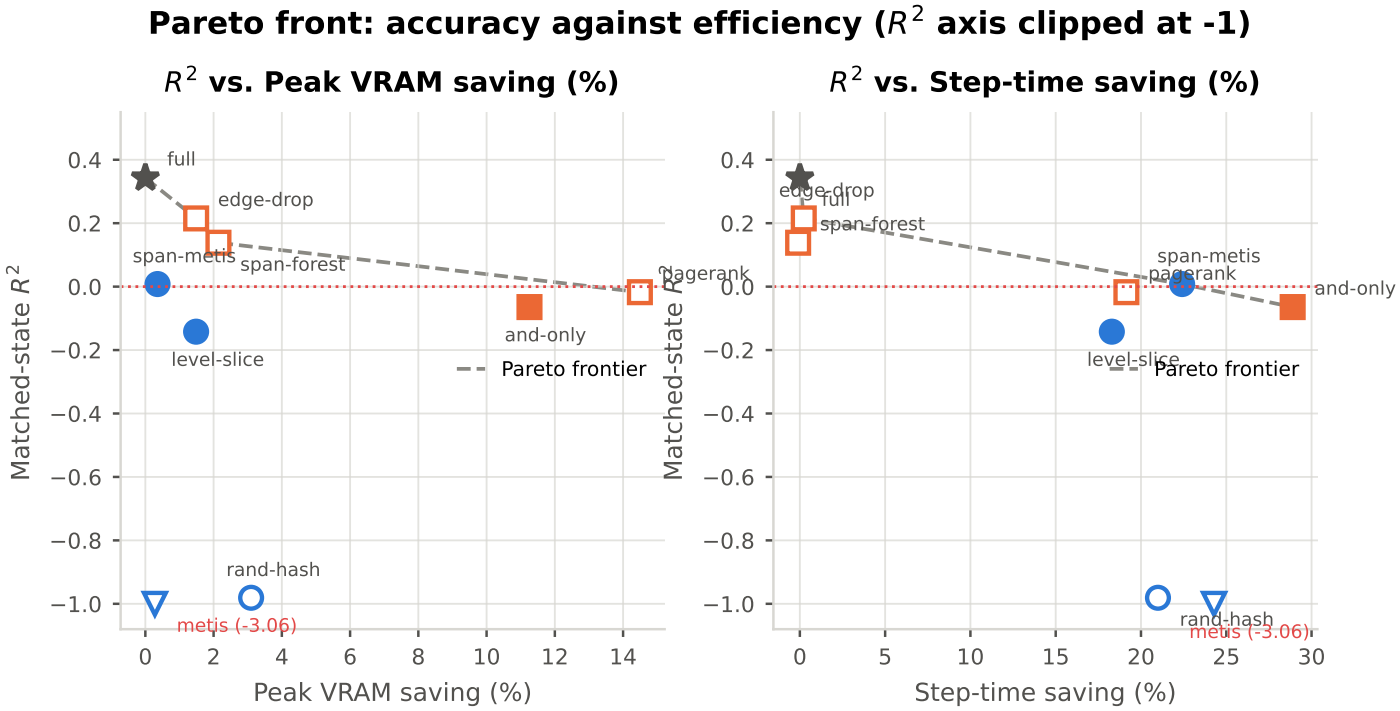

Figure 4.26: Accuracy against efficiency, with the dominance frontier drawn. The full-graph baseline sits on the same axes at zero saving, so "no reduction" competes directly against every reduction.

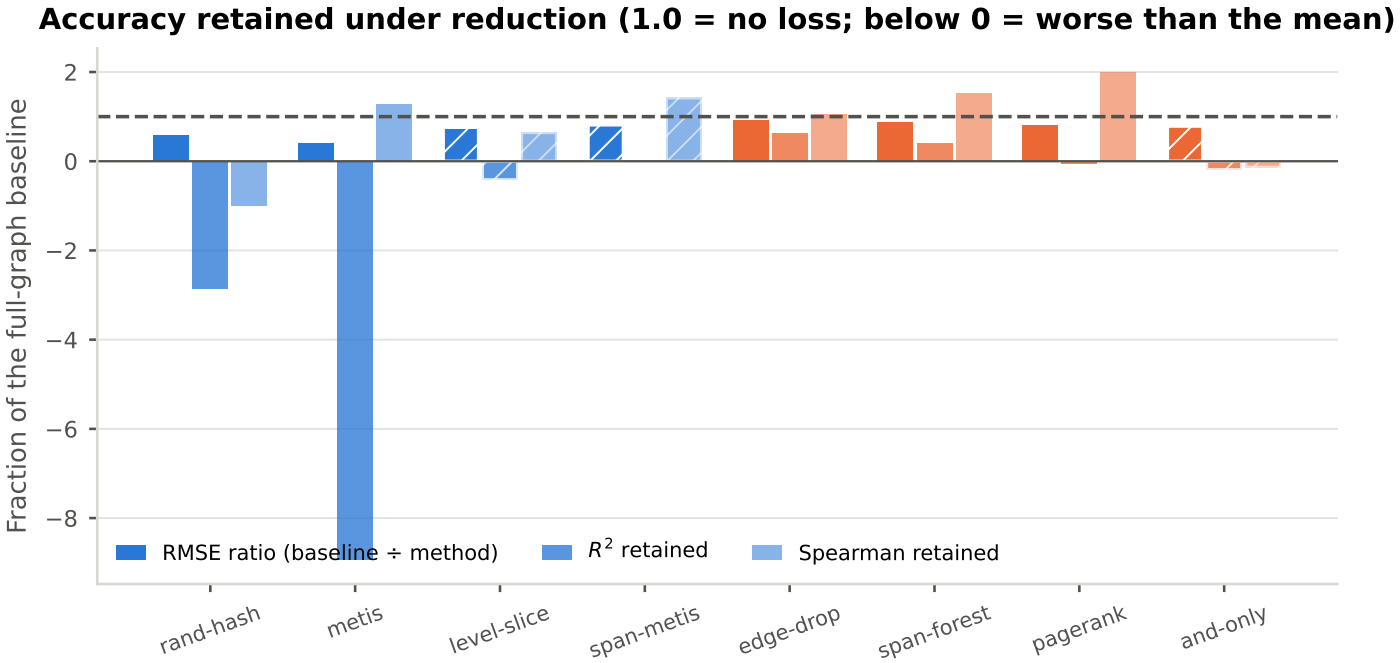

Figure 4.27: Each metric as a fraction of the full-graph baseline, so that this figure lines up row-for-row against the efficiency ranking of 4.2.4.

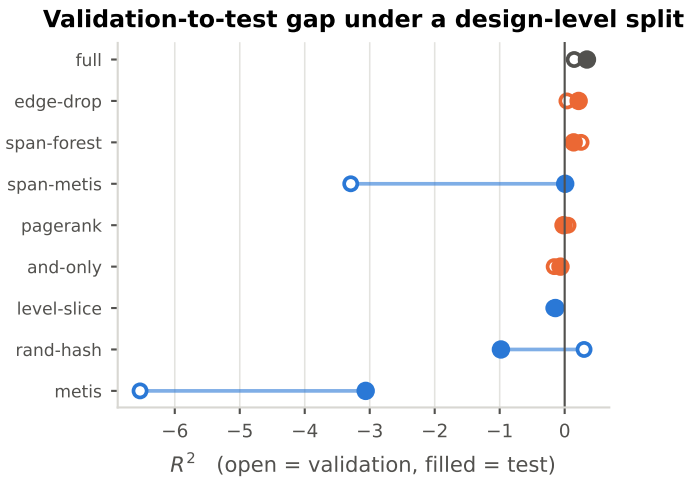

Figure 4.28: The same configurations scored on validation and on test. Under a design-level split these are different circuits rather than two samples of one population, so the size of this gap bounds how much of the validation-selected model choice transfers.

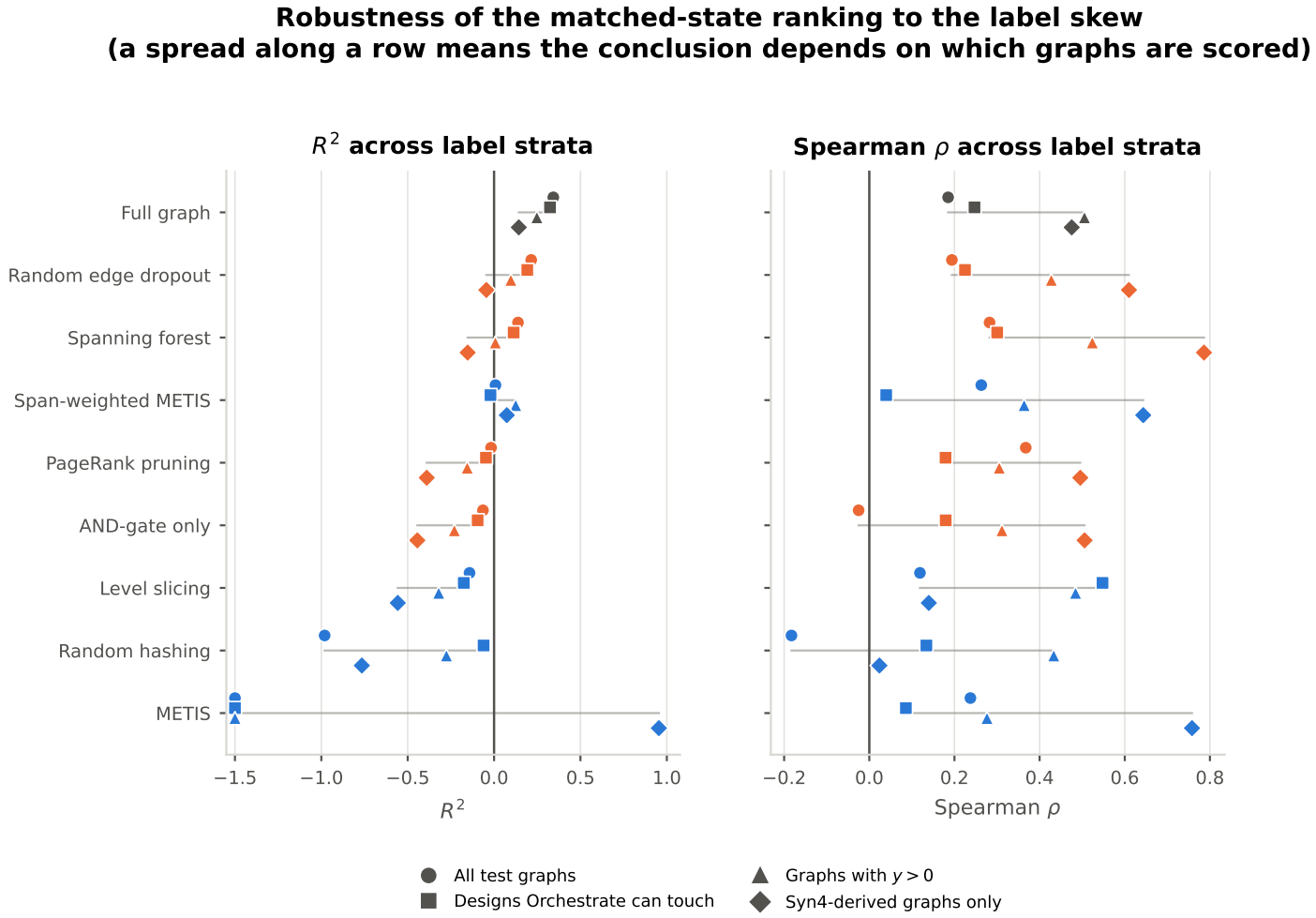

Figure 4.29: The matched-state result re-scored on four subsets of the same test split, from the persisted per-graph predictions, with no retraining. Read the strata as answering different questions rather than as more or less trustworthy: excluding the two fixed-point designs selects on a design property; restricting to $y > 0$ selects on the outcome variable and so its R^2 is not comparable to the pooled column; the Syn4 column is the easiest stratum. Two conclusions hold across all four: no reduction beats the full graph on R^2, and sparsification beats partitioning. Two do not: random hashing's R^2 moves from -0.98 to -0.06 once the fixed-point designs are excluded, and METIS is worst overall yet best on Syn4-derived graphs, which is the signature of a model that collapsed onto predicting high optimizability rather than of a failure of partitioning.

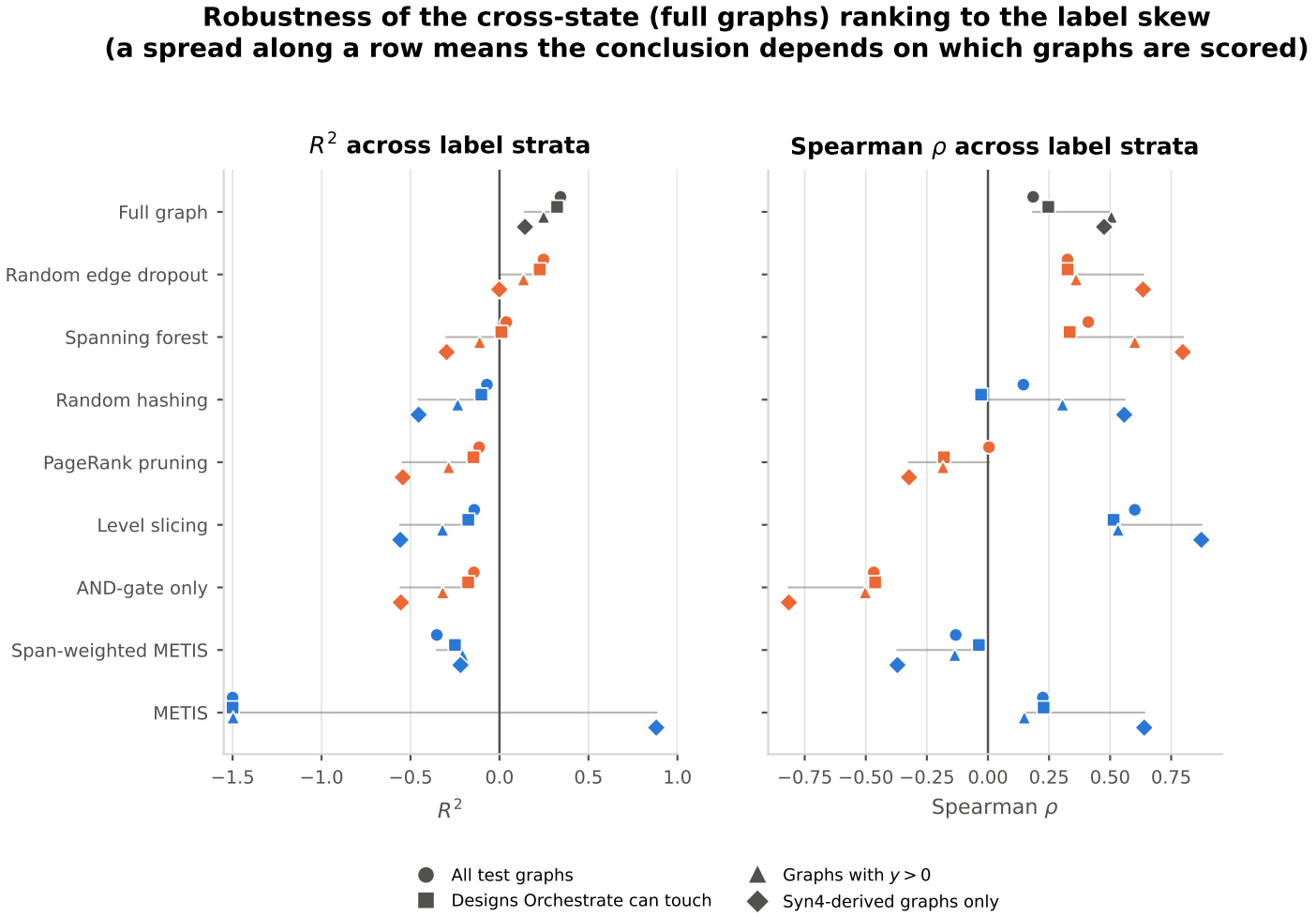

Figure 4.30: The same treatment for the full-graph evaluation pass. Ranking behaviour separates sharply from error here: level slicing reaches a Spearman of 0.601 on full graphs, above the full-graph baseline's 0.185, while AND-gate-only reaches -0.467, ordering circuits backwards. Neither is visible in R^2 alone.

Table 4.8: RQ3 matched-state accuracy: models trained and tested under the same reduction. RMSE, R^2 and Spearman are reported together throughout – a reduction can hold mean error steady while destroying explained variance.

Method	Family	Kind	Smooth L1	RMSE	R^2	Spearman	R^2 retained
Full graph	Baseline	Generic	0.014	0.043	0.343	0.185	1.000
Random hashing	Partition	Generic	0.038	0.075	-0.981	-0.183	-2.861
METIS	Partition	Generic	0.055	0.107	-3.062	0.237	-8.932
Level slicing	Partition	Domain	0.018	0.057	-0.142	0.119	-0.414
Span-weighted METIS	Partition	Domain	0.034	0.053	0.008	0.263	0.024
Random edge dropout	Sparsification	Generic	0.016	0.047	0.215	0.194	0.627
Spanning forest	Sparsification	Generic	0.016	0.049	0.138	0.282	0.404
PageRank pruning	Sparsification	Generic	0.019	0.053	-0.018	0.367	-0.051
AND-gate only	Sparsification	Domain	0.018	0.055	-0.064	-0.025	-0.188
[TODO/FAKE] Random within-type	Summarization	Generic	–	0.060	0.100	0.100	0.292
[TODO/FAKE] ConvMatch	Summarization	Generic	–	0.050	0.200	0.200	0.583
[TODO/FAKE] Cone coarsening	Summarization	Domain	–	0.045	0.300	0.300	0.875
[TODO/FAKE] MFFC coarsening	Summarization	Domain	–	0.040	0.350	0.350	1.021
[TODO/FAKE] Colour refinement (exact)	Summarization	Domain	–	0.043	0.343	0.185	1.000

Rows marked [TODO/FAKE] are invented. No summarization configuration has been trained or measured. Needs: sbatch src/shell/precompute_summarization.sh for the offline pass, then one training run and one src/test.py pass per method. Random within-type merging is not implemented at all.

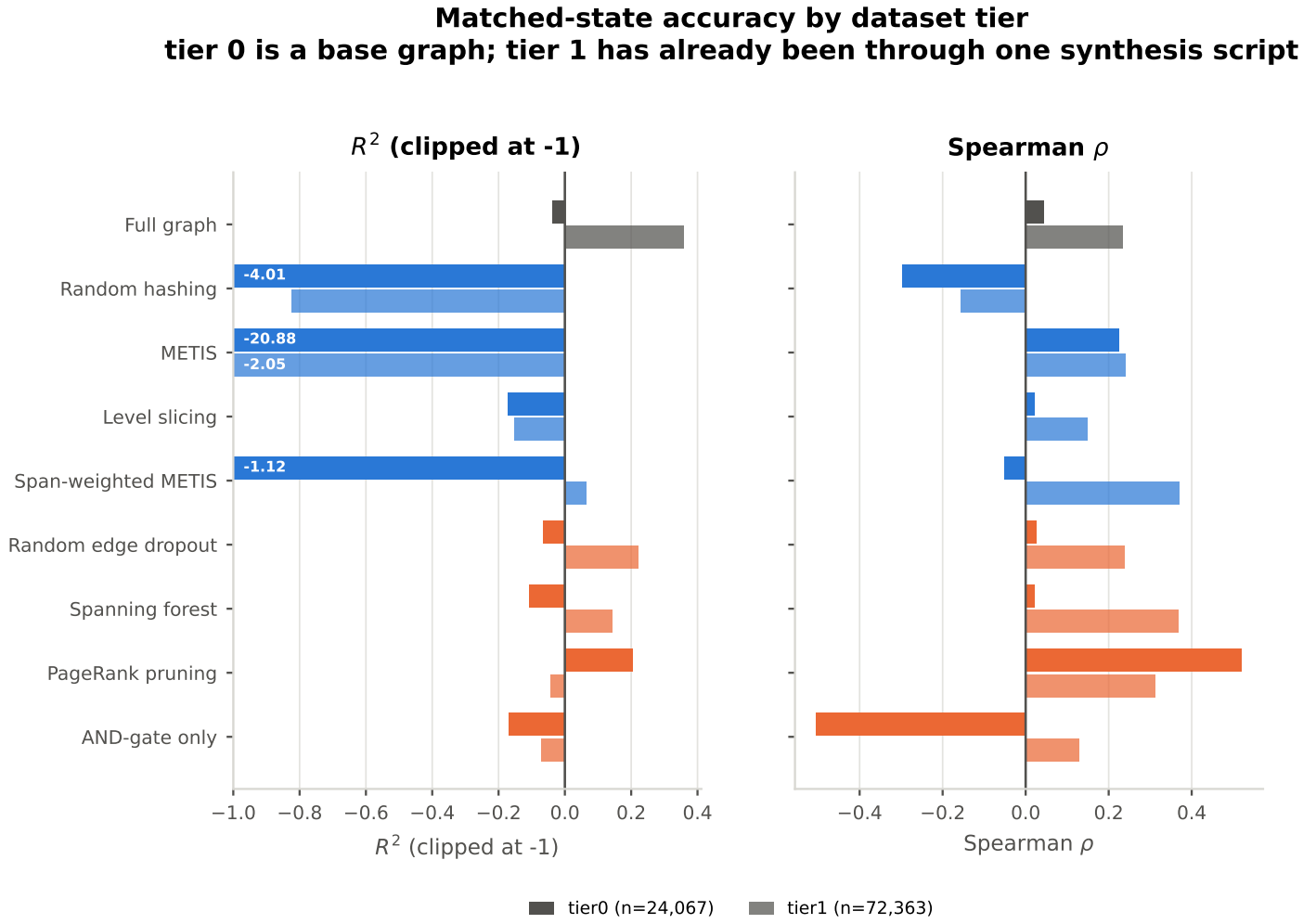

Figure 4.31: Every configuration split across the two stored tiers. The full-graph baseline scores $R^2 = -0.038$ on tier-0 base graphs against $+0.358$ on tier-1: its absolute error on tier 0 is smaller (RMSE 0.025 vs 0.047), but tier-0 labels have so much less spread that the model fails to beat their mean. PageRank pruning is the exception, scoring $R^2 = 0.204$ and $\rho = 0.521$ on tier 0, better than the unreduced baseline on that tier.

Table 4.9: Accuracy against efficiency, ranked by matched-state R^2.

Method	R^2	**Spearman**	**VRAM saving (%)**	**Time saving (%)**
Full graph	0.343	0.185	0.000	0.000
Random edge dropout	0.215	0.194	1.489	0.229
Spanning forest	0.138	0.282	2.142	-0.101
Span-weighted METIS	0.008	0.263	0.357	22.412
PageRank pruning	-0.018	0.367	14.492	19.145
AND-gate only	-0.064	-0.025	11.257	28.893
Level slicing	-0.142	0.119	1.487	18.291
Random hashing	-0.981	-0.183	3.100	20.996
METIS	-3.062	0.237	0.279	24.284

Table 4.10: Matched-state accuracy re-scored on four subsets of the same test split, from the persisted per-graph predictions. The pooled label is 49% exactly zero, so a pooled R^2 is carried by a minority of graphs; a conclusion that survives all four columns is a property of the reduction rather than of the label distribution.

Method	R^2 All test graphs	R^2 Designs Orchestrate can touch	R^2 Graphs with $y > 0$	R^2 Syn4-derived graphs only	ρ All test graphs	ρ Designs Orchestrate can touch	ρ Graphs with $y > 0$	ρ Syn4-derived graphs only
Random edge dropout	0.215	0.192	0.097	-0.045	0.194	0.225	0.427	0.610
Spanning forest	0.138	0.113	0.007	-0.153	0.282	0.300	0.524	0.786
Span-weighted METIS	0.008	-0.021	0.125	0.074	0.263	0.040	0.364	0.643
PageRank pruning	-0.018	-0.047	-0.155	-0.390	0.367	0.179	0.305	0.496
AND-gate only	-0.064	-0.095	-0.230	-0.444	-0.025	0.179	0.312	0.506
Level slicing	-0.142	-0.175	-0.320	-0.558	0.119	0.548	0.484	0.140
Random hashing	-0.981	-0.061	-0.276	-0.766	-0.183	0.134	0.433	0.024
METIS	-3.062	-3.181	-1.997	0.954	0.237	0.086	0.276	0.758

Strata: all test graphs; excluding the two designs on which Orchestrate is at a fixed point (16384, 8192, max y = 0.0003 over 32,000 graphs); graphs with a non-zero target; and graphs derived from Syn4, the one source script that leaves substantial optimizability behind.

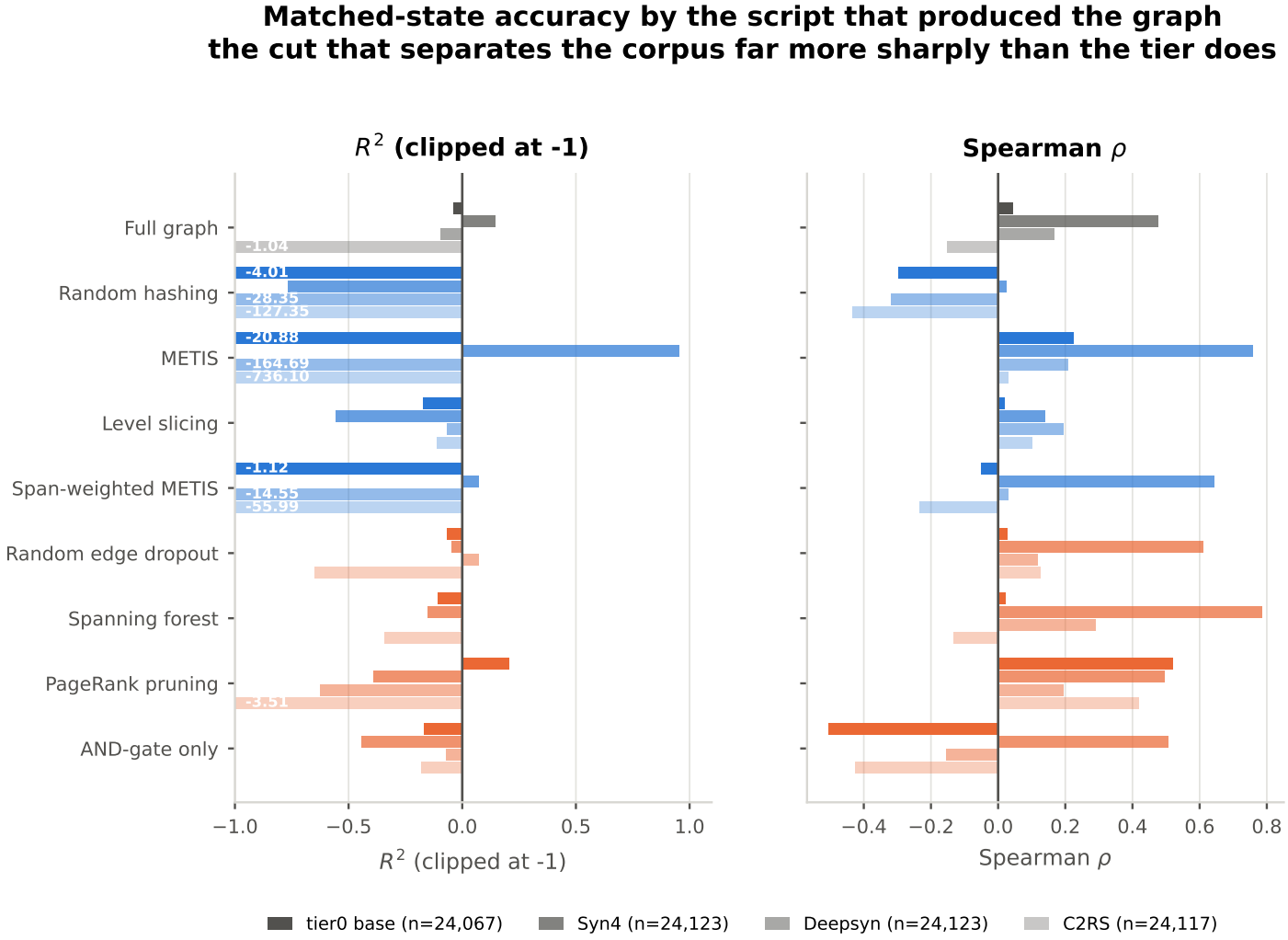

Figure 4.32: The same partition taken by the script that produced each graph, which separates the corpus far more sharply than the tier does (4.10). Every configuration degrades from Syn4-derived graphs to C2RS-derived ones, and the partitioners degrade catastrophically: METIS reaches $R^2 = -736$ on C2RS-derived graphs, where the labels are near zero and it predicts high optimizability regardless.

Table 4.11: Matched-state accuracy partitioned two ways: by dataset tier, and by the synthesis script that produced the graph. Both partition the same test split, so the columns within each cut are disjoint. The source script separates the corpus far more sharply than the tier does.

Method	R^2 C2RS	R^2 Deepsyn	R^2 Syn4	R^2 tier0	R^2 tier0 base	R^2 tier1	ρ C2RS	ρ Deepsyn	ρ Syn4	ρ tier0	ρ tier0 base	ρ tier1
Full graph	-1.039	-0.097	0.143	-0.038	-0.038	0.358	-0.153	0.167	0.475	0.044	0.044	0.234
Random hashing	-127.354	-28.352	-0.766	-4.012	-4.012	-0.823	-0.433	-0.319	0.024	-0.298	-0.298	-0.156
METIS	-736.099	-164.691	0.954	-20.880	-20.880	-2.046	0.030	0.208	0.758	0.225	0.225	0.241
Level slicing	-0.111	-0.068	-0.558	-0.172	-0.172	-0.154	0.102	0.193	0.140	0.021	0.021	0.149
Span-weighted METIS	-55.986	-14.553	0.074	-1.116	-1.116	0.064	-0.235	0.030	0.643	-0.050	-0.050	0.370
Random edge dropout	-0.649	0.071	-0.045	-0.065	-0.065	0.223	0.125	0.118	0.610	0.026	0.026	0.239
Spanning forest	-0.342	-0.006	-0.153	-0.108	-0.108	0.143	-0.133	0.289	0.786	0.021	0.021	0.368
PageRank pruning	-3.507	-0.626	-0.390	0.204	0.204	-0.043	0.418	0.194	0.496	0.521	0.521	0.312
AND-gate only	-0.178	-0.070	-0.444	-0.169	-0.169	-0.071	-0.427	-0.154	0.506	-0.504	-0.504	0.129

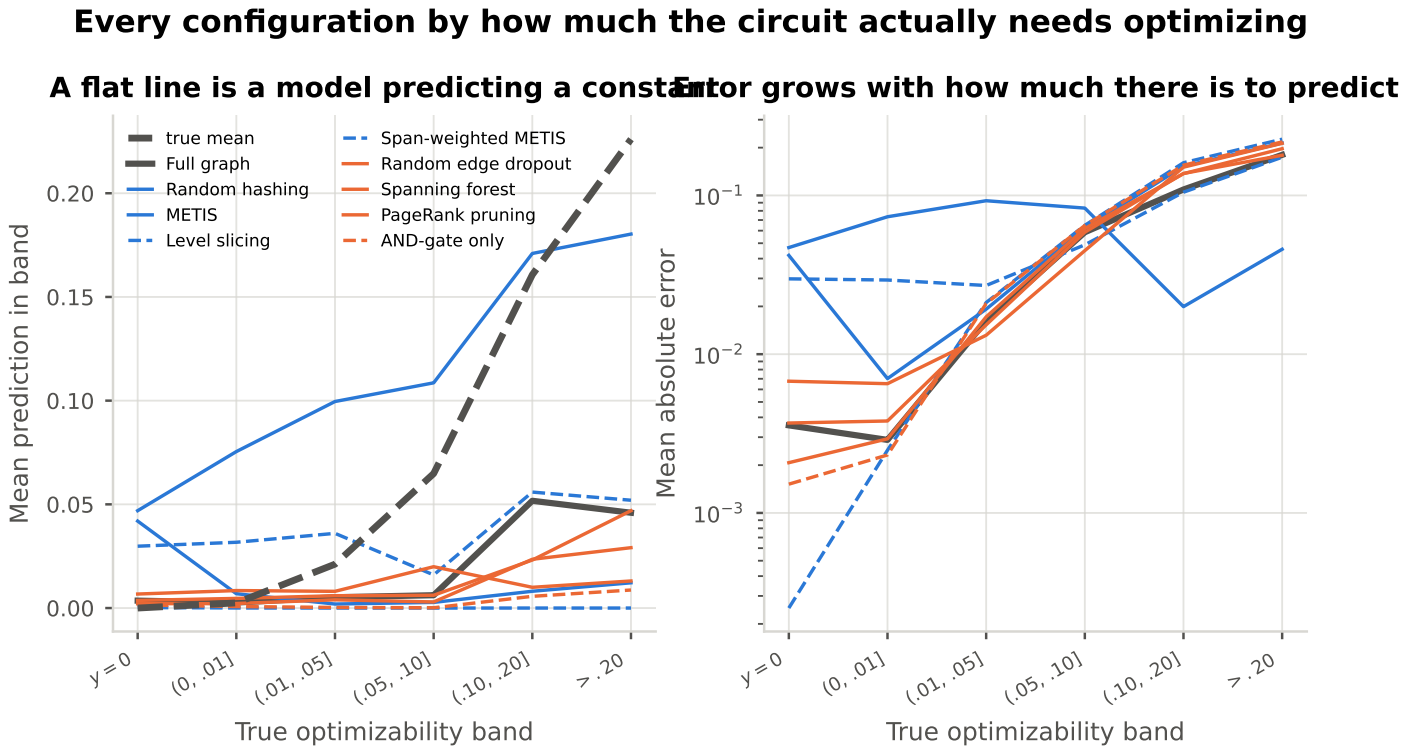

Figure 4.33: Every configuration as a function of how optimizable the circuit truly is. Level slicing predicts 0.0000 in every band, a fully collapsed model that the pooled metrics do not expose. METIS over-predicts everywhere, which is why its pooled R^2 is -3.06, yet that same miscalibration makes it the *most* accurate configuration on the highly optimizable circuits: MAE 0.046 above $y = 0.20$ against the unreduced baseline's 0.180. If the use case is finding circuits worth optimizing rather than scoring all of them, the pooled ranking inverts.

4.7.5 RQ4: Value of Domain-Informed Adaptation

Table 4.12: Mean prediction and mean absolute error within bands of the true optimizability. R^2 is not reported: inside a narrow band there is almost no label variance to divide by. Compare each method's predicted mean against the true mean in the first row. A row that stays flat across the bands is a model that has collapsed to a constant.

Method	pred $y = 0$	pred (0, .01]	pred (.01, .05]	pred (.05, .10]	pred (.10, .20]	pred $> .20$	MAE $y = 0$	MAE (0, .01]	MAE (.01, .05]	MAE (.05, .10]	MAE (.10, .20]	MAE $> .20$
(true mean in band)	0.000	0.002	0.021	0.065	0.161	0.226	–	–	–	–	–	–
Full graph	0.004	0.003	0.005	0.006	0.052	0.046	0.004	0.003	0.016	0.059	0.109	0.180
Random hashing	0.042	0.007	0.002	0.003	0.008	0.012	0.042	0.007	0.019	0.062	0.153	0.214
METIS	0.047	0.075	0.100	0.109	0.171	0.180	0.047	0.073	0.093	0.083	0.020	0.046
Level slicing	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.002	0.021	0.065	0.161	0.226
Span-weighted METIS	0.030	0.032	0.036	0.016	0.056	0.052	0.030	0.029	0.027	0.049	0.105	0.174
Random edge dropout	0.004	0.005	0.006	0.006	0.023	0.047	0.004	0.004	0.015	0.059	0.138	0.179
Spanning forest	0.002	0.002	0.004	0.003	0.024	0.029	0.002	0.003	0.017	0.062	0.137	0.197
PageRank pruning	0.007	0.008	0.008	0.020	0.010	0.013	0.007	0.007	0.013	0.045	0.151	0.213
AND-gate only	0.002	0.001	0.000	0.000	0.006	0.009	0.002	0.002	0.021	0.065	0.155	0.217

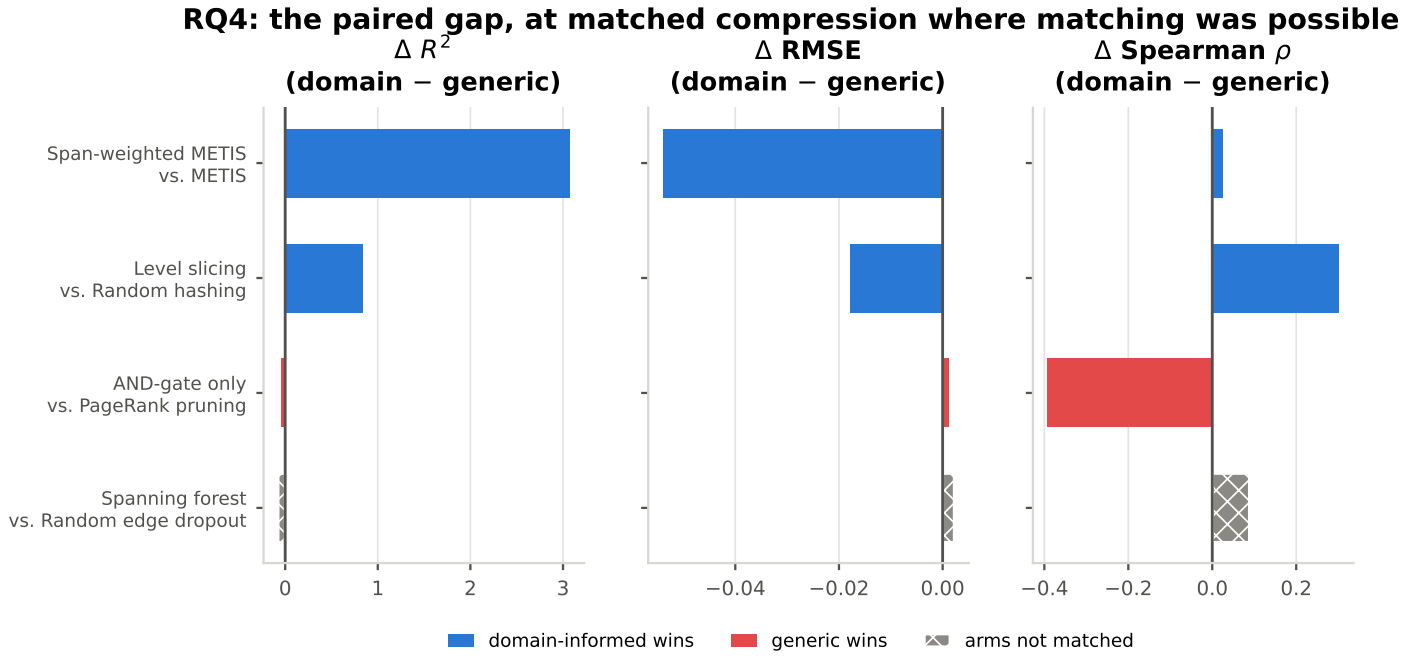

Figure 4.34: Domain-informed minus generic within each pairing. The first two pairings are matched by construction (same k, all nodes kept, only the cut rule differs); the third is matched by calibration; the fourth is *not* matched and is hatched accordingly. Read every bar against 4.35.

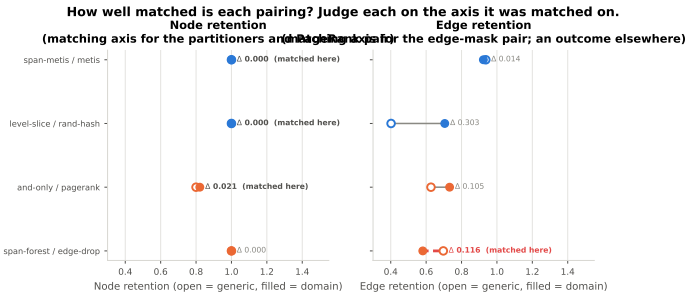

Figure 4.35: How closely matched each RQ4 pairing actually is. The spanning-forest / random-edge-dropout pair is far apart because the configured drop rate of 0.3 leaves 69.7% of edges against spanning forest's 58.1%; raising the rate to approximately 0.419 would match them.

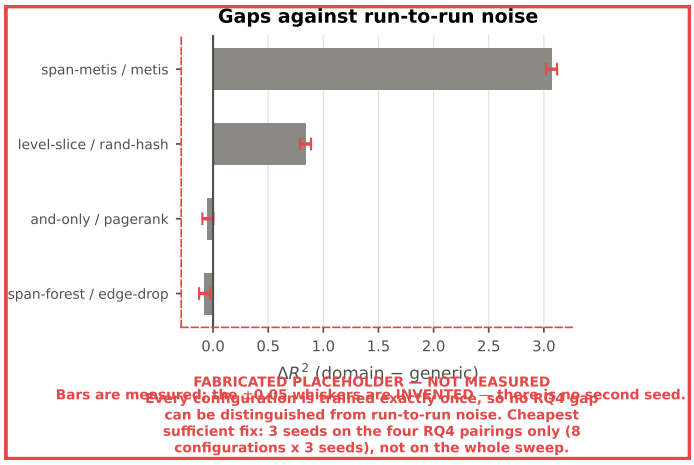

Figure 4.36: **ERROR BARS FABRICATED.** The measured RQ4 gaps with an invented ± 0.05 band. Every configuration in this thesis is trained exactly once, so there is no run-to-run variance to judge these gaps against. The figure exists to make that visible rather than to report an interval (4.4.2).

4.7.6 RQ5: Cross-Structural Generalization

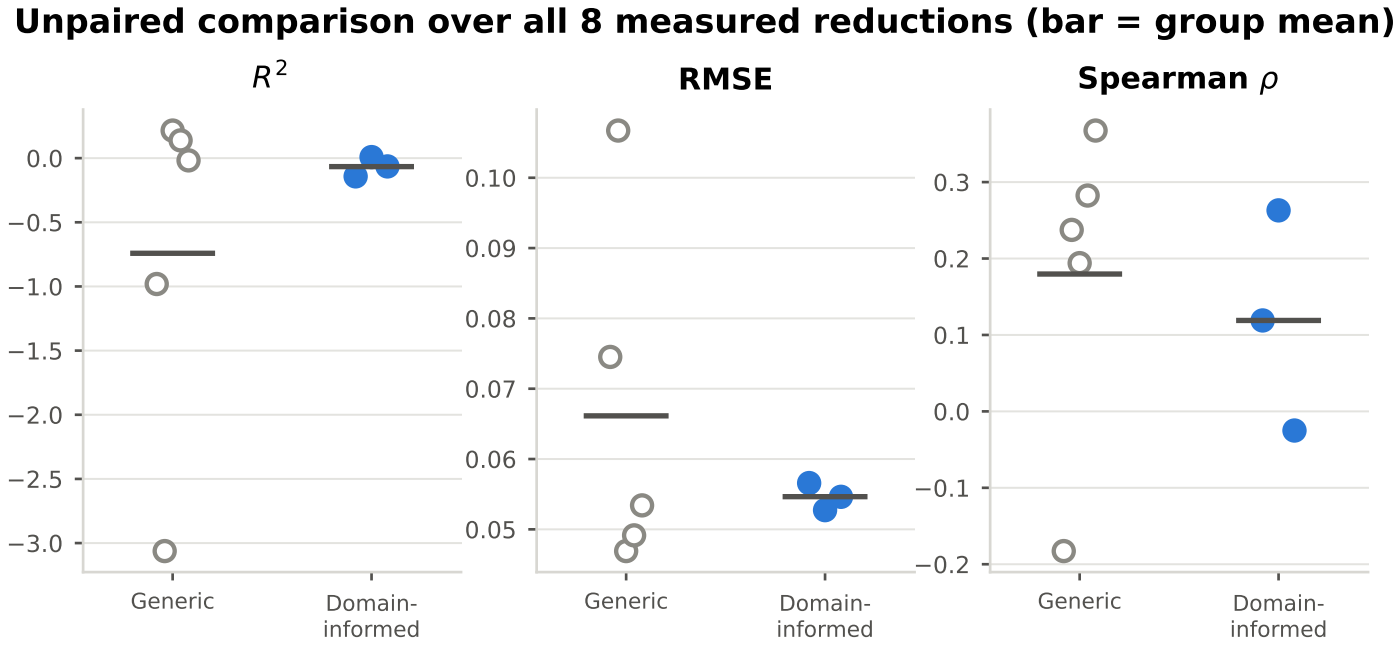

Figure 4.37: Every domain-informed method against every generic one on the same axes. Weaker evidence than the pairings, since the two groups did not remove the same amount, but it answers the question a reader asks first.

Table 4.13: RQ4 pairings. Δ is domain-informed minus generic, so a positive ΔR^2 favours the domain heuristic. The fourth pairing is NOT matched: random edge dropout at its configured rate keeps 69.7% of edges against spanning forest's 58.1%, so its gap conflates the heuristic with how much each arm removed.

Domain-informed	Generic	Matched	Edge ret. (dom.)	Edge ret. (gen.)	Δ RMSE	ΔR^2	Δ Spearman
Span-weighted METIS	METIS	by construction	0.923	0.937	-0.054	3.071	0.026
Level slicing	Random hashing	by construction	0.705	0.402	-0.018	0.839	0.302
AND-gate only	PageRank pruning	by calibration	0.733	0.627	0.001	-0.047	-0.393
Spanning forest	Random edge dropout	NOT MATCHED	0.581	0.697	0.002	-0.077	0.088

Each configuration is trained ONCE. These are point estimates with no run-to-run variance behind them, and RQ4's expected effect is small: a gap of a few percent cannot be distinguished from noise.

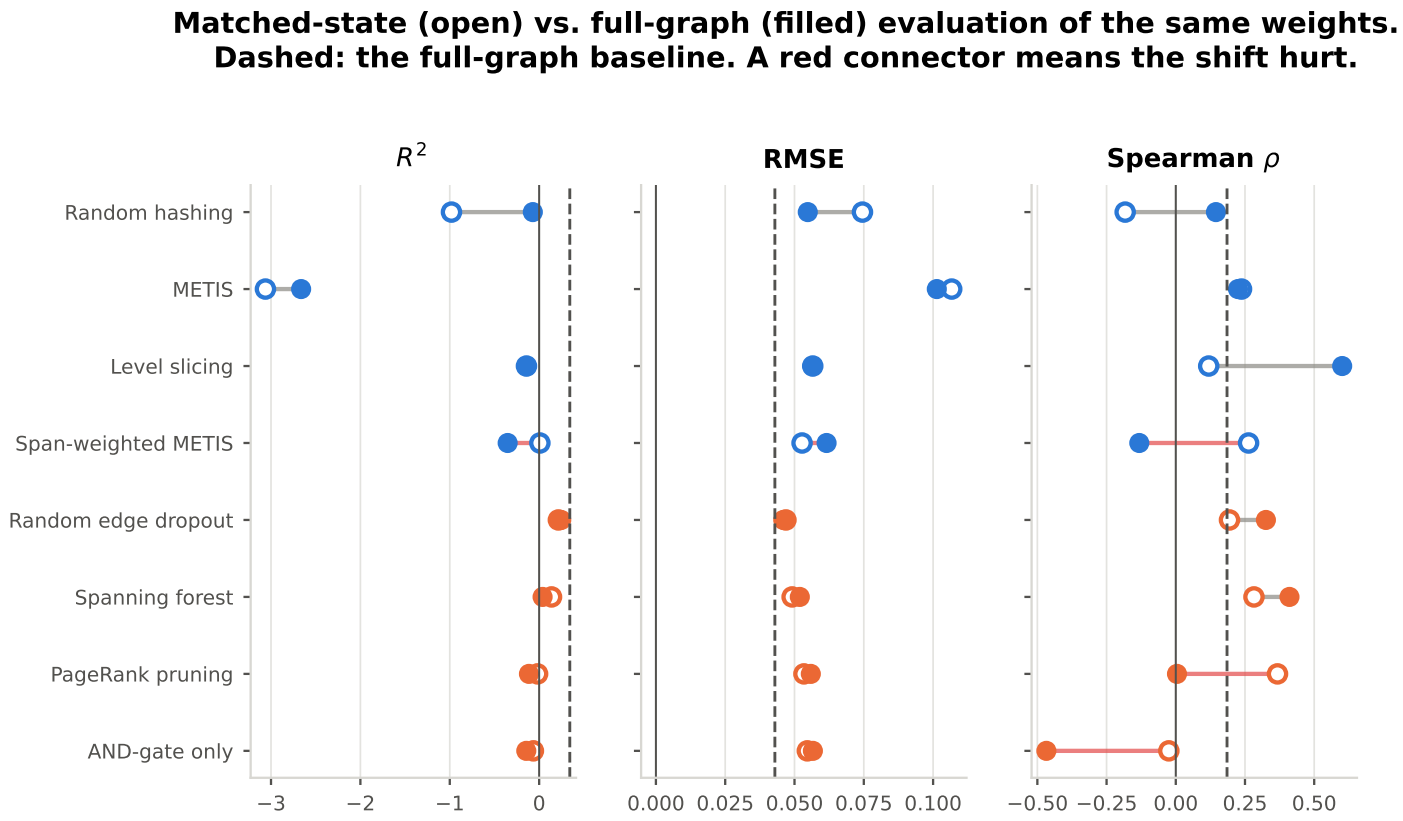

Figure 4.38: The same weights evaluated under the reduction they were trained with and on unreduced graphs. For every method here a full graph needs no conversion, since it already is a valid input, which is what makes this a direct test rather than a comparison across two input formats (3.2.3).

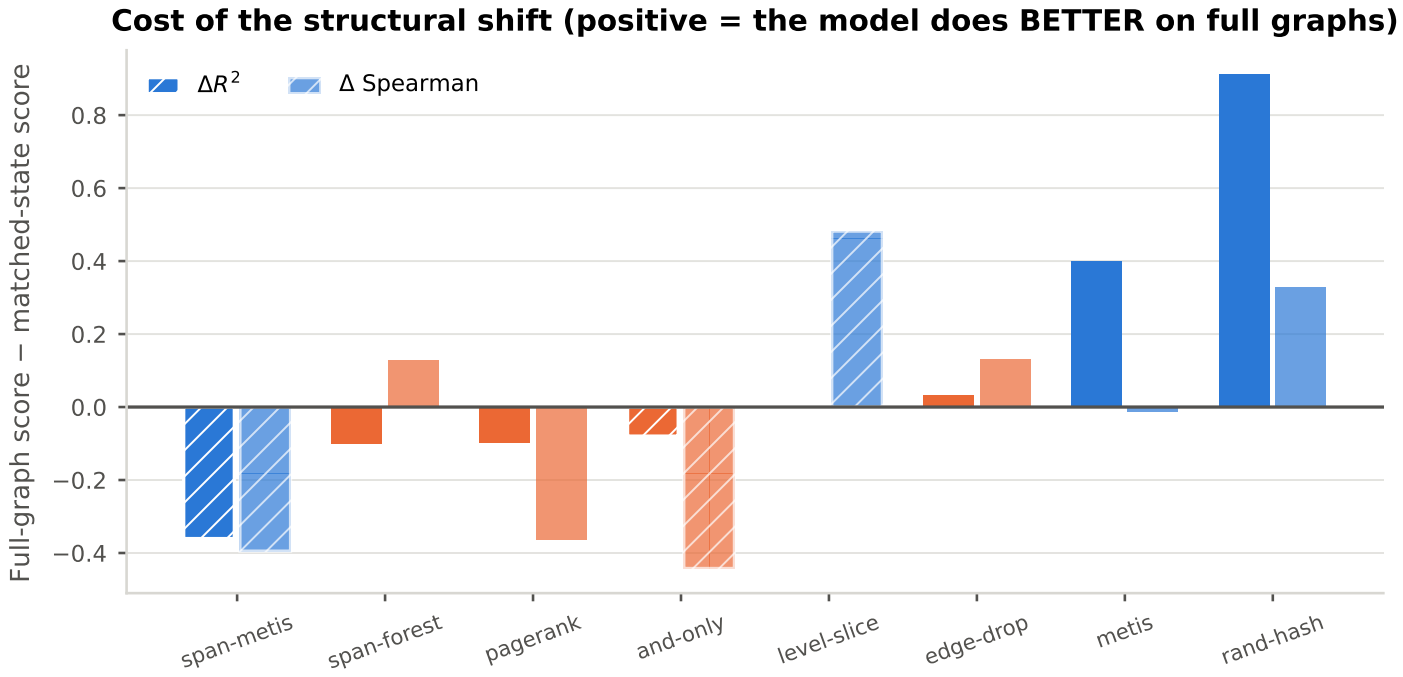

Figure 4.39: Full-graph score minus matched-state score, by method and family. A positive bar means the model scores *better* on graphs it never trained on, which would say the reduction was hurting the model rather than the evaluation.

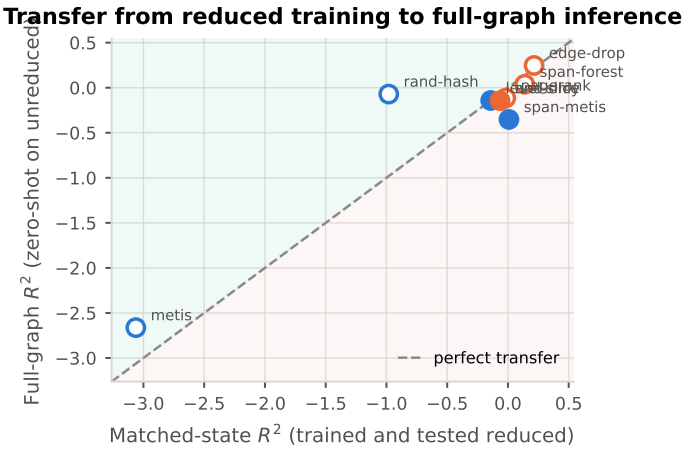

Figure 4.40: Matched-state against full-graph R^2. The diagonal is perfect transfer; below it the structural shift cost accuracy, above it the reduction was the thing doing damage.

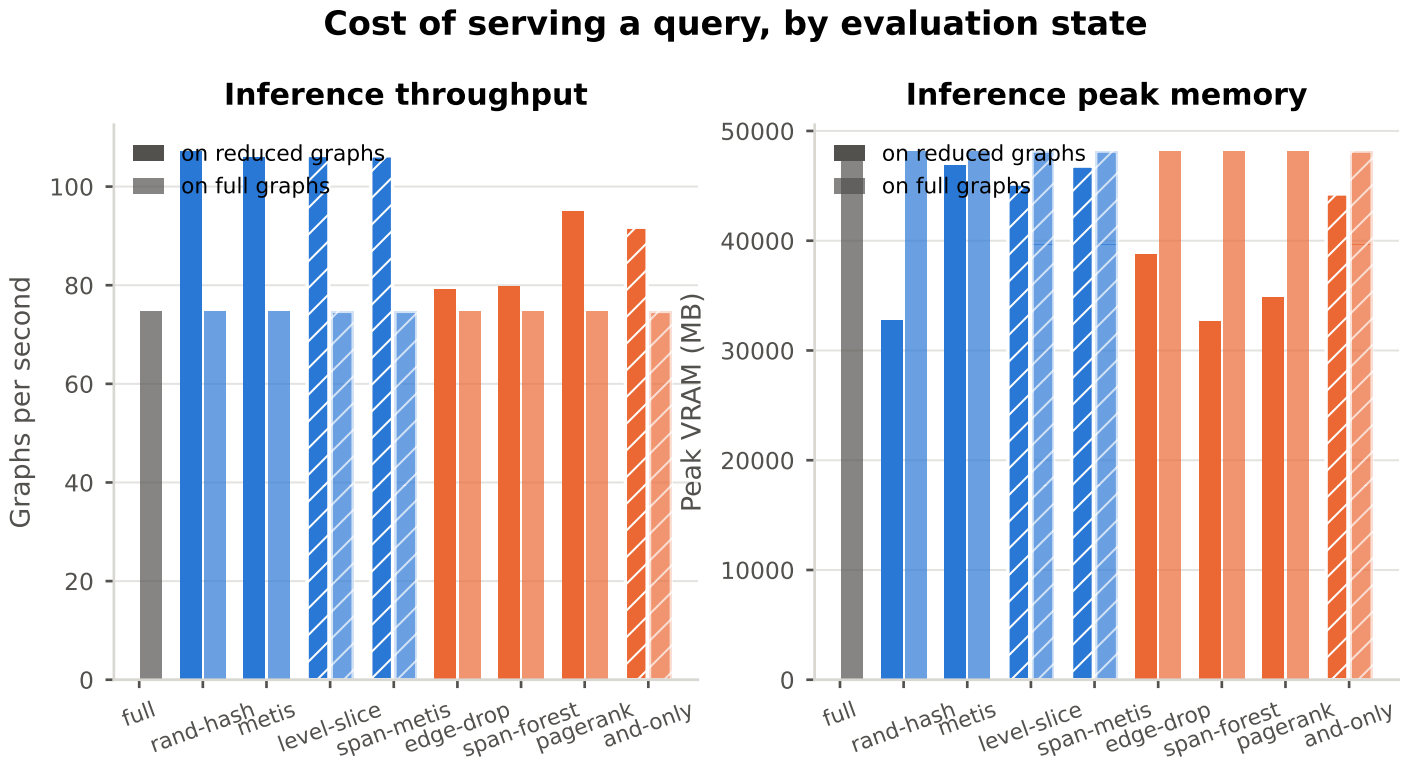

Figure 4.41: Throughput and peak memory for a query, under each evaluation state. Inference stores neither gradients nor activations, so the memory argument that forced the reduction at training time need not bind here at all, which is the practical claim of 1.2.5.

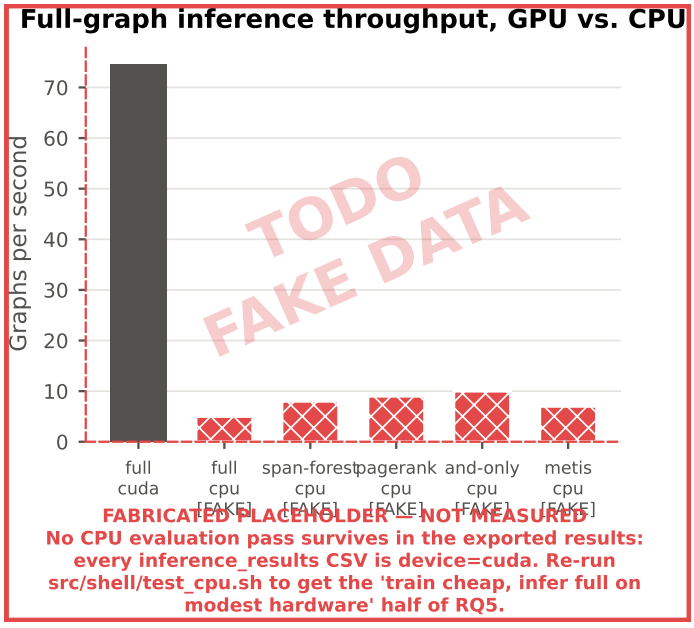

Figure 4.42: **MOSTLY FABRICATED.** The "infer full on modest hardware" half of RQ5. Every surviving inference CSV records `device=cuda`; the CPU bars are placeholders for a `src/shell/test_cpu.sh` pass that has not been run.

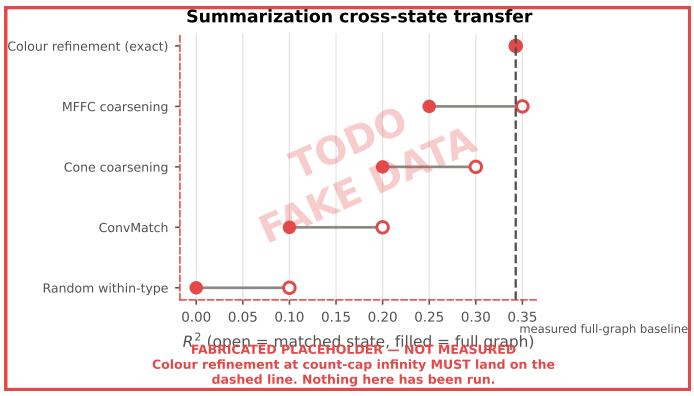

Figure 4.43: **ENTIRELY FABRICATED.** Colour refinement at count-cap ∞ merges only nodes the encoder provably cannot distinguish, so a model trained on those graphs *must* land on the dashed baseline when queried on full graphs. Without this control a poor transfer elsewhere cannot be told apart from a broken evaluation path, which is why the slot is held open rather than dropped.

4.7.7 Summarization and H1 (no data yet)

Table 4.14: RQ5: the same weights evaluated under the reduction they were trained with and on unreduced graphs. For every method here a full graph needs no conversion – it already is a valid input. The exact colour-refinement track is the exception and has not been run.

Method	Family	R^2 matched	R^2 full	ΔR^2	ρ matched	ρ full	Throughput full (g/s)
Full graph	Baseline	–	0.343	–	–	0.185	74.650
Random hashing	Partition	-0.981	-0.071	0.910	-0.183	0.145	74.669
METIS	Partition	-3.062	-2.663	0.399	0.237	0.225	74.668
Level slicing	Partition	-0.142	-0.142	0.000	0.119	0.601	74.690
Span-weighted METIS	Partition	0.008	-0.352	-0.360	0.263	-0.132	74.690
Random edge dropout	Sparsification	0.215	0.247	0.032	0.194	0.326	74.658
Spanning forest	Sparsification	0.138	0.038	-0.100	0.282	0.411	74.721
PageRank pruning	Sparsification	-0.018	-0.114	-0.097	0.367	0.004	74.721
AND-gate only	Sparsification	-0.064	-0.144	-0.080	-0.025	-0.467	74.684

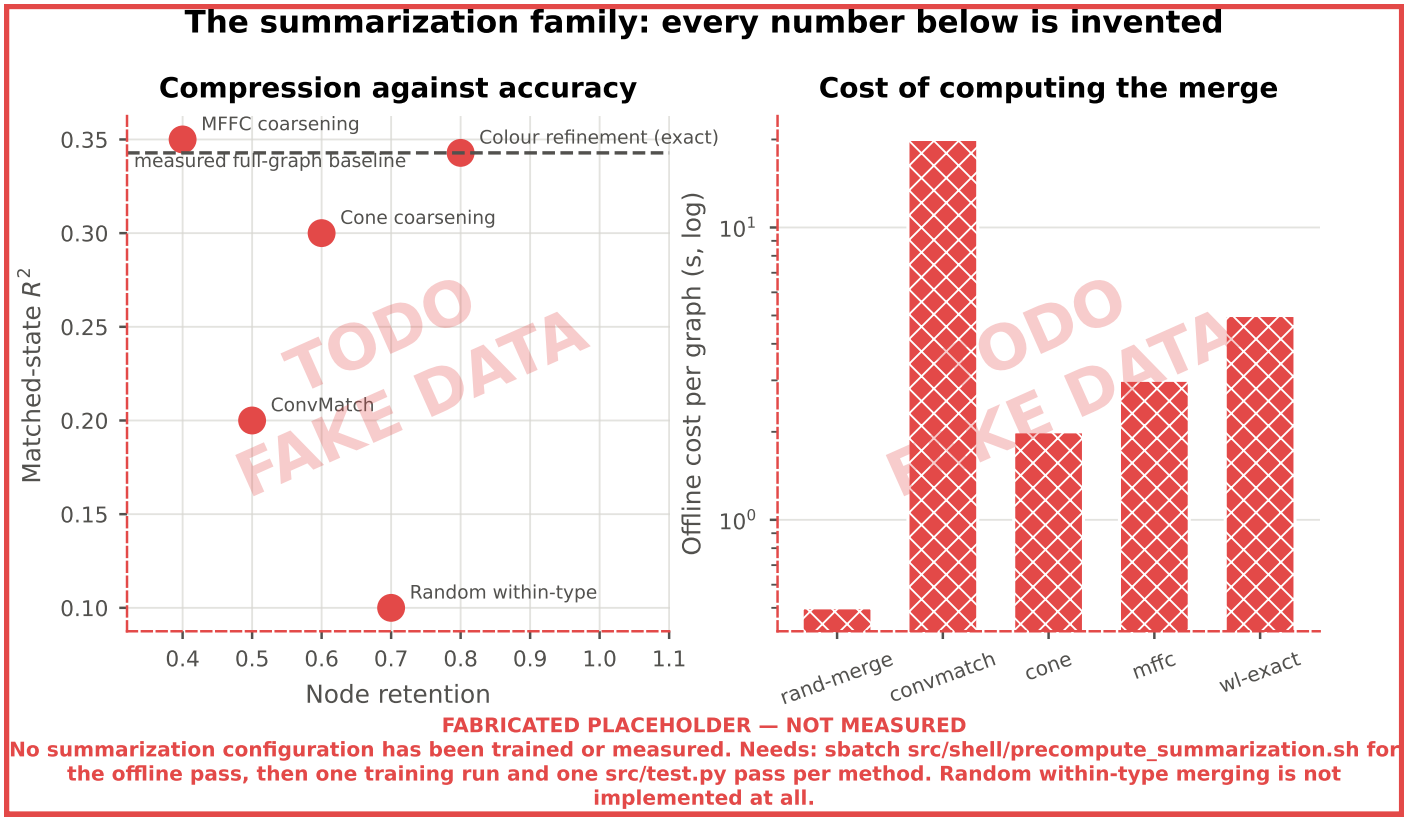

Figure 4.44: **ENTIRELY FABRICATED.** The summarization family in the compression/accuracy plane and its offline cost. No summarization configuration has been trained or measured, and random within-type merging, the instrument that makes RQ4's third pairing kind answerable at all (3.2.3), is not implemented.

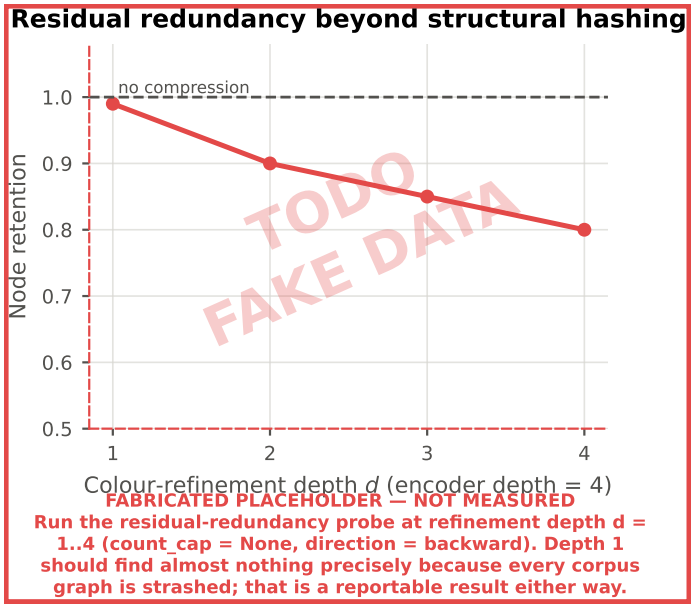

Figure 4.45: **ENTIRELY FABRICATED.** Node retention under colour refinement at depths $d = 1 \ldots 4$. Every corpus graph is structurally hashed, so depth 1 should find almost nothing; how much survives beyond one hop is a reportable statistic about AIGs in its own right and bounds what the exact track can compress.

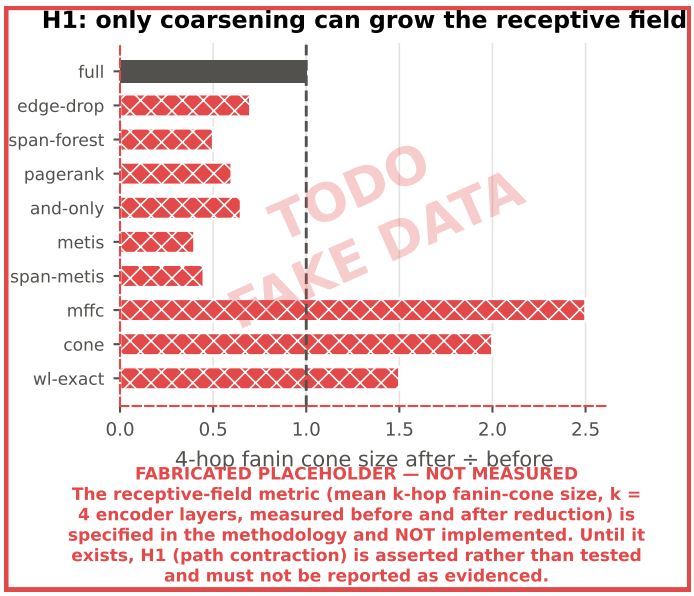

Figure 4.46: **ENTIRELY FABRICATED.** Mean 4-hop fanin-cone size after reduction relative to before. The metric is specified in 3.4.3 and *not implemented*; until it is, H1 is asserted rather than tested and must not be reported as evidenced.

4.7.8 Consolidated Summary

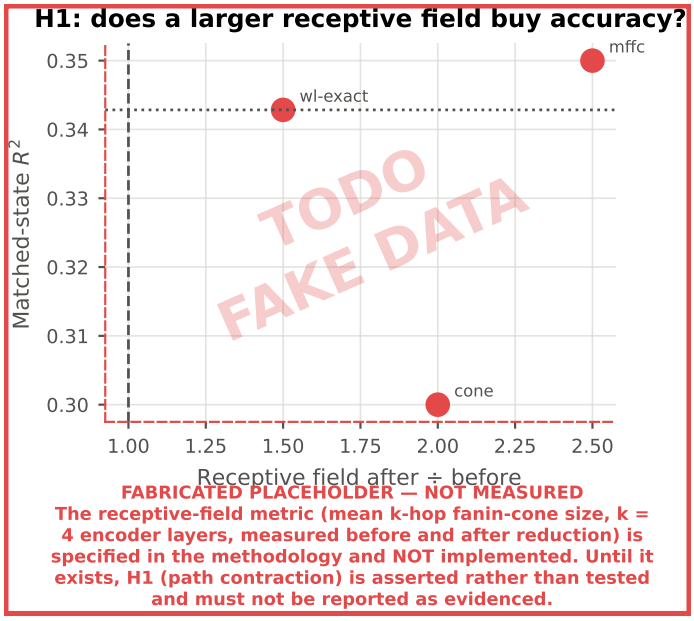

Figure 4.47: **ENTIRELY FABRICATED.** The form the H1 test would take: if coarsening helps by contracting paths, accuracy should rise with the receptive-field ratio. Both axes are invented.

Table 4.15: Every reduction method in one row: what it removes, what it costs offline, what it saves in training, what accuracy survives, and whether the model transfers to full graphs. Generic vs. domain-informed is a column, not a split, because it is one attribute of a method rather than the organising axis.

Method	Family	Kind	Node ret.	Edge ret.	Offline (s)	VRAM sav. (%)	Time sav. (%)	RMSE	R^2	ρ	R^2 cross-state
Full graph	Baseline	Generic	–	–	–	–	–	0.043	0.343	0.185	0.343
Random hashing	Partition	Generic	1.000	0.402	0.004	3.100	20.996	0.075	-0.981	-0.183	-0.071
METIS	Partition	Generic	1.000	0.937	3.196	0.279	24.284	0.107	-3.062	0.237	-2.663
Level slicing	Partition	Domain	1.000	0.705	1.151	1.487	18.291	0.057	-0.142	0.119	-0.142
Span-weighted METIS	Partition	Domain	1.000	0.923	3.701	0.357	22.412	0.053	0.008	0.263	-0.352
Random edge dropout	Sparsification	Generic	1.000	0.697	0.064	1.489	0.229	0.047	0.215	0.194	0.247
Spanning forest	Sparsification	Generic	1.000	0.581	3.309	2.142	-0.101	0.049	0.138	0.282	0.038
PageRank pruning	Sparsification	Generic	0.800	0.627	1.739	14.492	19.145	0.053	-0.018	0.367	-0.114
AND-gate only	Sparsification	Domain	0.821	0.733	0.099	11.257	28.893	0.055	-0.064	-0.025	-0.144
[TODO/FAKE] Random within-type	Summarization	Generic	0.700	0.700	0.500	–	–	0.060	0.100	0.100	0.000
[TODO/FAKE] ConvMatch	Summarization	Generic	0.500	0.600	20.000	–	–	0.050	0.200	0.200	0.100
[TODO/FAKE] Cone coarsening	Summarization	Domain	0.600	0.650	2.000	–	–	0.045	0.300	0.300	0.200
[TODO/FAKE] MFFC coarsening	Summarization	Domain	0.400	0.500	3.000	–	–	0.040	0.350	0.350	0.250
[TODO/FAKE] Colour refinement (exact)	Summarization	Domain	0.800	0.850	5.000	–	–	0.043	0.343	0.185	0.343

Rows marked [TODO/FAKE] are invented. No summarization configuration has been trained or measured. Needs: sbatch src/shell/precompute_summarization.sh for the offline pass, then one training run and one src/test.py pass per method. Random within-type merging is not implemented at all.

Chapter 5

Discussion

5.1 Comparison with Related Work

5.1.1 Positioning Against Optimizability Prediction Work

5.1.2 Positioning Against Graph Reduction Work

5.1.3 Where Direct Comparison Is Not Possible

5.2 Interpretation of Findings

5.2.1 Feasibility of Optimizability Prediction

5.2.2 What Reduction Buys and What It Costs

5.2.3 Why Domain-Informed Adaptation Did or Did Not Help

5.2.4 Structural Generalization and Deployment

5.2.5 Unexpected Results

5.3 Practical Implications

5.4 Limitations

5.4.1 Reproducibility

Every stochastic component of training is seeded, so an individual run repeats exactly. That is a weaker guarantee than it appears. Each configuration was trained once, so no run-to-run variance is available and every accuracy difference reported in Chapter 4 is a point estimate. This bears hardest on RQ4, where the expected effect is small: a gap of a few percentage points between a domain-informed method and its control cannot be distinguished from seed noise on the evidence collected here. The efficiency comparisons are better placed, being paired per graph with confidence intervals and a signed-rank test, but accuracy has no equivalent treatment.

Three further dependencies limit exact reproduction. Timing and memory measurements are specific to the cluster hardware and to the enabled compilation and mixed-precision settings, so they should be read as relative comparisons rather than as portable absolutes. The labels themselves depend on the version of the synthesis tool used to generate them; a different version would produce a different corpus, and the version was not pinned. The corpus is also not uniformly regenerable. One of the four generation scripts is stochastic, and its seed was drawn per graph and never recorded (A.1), so the quarter of the tier-1 set it produced could be reconstructed only in distribution, not exactly. Those graphs are training inputs rather than label sources, so the targets this thesis reports are unaffected, but the corpus itself is reproducible only in part.

5.4.2 Scalability

This is the limitation closest to the motivation and it should be stated without hedging. The argument for this work rests on industrial circuits of millions to billions of nodes. The evaluation runs on a corpus bounded at roughly a third of a million nodes per graph, because RQ1 requires a full-graph baseline and a baseline that does not fit in memory cannot be measured. The design that makes the study interpretable is the same design that keeps it away from the scale it argues about.

What does extrapolate: the relative ordering of reduction methods by offline cost, and the qualitative shape of the memory and throughput trade-offs, since these follow from properties of the algorithms and of message passing rather than from the corpus size. What does not: the absolute compression figures, since several methods' compression is a property of circuit structure rather than a parameter, and larger industrial circuits may have different redundancy profiles; and any claim that a method which retains accuracy here would retain it at a scale where the model sees a proportionally smaller fraction of each graph.

5.4.3 Generalizability

Three axes are held fixed, each narrowing what the conclusions cover.

One synthesis algorithm. Results characterise optimizability under `Orchestrate`. Graphs optimized by the other three scripts exist and are unused. If the reduction rankings hold across all four, the finding is about AIG structure; if they do not, it is about `Orchestrate`, and nothing here can tell the two apart.

One encoder. A four-layer edge-aware GCN+ was used throughout. Depth is not incidental to these results: methods were coupled to it by design, with refinement depth set equal to the number of layers. A deeper model would merge less under colour refinement and would have a different receptive-field profile, so the ranking could shift.

One target. Node reduction is the only quantity modelled. Delay and power are the objectives synthesis actually trades against, and nothing here speaks to them.

5.4.4 Reliability and Validity

Construct validity. Node reduction is a proxy for area, and area is one of three objectives. A circuit whose node count barely moves may still have been substantially improved in delay. The label measures what the tool removed, not how much better the

result is, and "optimizability" should be read in that narrower sense throughout.

Internal validity. Splitting by design was chosen to prevent a base graph and its own optimized descendants from straddling the train/test boundary. Whether it fully succeeds is testable rather than assumed, and the test is run as RQ1a (4.1.6): the gap between a design-level split and a random split measures directly how much of the score is design recognition rather than structural inference.

Construct validity of the comparison itself. Hyperparameters were tuned once on the full-graph configuration and held fixed. This protects the comparison from confounding retention with tuning effort, but it means reduced configurations run at settings chosen for a different input distribution. That biases against them, and therefore bounds how strongly any negative result about reduction can be stated.

5.4.5 Limitations of the Reduction Methods Tested

The reduction methods are not a representative sample of their literatures, and the summarization family in particular was scoped down deliberately: it runs one domain-specific method, one general GNN-aware method and one exact method, against a random control. The classical, guarantee-bearing spectral branch of the coarsening literature is not among them, so this thesis reports no trained result for it.

The domain-informed adaptations are simple. Edge-span weighting, level slicing, gate-role filtering and reconvergence-bounded merging are direct encodings of structural intuitions, not sophisticated methods. If they fail to beat their generic counterparts, that bounds these heuristics, not the idea of domain-aware reduction. That is a distinction 4.4.4 carries, and it should not be quietly dropped when the conclusion is written.

Finally, the boundary between structural compression and functional optimization is argued rather than measured. The claim that merging structurally equivalent nodes does not leak the label while merging functionally equivalent nodes would is well founded, but the negative control that would demonstrate it empirically, running FRAIG-style functional reduction [Mishchenko et al., 2005] as a summarizer and observing the label leak, was not run. The super-node content ablation is in the same position: member statistics are computed but not yet consumed by the encoder, so the finding in the literature that super-node content matters is cited rather than tested here.

5.5 Outlook

5.6 Ethical Considerations

Compute and energy cost. The work has a real environmental footprint: generating a corpus of several million AIGs required sustained CPU cluster time, and the training sweeps required substantial GPU time on top. The tension is worth stating rather than eliding: a study whose purpose is to reduce the compute cost of training spent a large amount of compute establishing how. The defence is that the cost is incurred once and the artifacts are stored and reusable, so a result showing which reductions preserve the signal saves compute for every subsequent user; but that is a claim about future savings against a certain present cost.

Benchmark provenance and licensing. The corpus is derived from published benchmark circuits, and their licences govern what may be redistributed. Every non-synthetic netlist was received through the OpenABC-D distribution, so the copies used are governed by its BSD 3-Clause terms, with the EPFL suite's MIT license applying in addition (3.3.1). A dataset release must therefore carry both attribution notices. The ISCAS-85 and MCNC circuits attach no formal terms; their inclusion rests on the same decades-long redistribution practice that OpenABC-D itself relies on.

Accessibility, and who benefits. Lowering the memory required to train circuit models cuts in two directions. It widens access, since research that currently requires large multi-device clusters could run on modest hardware, which matters for groups without industrial compute budgets. It also lowers the cost of capability for those who already have that budget. The honest position is that this work makes an existing capability cheaper rather than creating a new one, and that the widening effect is the more likely of the two to matter at this scale.

Dual use. Logic synthesis tooling is general-purpose infrastructure. Improvements to it apply to whatever circuits are being designed, and this work does nothing specific to any application domain. No mitigations are proposed because none are specific enough to be meaningful; the consideration is recorded because omitting it would be a worse answer than naming it.

Chapter 6

Conclusion

6.1 Answers to the Research Questions

6.1.1 RQ1: Task Feasibility and Baseline

6.1.2 RQ2: Reduction Efficiency

6.1.3 RQ3: Predictive Retention and Trade-offs

6.1.4 RQ4: Value of Domain-Informed Adaptation

6.1.5 RQ5: Reduced to Unreduced? Generalization

6.2 Contributions Revisited

6.3 Future Work

6.3.1 Extending Beyond a Single Synthesis Algorithm

6.3.2 Stronger Domain-Aware Reduction

6.3.3 Scaling to Industrial AIGs

6.3.4 From Prediction to Script Generation

Bibliography

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2019.

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations (ICLR)*, 2021. Oversquashing. The mechanism behind the hypothesis that path-contracting coarsening can improve rather than degrade accuracy on deep graphs.

Luca Amarú, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. The epfl combinational benchmark suite. In *Proceedings of the 24th International Workshop on Logic & Synthesis (IWLS)*, 2015. MIT licensed. Source of the eight arithmetic circuits in the corpus.

Surender Baswana and Sandeep Sen. A simple and linear time randomized algorithm for computing sparse spanners in weighted graphs. *Random Structures & Algorithms*, 30(4):532–563, 2007. doi: 10.1002/rsa.20130.

Berkeley Logic Synthesis and Verification Group. Abc: A system for sequential synthesis and verification. https://github.com/berkeley-abc/abc.

Per Bjesse and Arne Borälv. Dag-aware circuit compression for formal verification. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, pages 42–49. IEEE Computer Society / ACM, 2004. doi: 10.1109/ICCAD.2004.1382541. Origin of DAG-aware rewriting over precomputed equivalents.

Jeroen Bollen, Jasper Steegmans, Jan Van den Bussche, and Stijn Vansummeren. Learning graph neural networks using exact compression. In *Proceedings of the 6th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 8:1–8:9. ACM, 2023. doi: 10.1145/3594778.3594878. Collapses nodes in the same d-step colour-refinement class; provably identical GNN output. Graded variant cr_c with $c = \infty$ (colour refinement) and $c = 1$ (bisimulation).

Robert K. Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification (CAV)*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010. doi: 10.1007/978-3-642-14295-6_5. The citable record for the ABC system; the software itself is Berkeley Logic Synthesis and Verification Group.

Franc Brglez and Hideo Fujiwara. A neutral netlist of 10 combinational benchmark circuits and a target translator in fortran. In *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 677–692, 1985.

Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Computer Networks*, 30(1-7):107–117, 1998. doi: 10.1016/S0169-7552(98)00110-X.

Davide Buffelli, Pietro Lió, and Fabio Vandin. Sizeshiftreg: a regularization method for improving size-generalization in graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. Trains so embeddings stay consistent across coarsening ratios; the size-shift framing behind RQ5.

Jie Chen, Yousef Saad, and Zechen Zhang. Graph coarsening: From scientific computing to machine learning, 2022.

Animesh Basak Chowdhury, Benjamin Tan, Ramesh Karri, and Siddharth Garg. Openabc-d: A large-scale dataset for machine learning guided integrated circuit synthesis. *arXiv preprint arXiv:2110.11292*, 2021. SynthNet. Graph-level labels including percentage of nodes optimized – the closest published analogue of the optimizability label used here.

Animesh Basak Chowdhury, Shailja Thakur, Hammond Pearce, Ramesh Karri, and Siddharth Garg. Towards the imagenets of ml4eda. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023. doi: 10.1109/ICCAD57390.2023.10323663. Invited paper. Position piece on benchmark and dataset standards for ML4EDA, motivating the framing used here.

Giovanni De Micheli. *Synthesis and Optimization of Digital Circuits*. McGraw-Hill, 1994. Not indexed in DBLP; metadata taken from the publisher record.

Chenhui Deng, Zichao Yue, Cunxi Yu, Gokce Sarar, Ryan Carey, Rajeev Jain, and Zhiru Zhang. Less is more: Hop-wise graph attention for scalable and generalizable learning on circuits. In *Proceedings of the Design Automation Conference (DAC)*, 2024. HOGA. Hop-wise attention over precomputed features; no message passing at training time.

Charles Dickens, Eddie Huang, Aishwarya Reganti, Jiong Zhu, Karthik Subbian, and Lise Getoor. Convolution matching for efficient graph neural network training. In *Proceedings of the ACM Web Conference (WWW)*, 2024. ConvMatch / A-ConvMatch. Reference implementation: https://github.com/amazon-science/convolution-matching.

Vijay Prakash Dwivedi, Ladislav Rampásek, Michael Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. Long range graph benchmark. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.

Alessandro Generale, Till Blume, and Michael Cochez. Scaling r-gcn training with graph summarization, 2022. Trains on a summary graph and transfers weights back through a node-to-super-node mapping; ablation finds retaining super-node content critical.

Martin Grohe, Kristian Kersting, Martin Mladenov, and Erkal Selman. Dimension reduction via colour refinement. In *Algorithms – ESA 2014, 22th Annual European Symposium*, volume 8737 of *Lecture Notes in Computer Science*, pages 505–516. Springer, 2014. doi: 10.1007/978-3-662-44777-2_42. Colour refinement computes the coarsest equitable partition – the formal basis for losslessness under a permutation-equivariant GNN.

William L. Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1024–1034, 2017.

Mohammad Hashemi, Shengbo Gong, Juntong Ni, Wenqi Fan, B. Aditya Prakash, and Wei Jin. A comprehensive survey on graph reduction: Sparsification, coarsening, and condensation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 8058–8066, 2024.

Abdelrahman Hosny, Soheil Hashemi, Mohamed Shalan, and Sherief Reda. Drills: Deep reinforcement learning for logic synthesis. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2020. doi: 10.1109/ASP-DAC47756.2020.9045559. Representative learned-synthesis-script-generation work: an RL agent selects among the same primitive transformations at each step.

Zengfeng Huang, Shengzhong Zhang, Chong Xi, Tang Liu, and Min Zhou. Scaling up graph neural networks via graph coarsening. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2021. SCAL. Trains on the coarsened graph and infers directly with the same weights – the shared-weights precedent for RQ5.

Wenjing Jiang, Jin Yan, and Sachin S. Sapatnekar. Ml-based aig timing prediction to enhance logic optimization. In *Design, Automation and Test in Europe Conference (DATE)*, 2025. doi: 10.23919/DATE64628.2025.10993071. XGBoost AIG-timing predictor; one of the four surveyed papers reporting matched seen/unseen-design results.

Wei Jin, Lingxiao Zhao, Shichang Zhang, Yozen Liu, Jiliang Tang, and Neil Shah. Graph condensation for graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2022. GCond. Excluded here: no node correspondence, label-dependent, architecture-tied.

Raika Karimi, Faezeh Faez, Yingxue Zhang, Xing Li, Lei Chen, Mingxuan Yuan, and Mahdi Biparva. Logic synthesis optimization with predictive self-supervision via causal transformers. *arXiv preprint arXiv:2409.10653*, 2024. doi: 10.48550/arXiv.2409.10653. LSOformer. One of the four surveyed papers reporting matched seen/unseen-design QoR results.

George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. doi: 10.1137/S1064827595287997.

Barsat Khadka, Kawsher A. Roxy, and Md Rubel Ahmed. Cts-bench: Benchmarking graph coarsening trade-offs for gnns in clock tree synthesis. *arXiv preprint arXiv:2602.19330*, 2026. doi: 10.48550/arXiv.2602.19330. Nearest competitor. Post-placement gate-level netlists, clock-skew prediction; one bespoke spatial clustering heuristic. 17.2x memory / 3x speed at negative zero-shot R^2; calls for domain-aware coarsening. DBLP: journals/corr/abs-2602-19330, preprint only, no published venue as of verification.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956. doi: 10.1090/S0002-9939-1956-0078686-7. Not indexed in DBLP; metadata taken from the publisher record.

Andreas Kuehlmann, Viresh Paruthi, Florian Krohm, and Malay K. Ganai. Robust boolean reasoning for equivalence checking and functional property verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21(12):1377–1394, 2002. doi: 10.1109/TCAD.2002.804386. Structural hashing (one-level lookup on construction) for AIGs.

Amy Nicole Langville and Carl Dean Meyer. Deeper inside pagerank. *Internet Mathematics*, 1(3):335–380, 2003. doi: 10.1080/15427951.2004.10129091.

Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems*, 1(1):121–141, 1979. doi: 10.1145/357062.357071.

Min Li, Sadaf Khan, Zhengyuan Shi, Naixing Wang, Huang Yu, and Qiang Xu. Deepgate: Learning neural representations of logic gates. In *Proceedings of the Design Automation Conference (DAC)*, 2022.

Yingjie Li, Mingju Liu, Haoxing Ren, Alan Mishchenko, and Cunxi Yu. Dag-aware synthesis orchestration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(12):4666–4675, 2024. doi: 10.1109/TCAD.2024.3397052. The orchestration policy behind the target algorithm: per-node selection among rewrite, refactor and resubstitution instead of a fixed command sequence.

Jiawei Liu, Jianwang Zhai, Mingyu Zhao, Zhe Lin, Bei Yu, and Chuan Shi. Polargate: Breaking the functionality representation bottleneck of and-inverter graph neural network. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, 2024. doi: 10.1145/3676536.3676834. Polarity/functionality bottleneck in AIG GNNs – supports treating inverter polarity as a first-class relation.

Andreas Loukas. Graph reduction with spectral and cut guarantees. *Journal of Machine Learning Research*, 20:116:1–116:42, 2019. Local Variation coarsening; restricted spectral approximation guarantee.

Yuankai Luo, Lei Shi, and Xiao-Ming Wu. Can classic gnns be strong baselines for graph-level tasks? simple architectures meet excellence. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 267

of *PMLR*, 2025. GNN+ – the encoder this work builds on. arXiv:2502.09263. Implementation: https://github.com/LUOyk1999/GNNPlus.

Alan Mishchenko and Robert K. Brayton. Scalable logic synthesis using a simple circuit structure. In *Proceedings of the 15th International Workshop on Logic & Synthesis (IWLS)*, pages 15–22, 2006. Not indexed in DBLP; metadata taken from the paper. Source of the resubstitution command and the compress2rs script that C2RS aliases.

Alan Mishchenko, Satrajit Chatterjee, Roland Jiang, and Robert K. Brayton. Fraigs: A unifying representation for logic synthesis and verification, 2005. ERL Technical Report, UC Berkeley. Functionally-reduced AIGs via SAT sweeping – the functional end of the equivalence spectrum, used here only as a leakage control.

Alan Mishchenko, Satrajit Chatterjee, and Robert K. Brayton. Dag-aware aig rewriting: A fresh look at combinational logic synthesis. In *Proceedings of the 43rd Design Automation Conference (DAC)*, pages 532–535. ACM, 2006. doi: 10.1145/1146909.1147048. Defines the rewrite/refactor primitives and the MFFC as the unit refactoring operates on – the basis for the S6 domain argument.

Liwei Ni, Rui Wang, Miao Liu, Xingyu Meng, Xiaoze Lin, Junfeng Liu, Guojie Luo, Zhufei Chu, Weikang Qian, Xiaoyan Yang, Biwei Xie, Xingquan Li, and Huawei Li. Openlsdgf: An adaptive open-source dataset generation framework for machine-learning tasks in logic synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(10):3830–3843, 2025. doi: 10.1109/TCAD.2025.3555506. Adaptive open-source dataset generation framework for ML4EDA, positioned against the dataset built here.

David Peleg and Alejandro A. Schäffer. Graph spanners. *Journal of Graph Theory*, 13(1):99–116, 1989. doi: 10.1002/jgt.3190130114.

Ladislav Rampásek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.12454.

Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. In *International Conference on Learning Representations (ICLR)*, 2020.

Nasrin Shabani, Jia Wu, Amin Beheshti, Quan Z. Sheng, Jin Foo, Venus Haghighi, Ambreen Hanif, and Maryam Shahabikargar. A comprehensive survey on graph summarization with graph neural networks. *arXiv preprint*, 2023.

Zhengyuan Shi and TODO. Deepgate3: Towards scalable circuit representation learning, 2024. Pooling transformer over subcircuits. Scales the model, not the input.

Mathias Soeken, Heinz Riener, Winston Haaswijk, Eleonora Testa, Bruno Schmitt, Giulia Meuli, Fereshte Mozafari, Siang-Yun Lee, Alessandro Tempia Calvino, Dewmini Sudara Marakkalage, and Giovanni De Micheli. The epfl logic synthesis libraries, 2018. arXiv:1805.05121. mockturtle/kitty/lorina. Defines the random AIG sampling procedure behind the eight synthetic designs.

Mahito Sugiyama and Kazuki Sato. Kron reduction for directed graphs, 2023.

Veronika Thost and Jie Chen. Directed acyclic graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.

Yuanyuan Tian, Richard A. Hankins, and Jignesh M. Patel. Efficient aggregation for graph summarization. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2008. k-SNAP. Subsumed here by graded colour refinement at $d = 1$; cited, not run.

TODO. Rethinking reinforcement learning based logic synthesis. *arXiv preprint arXiv:2207.11437*, 2022. Joint Transformer (recipe) + GraphSAGE (circuit) QoR prediction.

TODO. Universal graph coarsening, 2024. LSH-based, linear-time coarsening. AH-UGC is the 2025 consistent-hashing follow-up.

TODO. Deepgate4: Efficient and effective representation learning for circuit design at scale, 2025. Sparse GAT transformer, sub-linear memory, virtual edges over k-hop cones; -84% inference time vs DeepGate3. Baseline model in this work.

Marcel Walter. aigverse: A python library for working with logic networks, synthesis, and optimization, 2025. MIT licensed. https://github.com/marcelwa/aigverse. Used by the ML pipeline to read and traverse AIGER files; pyproject.toml pins >=0.0.27.

Nan Wu, Jiwon Lee, Yuan Xie, and Cong Hao. Lostin: Logic optimization via spatio-temporal information with hybrid graph models, 2022. Encodes the synthesis recipe as a super-node – a temporal use of super-nodes, orthogonal to the structural super-nodes here.

Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Kenichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

Saeyang Yang. Logic synthesis and optimization benchmarks user guide: Version 3.0. Technical report, Microelectronics Center of North Carolina (MCNC), 1991.

Muhan Zhang, Shali Jiang, Zhicheng Cui, Roman Garnett, and Yixin Chen. D-vae: A variational autoencoder for directed acyclic graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Da Zheng, Chao Ma, Minjie Wang, Jinjing Zhou, Qidong Su, Xiang Song, Quan Gan, Zheng Zhang, and George Karypis. Distdgl: Distributed graph neural network training for billion-scale graphs. In *IEEE/ACM Workshop on Irregular Applications: Architectures and Algorithms (IA3)*, 2020. doi:

10.1109/IA351965.2020.00011. Graph partitioning across devices for distributed GNN training – the setting in which cutting edges is forced rather than chosen.

Appendix A

First Appendix

A.1 Synthesis Algorithm Configurations

This section records the concrete invocations behind 3.3.2 and 2.1.3. Every entry below is transcribed from the generation pipeline itself rather than from its documentation; where the two disagree, the pipeline is authoritative and the disagreement is noted.

Transformation recipes

Each recipe is 20 commands drawn independently and uniformly from `balance`, `rewrite`, `rewrite -z`, `refactor`, `refactor -z`, `resub` and `resub -z`, where `-z` enables zero-cost replacements. The 200 recipes used are the first 200 of the 1,500 released with OpenABC-D [Chowdhury et al., 2021] and are applied unmodified. The generator strips the input, output and hashing commands from each released recipe and appends a `write_aiger` after every surviving command, which is what makes each recipe prefix a stored graph.

Each released recipe continues past the 20 transformations into `dch` and then a technology-mapping tail, `map -B 0.9`, `topo`, `stime -c`, `buffer -c`, `upsize -c` and `dnsize -c`. The stripping rule does not remove that tail, so 27 write commands are emitted per recipe, but no standard-cell library is loaded and the mapped network cannot be written as an AIG, so only the 21 graphs up to and including `dch` are produced. The count assertion in the generation job, exactly 4,201 AIGs per design, is what establishes that the remaining six writes produced nothing.

Optimization invocations

Each algorithm is applied through the template in Table A.1, with the tool configuration file sourced first in every case.

Table A.1: Optimization invocations, one per algorithm, as issued by the generation pipeline.

Algorithm	Command
`Orchestrate`	`strash; orchestrate`
`Deepsyn`	`strash; &get; &deepsyn -S <seed> -T 20; &put`
`Syn4`	`strash; &get; &syn4; &put`
`C2RS`	`strash; c2rs`

Three of the four are invoked with no parameters at all, so their behaviour is the tool's own default in each case. The pass budgets and scoring metrics named in the dataset documentation are not passed on the command line and were not set by this work. `C2RS` is an alias expanding to `b -l; rs -K 6 -l; rw -l; rs -K 6 -N 2 -l; rf -l; rs -K 8 -l; b -l; rs -K 8 -N 2 -l; rw -l; rs -K 10 -l; rwz -l; rs -K 10 -N 2 -l; b -l; rs -K 12 -l; rfz -l; rs -K 12 -N 2 -l; rwz -l; b -l`, which is the source of the widening cut size reported in 2.1.3.

`Deepsyn` is the one stochastic algorithm. Its seed is drawn from the shell random number generator once per input graph, is interpolated into the command, and is written neither to the run log nor to the metadata tables, so those graphs cannot be regenerated (5.4.1).

A.2 Reduction Method Parameters

A.3 Baseline Hyperparameters

A.4 Experimental Configuration

A.5 Extended Results